

Project P: Your Pet on Android

Step-by-step guide: Godot game + home/lock screen pet (Live Wallpaper), together in one app

Read this first

Godot can't put anything on your home screen or lock screen. A Godot export is a normal app, and it only shows while that app is open. Android does have something that behaves exactly the way you want: a **Live Wallpaper**. It shows on the home screen and lock screen, hides automatically when you open an app, and can respond to touch. Live wallpapers have to be written in Android's own language (Kotlin), and Godot can't render inside one.

So the setup has two pieces that use the same pet frames, and a third step that joins them:

- **Part A - Godot app:** the actual Project P game, in portrait. It opens on a main menu with your animated background, a Pet Selection button (a grid of pets with splash art) and a Wallpaper Selection button. The pet can be tapped, dragged and resized, and moves around by itself. Exported as an APK.
- **Part B - Live wallpaper as its own app (optional):** a tiny Android Studio app that puts the pet on your home and lock screen, on top of the Project P wallpaper picture or a picture you choose (such as your current wallpaper).
- **Part C - One app:** the wallpaper packaged inside the Godot game, so there is a single APK with a single app icon to install and publish. The wallpaper setup opens from the **Wallpaper Selection** button in the game's main menu. You can do Part C without doing Part B.

These steps assume Godot 4.3 or newer and each pet's frames saved as separate PNG files (for example 128x128 pixels each).

Files that come with this guide

- `pet.gd` - the pet in the Godot game (Part A).
- `pet_gd_edits_2.txt` - this update's changes to `pet.gd` (one trick at a time, low energy, sleep) as eleven find-and-replace edits, for a `pet.gd` you have edited yourself. It starts from a `pet.gd` that already has the fourteen earlier edits in `pet_gd_edits.txt`.
- `pet_picker.gd` - the pet selection grid with splash art (step A5).
- `main_menu.gd` - the main menu the app opens on: animated background, Pet Selection and Wallpaper Selection buttons (step A6).
- `ProjectP_Menu_Assets.zip` - sample menu art with the right names and sizes: 16 background frames and 3-frame buttons (step A6).
- `ProjectP_Godot_Starter.zip` - a ready-made Godot project with test frames (Part A shortcut).
- `ProjectP_Wallpaper_Addon.zip` - the Godot add-on that packages the wallpaper into the game (Part C). It contains `default_wallpaper.png`, the Project P wallpaper picture (the living room), the folder `default_wallpaper_frames/` for its animation frames (step C10), and in `addons/projectp_wallpaper/android/src/` the wallpaper code: `PetCatalog.kt`, `PetWallpaperService.kt`, `SetWallpaperActivity.kt` and `WallpaperBridge.kt`. Part B uses the first three.
- `pet_default_bg.png` - the same picture under the name the wallpaper-only app needs (Part B).

Your pet's frames and how the pet behaves

Each pet has its own folder with its frames as separate PNG files, named the way the sprite-sheet-slicer exports them. For the dragon:

- **egg_idle_00 ... egg_idle_07** - egg_idle (loops)
- **egg_hatch_emerge_00 ... egg_hatch_emerge_07** - egg_hatch (plays once): the egg hatching and the pet coming out, as one animation. Older sets with two animations also work: **egg_hatch_00 ... _07** followed by **dragon_emerge_00 ... _07** (pet_emerge).
- **idle_a_00 ... idle_d_07** - pet_idle (32 frames, loops): 4 sets of 8 frames, one eye blink each. Rows a, b, c, d are played in order. The older names **dragon_idle_00 ... _07** also work.
- **jump_land_a_00 ... _07** - jump, and **jump_land_b_00 ... _07** - landing (play once)
- **ticklish_recovery_a_00 ... _07** - ticklish, and **ticklish_recovery_b_00 ... _07** - recovery (play once)
- **move_left_00 ... move_left_07** and **move_right_00 ... move_right_07** - move_left and move_right (loop while the pet travels): the left set is used when it goes left, the right set when it goes right. A pet with only one of the two uses it mirrored for the other direction.
- **fly_a_00 ... fly_d_07** - move: one travelling set for both directions, mirrored as needed, for pets without separate left and right frames. Pets that don't fly use **walk_**, **run_** or **hop_** instead.
- **trick_a_00 ... trick_b_07** - trick (plays once): the pet's tricks. **Each row of 8 frames is one trick of its own:** **trick_a_00 ... _07** is the first, **trick_b_00 ... _07** the second. When the pet does a trick it picks one of them, plays only that one, and goes back to blinking. More rows (**trick_c_**, **trick_d_**) join the choice by themselves.
- **low_energy_a_00 ... low_energy_a_07** (or **low_energy_00 ... _07**) - low_energy (plays once): what the pet does on a low battery. As with the tricks, each row of 8 frames is one version and one is played at a time. The names **low_battery_** and **tired_** also work.
- **sleep_00 ... sleep_07** - sleep (loops): the pet's resting animation at night. The names **sleeping_** and **asleep_** also work.

The pet's name in front is allowed, so other pets work the same way (**cat_emerge_00**, **cat_idle_00**, ...). Files that are only numbered (**01.png ... 64.png**) also work: they're handed out in the order of **ANIMS**. All frames should be the same size. Don't keep two sets for the same action in one folder (for example both **dragon_idle_** and **idle_a_**), or they are mixed into one animation.

How the pet behaves in the Godot game:

- **Egg:** the egg wobbles in place. Tap it 3 times and it plays egg_hatch (and then pet_emerge, if the pet has separate emerge frames), then becomes the pet. The pet stays hatched after that.
- **Tap the hatched pet:** ticklish, then recovery, then back to pet_idle.
- **Drag the hatched pet:** jump plays when you pick it up and holds its last frame while you carry it. When you let go, landing plays, then pet_idle. Dragging the egg just moves it.
- **Resize:** on a phone, put one finger on the pet, add a second finger and spread or pinch them. On PC, scroll the mouse wheel over the pet. The size is saved per pet.
- **By itself:** most of the time the hatched pet rests and blinks. Every 3-7 seconds, between two blinks, it picks one thing at random from the **ACTIONS** list and does it, then goes back to blinking: it keeps blinking a while longer, travels to a random spot on the left or right with its move_left or move_right frames (flying pets with **fly_** frames bob in the air), hops on the spot (jump, then landing), or plays **one** of its tricks. An action the pet has no frames for is skipped. Grabbing the pet stops it.

- **Low battery:** while the phone's battery is at 15% or less and the phone isn't charging, the pet picks from the `LOW_ENERGY_ACTIONS` list instead: it does one of its low-energy movements, or keeps blinking, and doesn't travel, hop or do tricks. Plug the phone in and it's its old self.
- **Night:** from 11 PM to 6 AM by the phone's clock the pet plays its sleep frames instead of blinking and does nothing by itself. **Tap it to wake it up.** It is then awake for 5 minutes (every further tap starts the 5 minutes again), and after that it goes back to sleep. In the morning it wakes up by itself.

On the home screen and lock screen (the wallpaper) the pet has the egg, hatch and tickle behaviour, and it does the same things by itself as in the game, from the same `ACTIONS` list. It is moved with two taps instead of a drag: tap the pet, then tap the spot where it should go (step C7). On the lock screen, where Android doesn't pass touches to wallpapers, it greets you when the screen turns on and reacts to shaking and tilting the phone (step C8). It is as big as you made it in the game, gets tired on a low battery and sleeps at night in the same way (step C12).

If your files use different names, add the name (without the frame number) to that animation's list in `ANIMS` in `pet.gd`. With the one-app setup (Part C) the wallpaper takes `ANIMS` and `ACTIONS` from `pet.gd` on every export. Only the wallpaper-only app of Part B has its own copy in `PetCatalog.kt`.

More pets

The code supports any number of pets. Every pet needs its own folder of frames, named like the dragon's above. Each pet keeps its own progress (hatched, size, position).

To add a pet, change these places (each is marked **[ADD PET]** in the code):

- **Godot game - `pet.gd`, SECTION 0:** put its frames in their own folder (for example `res://pets/cat/`) and add one line to `PETS`, for example `"cat": "res://pets/cat",`. On the pet selection grid from step A5, players choose their pet by its splash art; the grid grows by itself. Add a splash picture named after the new pet's id to `res://menu/splash/`.
- **Wallpaper inside the game (Part C):** nothing to do. The add-on reads `PETS` from `pet.gd` on every export and packs each pet's frames for the wallpaper.
- **Wallpaper as its own app (Part B):** add one line to `PETS` in `PetCatalog.kt`, for example `PetType("cat", "Cat", "pets/cat")`, with the same id as in `pet.gd`, and copy the frames into `app/src/main/assets/pets/cat/`.

Reading the code later

Every script in this guide starts with a header that says what the file does and lists its sections. Inside the code, five labels mark the places you're most likely to come back to:

- **[EDIT]** - settings you can safely change: sizes, speeds, frame counts, taps to hatch, button text.
- **[LOGIC]** - the rules for how the pet behaves, such as what plays after what.
- **[FIX]** - the first places to look when something goes wrong.
- **[ADD PET]** - the places to change when you add another pet.
- **[ADD ACTION]** - the place to add another movement the pet does by itself.

Use your editor's search (**Ctrl+F**) for a label or for "SECTION 3" to jump straight to that part.

Part A - The Godot app

Shortcut: `ProjectP_Godot_Starter.zip` is the finished result of steps A1-A4, tested in Godot 4.3 and 4.5. Unzip it, open it in Godot (**Import** > pick its `project.godot`), and press **F5**: a test egg with numbered frames appears. Replace the PNGs in its `pets/dragon` folder with your dragon's frames to see your pet. Compare it with your own project if something doesn't work.

A1. Project settings (orientation, pixel art, touch)

- 1 Open your project and go to **Project > Project Settings**. Turn on **Advanced Settings** (the toggle at the top right).
- 2 Go to **Display > Window** and set: **Viewport Width** `720`, **Viewport Height** `1280`, **Stretch > Mode** `canvas_items`, **Stretch > Aspect** `expand`, **Handheld > Orientation** `Portrait` (this makes it vertical).
- 3 Go to **Rendering > Textures > Canvas Textures > Default Texture Filter** and set it to `Nearest`. This keeps the pixel art sharp instead of blurry.
- 4 Go to **Rendering > Renderer > Rendering Method** and set it to `gl_compatibility`. This works on the most Android phones.
- 5 Go to **Input Devices > Pointing > Emulate Touch From Mouse** and turn it **On**, so you can test dragging with your mouse on PC.
- 6 Close the window.

A2. Import the pet's frames

- 1 In the **FileSystem** panel at the bottom left, right-click `res://`, choose **New > Folder**, and name it `pets`. Right-click `pets`, choose **New > Folder** again, and name it `dragon`.
- 2 Drag your dragon's frame PNGs (`egg_idle_00.png`, `idle_a_00.png`, `fly_a_00.png`, ...) from your computer into the `dragon` folder. A subfolder such as `dragon/frames/` is fine too.
- 3 The script in step A4 builds all the animations from this folder by itself, so no other animation file is needed. (If you added `pet_sheet.png` or `pet_frames.tres` from an earlier version, you can delete them.)
- 4 The Nearest filter from step A1 is applied automatically.

Check: the FileSystem panel shows `res://pets/dragon/` with your PNGs inside.

Using your own folder names? That's fine, for example `res://Pets/Dragon/frames/`. Then write exactly that path in **PETS** in `pet.gd` (SECTION 0), with the same capital letters: `"dragon": "res://Pets/Dragon/frames",`. Windows ignores capitals in folder names, but Android doesn't, so a path that works on your PC can fail on the phone if the capitals differ.

A3. Build the scene

- 1 Go to **Scene > New Scene**, click **2D Scene** (or **Other Node > Node2D**), and rename the node `Main`.
- 2 Right-click `Main`, choose **Add Child Node**, pick **Node2D**, and rename it `Pet`.
- 3 Right-click `Pet`, choose **Add Child Node**, and pick **AnimatedSprite2D**. Keep its name exactly `AnimatedSprite2D`.
- 4 Leave the `AnimatedSprite2D`'s **Sprite Frames** slot empty. The script fills it when the game runs.

- 5 Select `Pet` and set **Transform > Position** to `(360, 800)`.
- 6 Save the scene with **Ctrl+S** as `main.tscn`.

Check: the Scene panel shows `Main > Pet > AnimatedSprite2D`. You won't see the pet yet; it appears in step A4.

A4. Add the pet script (tap, drag, resize, move)

The script is the file `pet.gd` that came with this guide. Add it as a file. **Don't copy the code out of this PDF:** PDFs drop the tab indentation GDScript needs (and can paste page footers into the code), and the script then fails without showing anything.

Tapping, dragging and pinching use touch events directly, so you don't need to add anything to the Input Map. On PC your mouse acts as a finger because of step A1.

- 1 Drag `pet.gd` from your computer into the **FileSystem** panel, into the top folder (`res://`).
- 2 Drag `pet.gd` from the FileSystem panel onto the `Pet` node in the Scene panel. A small scroll icon appears next to `Pet`.
- 3 Look at the 2D view in the middle (click **2D** at the top of the window if you see a 3D grid). The first egg frame now shows where `Pet` is, inside the blue phone-shaped frame. If it doesn't, choose **Scene > Reload Saved Scene**.
- 4 Press **Ctrl+S**, then **F5**. If Godot asks for a main scene, click **Select Current**.

Check: a game window opens with the egg wobbling, and the **Output** panel at the bottom shows `Project P: frames per animation: egg_idle 8, egg_hatch 8, pet_emerge 0, pet_idle 32, ... move_left 8, move_right 8, trick 16, low_energy 8, sleep 8, (trick: 2 versions), (low_energy: 1 versions) and Project P: showing pet 'dragon' from res://pets/dragon (128px frames)`. Click the egg 3 times to hatch it, click the hatched pet to tickle it, drag it around, and scroll the mouse wheel over it to resize it. The hatched pet blinks, and every few seconds it does something by itself: flies to another spot, hops, or plays its trick. The pet remembers its place, its size and whether it has hatched.

If you don't see the pet:

- **Red text in Output or Debugger mentioning `pet.gd` and a line number:** the script got damaged, usually from copying it out of the PDF. Delete it and add the `pet.gd` file again as in steps 1-2.
- **"no PNG frames found in `res://pets/dragon`":** the folder's name or place is different. Rename or move it so the frames are in `res://pets/dragon/` (all lowercase), or change the path in `PETS`.
- **"no frames found for 'jump'"** (or another animation): those files have a name the script doesn't know. Add the name, without the frame number, to that animation's list in `ANIMS` (SECTION 0 of `pet.gd`).
- **"Node not found: `AnimatedSprite2D`":** the child node was renamed. Rename it back to `AnimatedSprite2D`.
- **No `Project P: showing pet` line at all:** the script isn't on `Pet` (no scroll icon). Repeat step 2.
- **Frames in the wrong action:** check the `frames per animation` line in Output, and the names in `ANIMS`.
- **The egg doesn't hatch when tapped:** `egg_hatch` shows 0 in that line. The hatching frames have a name the script doesn't know; add it to the `egg_hatch` line in `ANIMS`. With `egg_hatch_emerge_..` frames, `pet_emerge 0` and a yellow "no frames found for 'pet_emerge'" line are expected.

Settings you can change (select **Pet** and use the Inspector, or edit SECTION 0 and 1 of **pet.gd**):

- **Pet size:** **Pet Size** is the starting size (1 = the PNG's own size, 2 = twice as big; whole numbers keep pixel art sharpest). **Min Size** and **Max Size** limit how far players can resize it. Use these rather than **Pet's Transform > Scale**, which the script overrides.
- **Travelling:** **Move Speed** is pixels per second, and **Move Bob** is how high flying pets bob. If the pet moves backwards, untick **Move Frames Face Right**.
- **Animation speed:** in **ANIMS** (SECTION 0), the third value of each line is the speed in frames per second. With 32 idle frames, **8.0** makes one loop take 4 seconds; use **16.0** or more for a faster loop. To speed up or slow down everything at once, select **AnimatedSprite2D** and change **Speed Scale**.

To see the egg again while testing: **Project > Open User Data Folder**, then delete **pet.cfg**.

What the pet does by itself (the **ACTIONS** list in SECTION 0 of **pet.gd**):

```
const ACTIONS := [  
  ["blink", "", 4],           # keep blinking a while longer  
  ["travel", "move", 3],     # fly / walk / run to the left or right  
  ["hop", "jump", 2],        # jump on the spot (jump, then landing)  
  ["play", "trick", 2],      # one of its tricks (see ANIM_SETS below)  
]
```

Each line is **[what it does, animation, weight]**:

- **"blink"** - nothing new: the pet keeps resting and blinking, so the idle lasts longer.
- **"travel"** - it goes to a random spot on the left or right. With "move" it uses the **move_left** frames going left and the **move_right** frames going right; a pet without those uses its one **move** set, mirrored to face the way it goes.
- **"hop"** - it jumps up on the spot with that animation, then plays **landing**.
- **"play"** - it plays that animation once where it stands. If the animation is listed in **ANIM_SETS** (below), it plays one version of it.
- **Weight** - how often a line is picked compared with the others. With the weights above, out of 11 picks about 4 are more blinking, 3 a trip, 2 a hop and 2 a trick. A higher number makes it more frequent; 0 switches a line off.
- **How often:** **ACTION_WAIT_MIN** and **ACTION_WAIT_MAX** (3 and 7) are the seconds of blinking between two actions. To test quickly, set them to 1 and 2. For a calmer pet, raise them or raise the weight of "blink".
- **BLINK_FRAMES** (8) is the length of one blink. An action starts between two blinks, never in the middle of one.

To add your own movement (marked **[ADD ACTION]** in the code):

- 1 Save its frames in the pet's folder, for example **spin_00.png ... spin_07.png**.
- 2 In **ANIMS**, add a line for it with loop set to **false**: **["spin", 8, 12.0, false, ["spin"]]**,
- 3 In **ACTIONS**, add a line: **["play", "spin", 2]**,
- 4 Press **F5**. The Output line **frames per animation** must show **spin 8**. The wallpaper picks it up on the next export.

One trick at a time (**ANIM_SETS** in SECTION 0 of **pet.gd**):

```
const ANIM_SETS := {  
  "trick": 8,  
  "low_energy": 8,  
}
```

An animation listed here is not one long movement but several separate ones in a row, each that many frames long. With `"trick": 8`, your 16 trick frames are two tricks of 8 frames: `trick_a_00 ... _07` and `trick_b_00 ... _07`. Each time the pet does a trick, it picks one of them at random, plays just that one, and goes back to blinking.

- **More tricks:** add another row of 8 frames (`trick_c_00 ... _07`). Nothing to change in the code.
- **Tricks of another length:** change the number, for example `"trick": 12`.
- **One long trick again:** delete the `"trick": 8`, line. The whole animation then plays from its first to its last frame.
- If the files aren't named `trick_a_00 ...` or `trick_00 ... trick_15`, add their name (without the number) to the list at the end of the `trick` line in `ANIMS`.

Check: after F5 the Output line ends with `(trick: 2 versions)`. Watch the pet: a trick now lasts 8 frames, and it isn't always the same one.

Low energy (`LOW_BATTERY_PERCENT` and `LOW_ENERGY_ACTIONS` in SECTION 0):

```
const LOW_BATTERY_PERCENT := 15
const LOW_ENERGY_ACTIONS := [
    ["blink", "", 3],          # keep blinking a while longer
    ["play", "low_energy", 4], # one of its low-energy movements
]
```

While the battery is at or below `LOW_BATTERY_PERCENT` and the phone isn't charging, the pet picks from this list instead of `ACTIONS`. The lines work like the ones in `ACTIONS`, so you can change the weights or add lines.

- 1 Save the low-energy frames in the pet's folder as `low_energy_00.png ... low_energy_07.png`. With more than one movement, name the rows `low_energy_a_00 ...`, `low_energy_b_00 ...`; one row is played at a time.
 - 2 Press **F5**. The Output line must show `low_energy 8` (or 16, 24...).
 - 3 To see it on the computer, where there is no battery: stop the game, select the **Pet** node, and set **Test Battery Percent** in the Inspector to `10`. Press **F5**: the pet now only blinks and does its low-energy movements. Set it back to `-1` afterwards.
- A pet without low-energy frames keeps doing its normal actions.
 - On the phone the battery level is read through the wallpaper add-on (Part C), so in the game this works in the one-app build. Reading the battery level needs no permission.

Sleep (`SLEEP_FROM_HOUR`, `SLEEP_UNTIL_HOUR` and `AWAKE_MINUTES` in SECTION 0):

```
const SLEEP_FROM_HOUR := 23
const SLEEP_UNTIL_HOUR := 6
const AWAKE_MINUTES := 5.0
```

The hours are on the 24-hour clock: 23 is 11 PM, 6 is 6 AM. Between them the hatched pet plays its sleep frames in a loop instead of `pet_idle`, and does nothing by itself.

- 1 Save the sleep frames in the pet's folder as `sleep_00.png ... sleep_07.png`.
- 2 Press **F5**. The Output line must show `sleep 8`.
- 3 To see it without waiting for the night: stop the game, select the **Pet** node, set **Test Hour** to `23`, and press **F5**. The pet sleeps. Set it back to `-1` afterwards.

- **Waking it up:** tap it. The first tap only wakes it (it blinks again); from then on it behaves as in the daytime, so the next tap tickles it and it does its actions. It stays awake for `AWAKE_MINUTES`, and every tap starts those minutes again. Then it falls asleep again, between two blinks.
- **Other hours:** change the two numbers. Set both to the same number to switch sleeping off.
- **A different speed:** the third value of the `sleep` line in `ANIMS` (6.0 frames per second).
- A pet without sleep frames stays awake.

Before you export: `Test Hour` and `Test Battery Percent` must both be `-1`, or the pet in the game sleeps or stays tired all day.

Updating a `pet.gd` you edited yourself: open `pet_gd_edits_2.txt` in a text editor. It lists eleven edits, each with the text to search for and the text to put there. Copy from that file, not from this PDF, so the tabs survive. If your `pet.gd` doesn't have `move_left` in it yet, do the fourteen edits in `pet_gd_edits.txt` first. The easier way is to use the new `pet.gd` as it is and put your own pets back into its `PETS` list.

For reference, this is what `pet.gd` contains (read it here, but add the file itself to Godot):

```
# =====
# pet.gd - THE PET IN THE GODOT GAME
# =====
# What this file does:
# Controls the pet inside the game: plays its animations, hatches the egg after a
# few taps, plays the tickle animation when the hatched pet is tapped, lets the
# player drag it around and resize it (pinch or mouse wheel), and remembers its
# position, size and whether it has hatched. By itself the pet mostly rests and
# blinks, and now and then does one thing picked at random from ACTIONS (SECTION 0):
# travels left or right, hops, plays one of its tricks, or just keeps blinking.
# On a low battery it does its low-energy movements instead, and at night it sleeps
# until it is tapped (LOW ENERGY and SLEEP in SECTION 0).
#
# Where it goes:
# Attached to the "Pet" node (Node2D), with a child AnimatedSprite2D.
# The script builds the animations from the pet's frame folder by itself when the
# game starts, so the AnimatedSprite2D's Sprite Frames slot can stay empty.
# In the editor it shows the first egg frame as a preview, so you can see the pet
# while building the scene.
#
# IMPORTANT: add this file to Godot as a file (drag pet.gd into the FileSystem panel).
# Don't copy the code out of the PDF guide: PDFs lose the tab indentation that
# GDScript needs, and the script then fails without showing the pet.
#
# >>> HOW TO ADD A NEW PET (Godot) <<<
# 1. Export its frames as separate PNGs (see "Frame file names" in SECTION 0).
# 2. Put them in their own folder in the project, e.g. res://pets/cat/
#    (subfolders inside it are fine, e.g. res://pets/cat/frames/).
# 3. Add one line to PETS below, e.g. "cat": "res://pets/cat",
# 4. To show it, set the Pet node's "Pet Id" in the Inspector to "cat"
#    (or call set_pet("cat") from another script).
# Also add the pet to PetCatalog.kt in the Android wallpaper app.
#
# How to find things:
# [EDIT] = a setting you can safely change
# [LOGIC] = rules for how the pet behaves (what plays after what)
# [FIX] = places to look first if something goes wrong
# [ADD PET] = where new pets go
# [ADD ACTION] = where new movements the pet does by itself go
#
```

```

# Sections in this file:
# 0. Pets and animations ([ADD PET] is here)      -> search "SECTION 0"
# 1. Settings                                    -> search "SECTION 1"
# 2. Pet state                                    -> search "SECTION 2"
# 3. Start-up                                    -> search "SECTION 3"
# 4. Touch: tap and drag                         -> search "SECTION 4"
# 4b. Resizing: pinch / mouse wheel             -> search "SECTION 4b"
# 4c. What the pet does by itself (ACTIONS)      -> search "SECTION 4c"
# 5. Animation rules (hatch, tickle, jump/landing) -> search "SECTION 5"
# 6. Helpers (screen position, keep on screen)   -> search "SECTION 6"
# 7. Saving and loading                         -> search "SECTION 7"
# 8. Building animations from a frame folder     -> search "SECTION 8"
# 9. Editor preview                             -> search "SECTION 9"
#
# Animation names: egg_idle, egg_hatch, pet_emerge, pet_idle, jump, landing, ticklish,
# recovery, move, move_left, move_right, trick, low_energy, sleep. Inside the game's
# app the Android wallpaper gets ANIMS, ACTIONS and the low-energy and sleep settings
# from this file on every export, so the two always match.
# =====
@tool # lets the script draw a preview of the pet in the editor (SECTION 9)
extends Node2D

# ===== SECTION 0: PETS AND ANIMATIONS =====
# [ADD PET] One line per pet: "id": "folder with its frame PNGs".
# The id is used in saved data, so don't change it after release.
# Keep this list in sync with PETS in PetCatalog.kt (Android).
# If the folder isn't found, the script also searches the project for a
# folder named like the id (e.g. any folder called "dragon") and says so.
const PETS := {
    "dragon": "res://pets/dragon",
    # "cat": "res://pets/cat",
    # "fox": "res://pets/fox",
}

# [EDIT] The animations: [name, frames, fps, loop, file names it accepts].
# frames = how many frames it has when files are only numbered (see below)
# file names = what this animation's files are called, without the frame number.
# The pet's name in front is allowed: "idle" also matches dragon_idle_00,
# owl_idle_00, cat_idle_00... Add your own names here if yours differ.
# The order of this list is also the order of numbered files (1-8 egg_idle, ...).
# pet_idle: 4 sets of 8 frames (idle_a .. idle_d), one blink each, played in a loop.
# The wallpaper inside the game's app reads this list on every export.
const ANIMS := [
    ["egg_idle", 8, 6.0, true, ["egg_idle"]], # egg_idle_00..07
    ["egg_hatch", 8, 10.0, false, ["egg_hatch", "egg_hatch_emerge"]], # egg_hatch_emerge...
    ["pet_emerge", 8, 10.0, false, ["pet_emerge", "emerge"]], # dragon_emerge_00..07
    ["pet_idle", 8, 8.0, true, ["pet_idle", "idle"]], # idle_a_00..idle_d_07
    ["jump", 8, 12.0, false, ["jump", "jump_land_a", "jump_landing_a"]], # jump_land_a...
    ["landing", 8, 12.0, false, ["landing", "jump_land_b", "jump_landing_b"]], # jump_land_b...
    ["ticklish", 8, 12.0, false, ["ticklish", "ticklish_recovery_a"]], # ticklish_recovery_a...
    ["recovery", 8, 10.0, false, ["recovery", "ticklish_recovery_b"]], # ticklish_recovery_b...
    ["move", 32, 12.0, true, ["move", "fly", "walk", "run", "hop"]], # fly_a.. / walk_a..
    ["move_left", 8, 12.0, true, ["move_left", "fly_left", "walk_left", "run_left"]],
    ["move_right", 8, 12.0, true, ["move_right", "fly_right", "walk_right", "run_right"]],
    ["trick", 16, 10.0, false, ["trick", "extra", "special", "dance"]], # trick_a.. trick_b..
    ["low_energy", 8, 8.0, false, ["low_energy", "low_battery", "tired"]], # low_energy...
    ["sleep", 8, 6.0, true, ["sleep", "sleeping", "asleep"]], # sleep_00..
]

# [EDIT] WHAT THE PET DOES BY ITSELF. Most of the time the hatched pet rests and blinks
# (pet_idle: 4 sets of 8 frames, one blink each). After a random wait it picks ONE
# line below at random, does it, and goes back to resting and blinking.
# One line per action: [what it does, animation, weight]
# "blink" = nothing new: it keeps resting and blinking (makes the idle longer)
# "travel" = goes to a random spot on the left or right with this animation,
# facing the way it goes (fly_frames bob in the air)
# "hop" = jumps up on the spot with this animation, then plays "landing"
# "play" = plays this animation once where it stands
# weight = how often it is picked compared with the others: 4 is picked four
# times as often as 1. Use 0 to switch a line off.
# An action whose animation has no frames for this pet is skipped.
# [ADD ACTION] To add your own movement:
# 1. Add its animation to ANIMS above, with loop = false, e.g.
# ["spin", 8, 12.0, false, ["spin"]], # spin_00..07
# 2. Add a line here, e.g.
# ["play", "spin", 2],
# The wallpaper picks both up from this file on the next export.
const ACTIONS := [
    ["blink", "", 4], # keep blinking a while longer

```

```

    ["travel", "move", 3],      # fly / walk / run to the left or right
    ["hop", "jump", 2],        # jump on the spot (jump, then landing)
    ["play", "trick", 2],      # one of its tricks (see ANIM_SETS below)
]
# [EDIT] Seconds of resting and blinking between two actions (a random time in this range).
const ACTION_WAIT_MIN := 3.0
const ACTION_WAIT_MAX := 7.0
# [EDIT] How many frames one blink has. An action starts between two blinks, never in the
#       middle of one.
const BLINK_FRAMES := 8
# [EDIT] The hop on the spot: how high (as a share of the pet's height) and how long.
const HOP_HEIGHT := 0.5
const HOP_SECONDS := 0.7

# [EDIT] MOVEMENTS WITH SEVERAL VERSIONS. An animation listed here is not one long
#       movement but several separate ones in a row, each this many frames long: with
#       "trick": 8, trick_a_00..07 is one trick and trick_b_00..07 another. When the
#       pet plays it, it picks ONE version at random, plays just that one, and goes
#       back to resting. Add more rows of frames (trick_c_.., trick_d_..) and they
#       join the choice by themselves.
#       Remove a line to play that animation from its first to its last frame again.
const ANIM_SETS := {
    "trick": 8,
    "low_energy": 8,
}

# [EDIT] LOW ENERGY. When the phone's battery is at or below LOW_BATTERY_PERCENT and
#       the phone is not charging, the pet picks from this list instead of ACTIONS:
#       it does its low-energy movements instead of travelling, hopping and tricks.
#       Same kind of lines as ACTIONS. A pet without low_energy frames keeps ACTIONS.
const LOW_BATTERY_PERCENT := 15
const LOW_ENERGY_ACTIONS := [
    ["blink", "", 3],          # keep blinking a while longer
    ["play", "low_energy", 4], # one of its low-energy movements
]

# [EDIT] SLEEP. Between these hours of the phone's clock (24-hour clock: 23 = 11 PM,
#       6 = 6 AM) the hatched pet sleeps: it plays "sleep" instead of pet_idle and
#       does nothing by itself. Tapping it wakes it up for AWAKE_MINUTES; every
#       further tap keeps it awake that long again. Then it goes back to sleep.
#       A pet without sleep frames never sleeps.
const SLEEP_FROM_HOUR := 23
const SLEEP_UNTIL_HOUR := 6
const AWAKE_MINUTES := 5.0

# Frame file names - any of these styles works:
#   A) Named by animation (what the sprite-sheet-slicer skill exports), e.g. the dragon:
#       egg_idle_00.png ... egg_idle_07.png      -> egg_idle
#       egg_hatch_emerge_00.png ... _07.png      -> egg_hatch (hatching and the pet
#       coming out, as one animation)
#       Older sets with two animations also work: egg_hatch_00.. -> egg_hatch and
#       dragon_emerge_00.. -> pet_emerge, played one after the other.
#       idle_a_00.png ... idle_d_07.png          -> pet_idle (rows a, b, c, d in order)
#       jump_land_a_00.png ... _07.png           -> jump
#       jump_land_b_00.png ... _07.png           -> landing
#       ticklish_recovery_a_00.png ... _07.png    -> ticklish
#       ticklish_recovery_b_00.png ... _07.png    -> recovery
#       move_left_00.png ... / move_right_00.png ... -> move_left / move_right: used
#       when the pet travels that way
#       fly_a_00.png ... fly_d_07.png            -> move (or walk_, run_, hop_): one set

```

```

#                                     for both ways, mirrored as needed
#      trick_a_00.png ... trick_b_07.png      -> trick: each row is one trick, and
#                                              one is picked at a time (ANIM_SETS)
#      low_energy_a_00.png ...                -> low_energy: played on a low battery
#      sleep_00.png ...                      -> sleep: its resting animation at night
#      Each file goes to the animation whose name it matches (see ANIMS above).
#      B) Only numbered, in the order of ANIMS: 01.png ... 64.png, frame_1 ... frame_64.
#      The first 8 files go to egg_idle, the next 8 to egg_hatch, and so on.
#      Numbers are compared as numbers, so frame_2 comes before frame_10.
#      The Output panel says which style was found and how many frames each animation got.

# ===== SECTION 1: SETTINGS =====
# [EDIT] Which pet this node shows (an id from PETS). Change it in the Inspector.
@export var pet_id: String = "dragon":
    set(value):
        pet_id = value
        if Engine.is_editor_hint():
            queue_redraw() # update the editor preview (SECTION 9)
# [EDIT] How close to the pet's centre a touch must land to grab it (in pixels).
#      Make it bigger if the pet is hard to grab.
# [EDIT] PET SIZE: 1 = the PNG's own size, 2 = twice as big. Starting size;
#      players can resize it by pinching (phone) or mouse wheel (PC), see SECTION 4b.
@export var pet_size: float = 2.0:
    set(value):
        pet_size = value
        scale = Vector2.ONE * pet_size
# [EDIT] Smallest and biggest size players can resize the pet to.
@export var min_size: float = 1.0
@export var max_size: float = 5.0
# [EDIT] How much one mouse-wheel step changes the size on PC.
const WHEEL_STEP := 0.25
# [EDIT] TRAVELLING: moving speed in pixels per second, and how high flying pets bob in
#      the air. Pets travel with their fly_, walk_, run_ or hop_ frames (whichever they
#      have); only pets with fly_ frames bob up and down.
#      How often the pet does something by itself is ACTION_WAIT_MIN / MAX in SECTION 0.
@export var move_speed := 180.0
@export var move_bob := 12.0
# [EDIT] Which way the pet faces in its moving frames. If it moves backwards, untick
#      this in the Inspector (or set it to false).
@export var move_frames_face_right := true
# [EDIT] Grab area at size 1 (multiplied by pet_size).
@export var grab_radius: float = 56.0
# [EDIT] How many taps on the egg before it hatches.
@export var taps_to_hatch: int = 3
# [EDIT] FOR TESTING: pretend the battery is at this percent (e.g. 10 to see the low-
#      energy movements) and pretend it is this hour (e.g. 23 to see the pet sleep).
#      -1 = use the real battery and the real clock. Set them in the Inspector and
#      put them back to -1 before you export.
@export var test_battery_percent: int = -1
@export var test_hour: int = -1
# [EDIT] How far a finger can move and still count as a tap instead of a drag.
const TAP_SLOP := 12.0
# Where the save file lives (Godot's private user folder).
# [FIX] To see the egg again while testing: Project > Open User Data Folder,
#      then delete pet.cfg.
const SAVE_PATH := "user://pet.cfg"

# The pet's AnimatedSprite2D child node.
# [FIX] If you get "node not found", check the child is named exactly AnimatedSprite2D.
@onready var sprite: AnimatedSprite2D = $AnimatedSprite2D

```

```

# ===== SECTION 2: PET STATE =====
var hatched := false      # has the egg hatched? (saved per pet, so it stays hatched)
var _busy := false        # a one-shot animation (hatching, tickle) is playing
var _egg_taps := 0         # taps counted on the egg so far
var _dragging := false    # a finger went down on the pet
var _moved := false       # the finger moved far enough to count as a drag
var _finger := -1         # which finger is holding the pet (for multi-touch)
var _offset := Vector2.ZERO # where on the pet the finger grabbed it
var _press_pos := Vector2.ZERO # where the finger first touched
var _touches := {}        # every finger on the screen: finger index -> position
var _pinching := false    # two fingers are resizing the pet
var _pinch_start_dist := 0.0 # distance between the fingers when the pinch began
var _pinch_start_size := 1.0 # pet_size when the pinch began
var _moving := false      # the pet is travelling across the screen
var _move_target_x := 0.0 # where it's going to (x position)
var _move_base_y := 0.0   # its height before setting off (flyers bob around this)
var _move_time := 0.0     # seconds since setting off (for the bob)
var _next_move := 0.0     # seconds until the next trip
var _move_bobs := false   # true if this pet's moving frames are fly_frames
var _hopping := false     # the pet is in the air, hopping on the spot
var _hop_time := 0.0      # seconds since it took off
var _hop_base_y := 0.0    # its height before the hop
var _awake_until := 0.0   # woken up at night: stays awake until this time
var _no_battery_info := false # the phone side can't tell the battery level (old add-on)
var _frame_size := 128    # size of one frame, measured from the first PNG in _build_frames()

# ===== SECTION 3: START-UP =====
# Runs once when the scene starts.
func _ready() -> void:
    scale = Vector2.ONE * pet_size    # apply PET SIZE
    if Engine.is_editor_hint():
        queue_redraw() # in the editor: only draw the preview (SECTION 9)
        return
    # Call _on_animation_finished() whenever a play-once animation ends.
    sprite.animation_finished.connect(_on_animation_finished)
    set_pet(pet_id)

# Switches this node to another pet: loads its frames and its saved progress.
# Other scripts can call it, e.g. $Pet.set_pet("cat")
# [FIX] If the pet doesn't appear, check the id is in PETS and the folder path is right.
func set_pet(id: String) -> void:
    if not PETS.has(id):
        push_warning("Unknown pet id '%s', using the first pet instead" % id)
        id = PETS.keys()[0]
    pet_id = id
    var frames := _build_frames(PETS[id])
    if frames == null:
        return # folder not found: the error message is in the Output panel
    sprite.sprite_frames = frames
    _egg_taps = 0
    _moving = false # the new pet doesn't finish the old pet's trip or hop
    _hopping = false
    sprite.flip_h = false
    _load() # restore this pet's position and hatched state
    _play_idle() # start resting: egg_idle or pet_idle
    _reset_move_timer()
    print("Project P: showing pet '%s' from %s (%dpix frames)" % [id, PETS[id], _frame_size])

# ===== SECTION 4: TOUCH (TAP AND DRAG) =====

```

```

# [LOGIC] Finger down on the pet -> up without moving = tap (_on_tap).
#       Finger down and moving = drag: pet plays "jump", then "landing" on release.
# Works with the mouse on PC because "Emulate Touch From Mouse" is on in Project Settings.
func _unhandled_input(event: InputEvent) -> void:
    if Engine.is_editor_hint():
        return # no tapping or dragging inside the editor
    if _handle_resize(event):
        return # the event was a pinch or mouse-wheel resize (SECTION 4b)
    # --- Finger touches or lifts ---
    if event is InputEventScreenTouch:
        var touch := event as InputEventScreenTouch
        var p := _to_world(touch.position)
        # Finger goes down on the pet: start tracking it
        if touch.pressed and not _dragging \
            and p.distance_to(global_position) <= grab_radius * pet_size:
            if _busy and not hatched:
                return # leave the egg alone while it hatches
            if _moving:
                _stop_moving() # grabbing the pet mid-trip makes it stop
            _dragging = true
            _moved = false
            _finger = touch.index
            _press_pos = p
            _offset = global_position - p
            get_viewport().set_input_as_handled()
        # Finger lifts: finish the drag (land + save) or count it as a tap
        elif not touch.pressed and touch.index == _finger:
            _dragging = false
            _finger = -1
            if _moved:
                if hatched:
                    sprite.play("landing")
                    _save()
            else:
                _on_tap()
            get_viewport().set_input_as_handled()
    # --- Finger moves while holding the pet ---
    elif event is InputEventScreenDrag and _dragging:
        var drag := event as InputEventScreenDrag
        if drag.index != _finger:
            return
        var q := _to_world(drag.position)
        # Moved past TAP_SLOP: this is a drag, not a tap
        if not _moved and q.distance_to(_press_pos) > TAP_SLOP:
            _moved = true
            if hatched:
                _busy = false
                sprite.play("jump") # holds its last frame while the finger is down
            # Move the pet with the finger, kept inside the screen
            if _moved:
                global_position = _clamp_to_screen(q + _offset)
            get_viewport().set_input_as_handled()

# ===== SECTION 4b: RESIZING (pinch on phones, mouse wheel on PC) =====
# One finger on the pet + a second finger: spread = bigger, pinch = smaller.
# On PC: scroll the mouse wheel over the pet. Size is saved with the pet.
func _handle_resize(event: InputEvent) -> bool:
    # Keep track of every finger on the screen
    if event is InputEventScreenTouch:
        var t := event as InputEventScreenTouch

```

```

        if t.pressed:
            _touches[t.index] = _to_world(t.position)
        else:
            _touches.erase(t.index)
    elif event is InputEventScreenDrag:
        var d := event as InputEventScreenDrag
        _touches[d.index] = _to_world(d.position)

# A second finger arrives while the first one holds the pet: start pinching
if not _pinching and _dragging and _touches.size() == 2:
    _pinching = true
    _dragging = false
    _finger = -1
    if sprite.animation == "jump":
        _play_idle()
    _pinch_start_dist = _finger_distance()
    _pinch_start_size = pet_size
    get_viewport().set_input_as_handled()
    return true
if _pinching:
    if _touches.size() < 2:          # a finger lifted: the pinch is over
        _pinching = false
        _save()
    elif event is InputEventScreenDrag and _pinch_start_dist > 0.0:
        _set_size(_pinch_start_size * _finger_distance() / _pinch_start_dist)
    get_viewport().set_input_as_handled()
    return true

# Mouse wheel over the pet (PC)
if event is InputEventMouseButton and event.pressed:
    var mb := event as InputEventMouseButton
    var up := mb.button_index == MOUSE_BUTTON_WHEEL_UP
    var down := mb.button_index == MOUSE_BUTTON_WHEEL_DOWN
    if (up or down) and _to_world(mb.position).distance_to(global_position) \
        <= grab_radius * pet_size:
        _set_size(pet_size + (WHEEL_STEP if up else -WHEEL_STEP))
        _save()
        get_viewport().set_input_as_handled()
        return true
return false

# Changes the size within min_size..max_size and keeps the pet on screen.
func _set_size(new_size: float) -> void:
    pet_size = clampf(new_size, min_size, max_size)
    global_position = _clamp_to_screen(global_position)

# Distance between the first two fingers on the screen.
func _finger_distance() -> float:
    var points := _touches.values()
    return (points[0] as Vector2).distance_to(points[1] as Vector2)

# ===== SECTION 4c: WHAT THE PET DOES BY ITSELF =====
# [LOGIC] While the hatched pet rests (pet_idle, blinking) and nobody is touching it, a
#         timer runs down (ACTION_WAIT_MIN..ACTION_WAIT_MAX seconds). Then, between two
#         blinks, it picks one action from ACTIONS (SECTION 0) at random, does it, and
#         goes back to resting. The timer starts again.
func _process(delta: float) -> void:
    if Engine.is_editor_hint():
        return

```

```

if _moving:
    _move_step(delta)
    return
if _hopping:
    if _dragging:
        # grabbed in the air: put it down, the drag takes over
        _hopping = false
        global_position.y = _hop_base_y
        _play_idle()
    else:
        _hop_step(delta)
    return
if not hatched or _busy or _dragging or _pinching or sprite.sprite_frames == null:
    return
# Falling asleep or waking up: the resting animation it should have now is another
# one than it is playing (the clock passed SLEEP_FROM_HOUR, or its time awake is over)
var playing := String(sprite.animation)
var rest := _rest_anim()
if (playing == "pet_idle" or playing == "sleep") and playing != rest:
    if playing == "sleep" or sprite.frame % BLINK_FRAMES == 0:
        _play_idle()
        # (an awake pet finishes its blink first)
    return
if playing != "pet_idle":
    return
    # asleep: it does nothing by itself
_next_move -= delta
if _next_move > 0.0:
    return
if sprite.frame % BLINK_FRAMES != 0:
    return
    # let the current blink finish first
_do_random_action()

# Picks a new random wait before the next action.
func _reset_move_timer() -> void:
    _next_move = randf_range(ACTION_WAIT_MIN, ACTION_WAIT_MAX)

# [LOGIC] Picks one action from ACTIONS at random (by weight) and starts it.
# Actions this pet has no frames for are left out.
func _do_random_action() -> void:
    _reset_move_timer()
    var choices := []
    var total := 0.0
    for a in _actions_now():
        # ACTIONS, or LOW_ENERGY_ACTIONS on a low battery
        if float(a[2]) > 0.0 and (a[0] == "blink" or _can_do(a[0], a[1])):
            choices.append(a)
            total += float(a[2])
    var pick := randf() * total
    for a in choices:
        pick -= float(a[2])
        if pick <= 0.0:
            _do_action(a[0], a[1])
            return

# Starts one action. `kind` is the first word of its line in ACTIONS.
# [FIX] An action never happens: read the "frames per animation" line in the Output
# panel. Its animation must have more than 0 frames.
func _do_action(kind: String, anim: String) -> void:
    match kind:
        "travel":
            _start_moving(anim)
        "hop":
            _start_hop(anim)

```

```

    "play":
        if sprite.sprite_frames.get_animation_loop(anim):
            return # a looping animation would never end
        _busy = true # no tickling until it has finished
        sprite.play(_one_version(anim)) # e.g. "trick#2"; when it ends -> resting
    _:
        pass # "blink": it simply keeps resting

# [LOGIC] An animation listed in ANIM_SETS has several versions, built when the pet is
# loaded: "trick#1", "trick#2", ... This picks one of them at random. For any
# other animation it returns the animation itself.
func _one_version(anim: String) -> String:
    var count := 0
    while sprite.sprite_frames.has_animation("%s%d" % [anim, count + 1]):
        count += 1
    if count == 0:
        return anim
    return "%s%d" % [anim, randi_range(1, count)]

# [LOGIC] The list the pet picks its next action from: LOW_ENERGY_ACTIONS while the
# battery is low and the pet has frames for something in it, otherwise ACTIONS.
func _actions_now() -> Array:
    if _is_low_energy():
        for a in LOW_ENERGY_ACTIONS:
            if a[0] != "blink" and float(a[2]) > 0.0 and _can_do(a[0], a[1]):
                return LOW_ENERGY_ACTIONS
    return ACTIONS

# Is the phone's battery low (and not charging)? Always false on the computer, unless
# test_battery_percent is set in the Inspector.
# On the phone the battery is read through the wallpaper add-on (WallpaperBridge.kt).
func _is_low_energy() -> bool:
    if test_battery_percent >= 0:
        return test_battery_percent <= LOW_BATTERY_PERCENT
    if _no_battery_info or not Engine.has_singleton("ProjectPWallpaper"):
        return false
    var phone = Engine.get_singleton("ProjectPWallpaper")
    var percent = phone.call("batteryPercent")
    if not (percent is int):
        _no_battery_info = true # an older add-on without battery info: stop asking
        return false
    if percent < 0 or phone.call("isCharging") == true:
        return false
    return percent <= LOW_BATTERY_PERCENT

# [LOGIC] Is it the pet's bedtime right now? True between SLEEP_FROM_HOUR and
# SLEEP_UNTIL_HOUR by the phone's clock, unless a tap woke it up a moment ago.
func _is_sleep_time() -> bool:
    if Time.get_unix_time_from_system() < _awake_until:
        return false # woken up: awake for a while
    var hour: int = test_hour if test_hour >= 0 else Time.get_time_dict_from_system()["hour"]
    if SLEEP_FROM_HOUR <= SLEEP_UNTIL_HOUR:
        return hour >= SLEEP_FROM_HOUR and hour < SLEEP_UNTIL_HOUR
    return hour >= SLEEP_FROM_HOUR or hour < SLEEP_UNTIL_HOUR # (through midnight)

# The animation the pet rests with right now: the egg's idle before hatching, "sleep"
# at night (if it has sleep frames), otherwise pet_idle (blinking).
func _rest_anim() -> String:
    if not hatched:
        return "egg_idle"

```

```

    if _is_sleep_time() and _has_frames("sleep"):
        return "sleep"
    return "pet_idle"

# Does this pet have frames for an animation?
func _has_frames(anim: String) -> bool:
    return sprite.sprite_frames != null and sprite.sprite_frames.has_animation(anim) \
        and sprite.sprite_frames.get_frame_count(anim) > 0

# Can this pet do an action? "travel" needs frames for its animation or for the
# _left / _right versions of it; everything else needs frames for its animation.
func _can_do(kind: String, anim: String) -> bool:
    if kind == "travel":
        return not _travel_anim(anim, true).is_empty()
    return _has_frames(anim)

# [LOGIC] Which animation the pet travels with, and whether it is drawn mirrored.
#         `base` is the animation named in ACTIONS, normally "move".
#         going left -> "move_left" frames, if the pet has them
#         going right -> "move_right" frames, if the pet has them
#         only one of the two -> that one, mirrored for the other direction
#         neither -> "move" itself (fly_ / walk_ / run_ frames), mirrored when it
#         goes the other way than the frames face (move_frames_face_right)
#         Returns [animation, mirrored], or [] if the pet has no frames to travel with.
func _travel_anim(base: String, going_left: bool) -> Array:
    var own := base + ("_left" if going_left else "_right")
    var other := base + ("_right" if going_left else "_left")
    if _has_frames(own):
        return [own, false]
    if _has_frames(other):
        return [other, true]
    if _has_frames(base):
        return [base, going_left == move_frames_face_right]
    return []

# "travel": sets off towards a random spot on the left or right.
func _start_moving(anim: String = "move") -> void:
    var view := get_viewport_rect().size
    var half := _frame_size * 0.5 * scale.x
    var target := randf_range(half, view.x - half)
    if absf(target - global_position.x) < view.x * 0.25:
        # Too short a trip: go towards the farther side of the screen instead
        target = half if global_position.x > view.x / 2.0 else view.x - half
    var going_left := target < global_position.x
    var travel := _travel_anim(anim, going_left) # e.g. ["move_left", false]
    if travel.is_empty():
        return # this pet has no frames to travel with
    _move_target_x = target
    _move_base_y = global_position.y
    _move_time = 0.0
    _moving = true
    _busy = true # no tickling while it's on the move
    # Only pets whose travelling frames are named fly_ bob in the air
    var first := sprite.sprite_frames.get_frame_texture(travel[0], 0)
    _move_bobs = first.resource_path.get_file().to_lower().begins_with("fly")
    sprite.flip_h = travel[1] # mirrored only when it has no frames for this side
    sprite.play(travel[0])

# Moves the pet a little each frame until it reaches its spot.
func _move_step(delta: float) -> void:

```

```

_move_time += delta
var step := move_speed * delta
var gap := _move_target_x - global_position.x
if absf(gap) <= step:
    global_position = Vector2(_move_target_x, _move_base_y)
    _stop_moving()
    return
global_position.x += signf(gap) * step
if _move_bobs:
    global_position.y = _move_base_y - absf(sin(_move_time * 3.0)) * move_bob * scale.y

# Stops: back to resting, facing forward, position saved, next action scheduled.
func _stop_moving() -> void:
    _moving = false
    global_position.y = _move_base_y
    sprite.flip_h = false
    _play_idle()
    _save()
    _reset_move_timer()
# "hop": jumps up on the spot. The animation (jump) plays while it is in the air; when
# it comes down, "landing" plays and then it rests (see _on_animation_finished).
func _start_hop(anim: String = "jump") -> void:
    _hopping = true
    _hop_time = 0.0
    _hop_base_y = global_position.y
    _busy = true # no tickling while it's in the air
    sprite.play(anim)

# Moves the hopping pet up and back down in an arc.
func _hop_step(delta: float) -> void:
    _hop_time += delta
    var t := _hop_time / HOP_SECONDS # 0 at take-off, 1 at touchdown
    if t >= 1.0:
        _hopping = false
        global_position.y = _hop_base_y
        if _has_frames("landing"):
            sprite.play("landing") # -> resting when it ends
        else:
            _play_idle()
        return
    global_position.y = _hop_base_y - sin(t * PI) * HOP_HEIGHT * _frame_size * scale.y

# [LOGIC] What a tap on the pet does.
# Egg: count taps, hatch after taps_to_hatch (SECTION 1).
# Pet: play ticklish (then recovery, see _on_animation_finished).
func _on_tap() -> void:
    if _busy:
        return # ignore taps while hatching or being tickled
    if not hatched:
        _egg_taps += 1
        if _egg_taps >= taps_to_hatch:
            _busy = true
            if _has_frames("egg_hatch"):
                sprite.play("egg_hatch") # the egg hatches (and the pet emerges)
            elif _has_frames("pet_emerge"):
                sprite.play("pet_emerge") # only emerge frames: play those
            else:
                hatched = true # no hatching frames at all: hatch at once
            _save()

```

```

        _play_idle()
    else:
        # Any tap keeps the pet awake for AWAKE_MINUTES (this only matters at night)
        var was_asleep := String(sprite.animation) == "sleep"
        _awake_until = Time.get_unix_time_from_system() + AWAKE_MINUTES * 60.0
        if was_asleep:
            _play_idle()          # it wakes up: back to blinking
            _reset_move_timer()
            return
        _busy = true
        sprite.play("ticklish")

# ===== SECTION 5: ANIMATION RULES =====
# [LOGIC] What plays after each play-once animation ends.
# Change these lines to change the order of animations.
# [FIX] If an animation never moves on, make sure its loop is false in ANIMS
# (only egg_idle and pet_idle should loop), otherwise it never "finishes".
func _on_animation_finished() -> void:
    match String(sprite.animation):
        "egg_hatch":
            if _has_frames("pet_emerge"):
                sprite.play("pet_emerge") # egg cracks -> pet comes out
            else:
                hatched = true             # hatching and emerging are one animation:
                _save()                    # the pet is out -> remember it
                _play_idle()
        "pet_emerge":
            hatched = true                 # pet is out -> remember it
            _save()
            _play_idle()
        "ticklish":
            sprite.play("recovery")        # tickled -> calms down
        "recovery", "landing":
            _play_idle()                  # -> back to resting
        "jump":
            # In the air or in the hand it holds its last frame; otherwise -> resting
            if not _hopping and not _moving and not _dragging:
                _play_idle()
        _:
            # Any other play-once animation (e.g. "trick") -> back to resting. While
            # the pet travels or hops with it, it holds the last frame until it arrives.
            if not _moving and not _hopping:
                _play_idle()

# Go back to resting: egg_idle before hatching; after it pet_idle (blinking), or
# "sleep" at night (see _rest_anim in SECTION 4c).
func _play_idle() -> void:
    _busy = false
    sprite.play(_rest_anim())

# ===== SECTION 6: HELPERS =====
# Converts a touch position on the screen into a position in the game world.
func _to_world(screen_pos: Vector2) -> Vector2:
    return get_canvas_transform().affine_inverse() * screen_pos

# Keeps the pet fully inside the screen.
func _clamp_to_screen(p: Vector2) -> Vector2:
    var view := get_viewport_rect().size
    var half := Vector2(_frame_size, _frame_size) * 0.5 * scale # half of one frame
    return p.clamp(half, view - half)

```

```

# ===== SECTION 7: SAVING AND LOADING =====
# Saves the pet's position and whether it has hatched to user://pet.cfg.
# Each pet has its own section in the file (named after its id), so every pet keeps
# its own progress.
# [EDIT] Add more cfg.set_value(...) lines here to save more things (e.g. stats).
func _save() -> void:
    var cfg := ConfigFile.new()
    cfg.load(SAVE_PATH) # keep the other pets' data already in the file
    cfg.set_value(pet_id, "position", global_position)
    cfg.set_value(pet_id, "hatched", hatched)
    cfg.set_value(pet_id, "size", pet_size)
    cfg.save(SAVE_PATH)

# Loads this pet's data back. If there's no save yet, it starts as an egg where the
# Pet node was placed in the editor.
func _load() -> void:
    hatched = false
    var cfg := ConfigFile.new()
    if cfg.load(SAVE_PATH) == OK:
        global_position = cfg.get_value(pet_id, "position", global_position)
        hatched = cfg.get_value(pet_id, "hatched", false)
        pet_size = cfg.get_value(pet_id, "size", pet_size)

# ===== SECTION 8: BUILDING ANIMATIONS FROM A FRAME FOLDER =====
# Finds the pet's PNG frames, sorts them into animations (by name or by number, see
# SECTION 0) and returns them as SpriteFrames, ready for the AnimatedSprite2D.
# [FIX] Read the "Project P:" lines in the Output panel: they list the folder used,
# how the files were matched, and how many frames each animation got.
func _build_frames(folder: String) -> SpriteFrames:
    var groups := _frame_groups(folder)
    if groups.is_empty():
        return null
    var frames := SpriteFrames.new()
    frames.remove_animation("default") # SpriteFrames starts with an empty one
    var summary := PackedStringArray()
    for a in ANIMS:
        var anim_name: String = a[0]
        frames.add_animation(anim_name)
        frames.set_animation_speed(anim_name, a[2])
        frames.set_animation_loop(anim_name, a[3])
        var paths: Array = groups.get(anim_name, [])
        for path in paths:
            frames.add_frame(anim_name, load(path))
        summary.append("%s %d" % [anim_name, paths.size()])
    if paths.is_empty():
        push_warning("Project P: no frames found for '%s'." % anim_name)
# Movements with several versions (ANIM_SETS, SECTION 0): besides the whole animation,
# one extra animation per version is made, named "trick#1", "trick#2", ...
for set_name in ANIM_SETS:
    var per_version: int = ANIM_SETS[set_name]
    var set_paths: Array = groups.get(set_name, [])
    if per_version <= 0 or not frames.has_animation(set_name):
        continue
    @warning_ignore("integer_division")
    var versions: int = set_paths.size() / per_version
    for v in versions:
        var version_name := "%s#%d" % [set_name, v + 1]
        frames.add_animation(version_name)
        frames.set_animation_speed(version_name, frames.get_animation_speed(set_name))

```

```

        frames.set_animation_loop(version_name, frames.get_animation_loop(set_name))
        for path in set_paths.slice(v * per_version, (v + 1) * per_version):
            frames.add_frame(version_name, load(path))
    if versions > 0:
        summary.append("%s: %d versions" % [set_name, versions])
    print("Project P: frames per animation: " + ", ".join(summary))
    for a in ANIMS:
        # measure the frame size from the first frame
        var first: Array = groups.get(a[0], [])
        if not first.is_empty():
            _frame_size = (load(first[0]) as Texture2D).get_width()
            break
    return frames

# Sorts a pet's PNG files into animations. Returns { "egg_idle": [paths...], ... }.
func _frame_groups(folder: String) -> Dictionary:
    var files := _png_files(folder)
    if files.is_empty():
        # Not found where PETS says: look for a folder named like the pet anywhere
        var found := _find_folder("res://", folder.get_file().to_lower())
        if found != "":
            push_warning("Project P: no frames in %s, using %s instead. Update PETS in "
                % [folder, found] + "pet.gd (SECTION 0) to this path.")
            files = _png_files(found)
    if files.is_empty():
        push_error("Project P: no PNG frames found in %s (or its subfolders). Check the "
            % folder + "folder name and place match PETS in pet.gd (SECTION 0).")
        return {}
    files.sort_custom(func(x, y): return x.naturalnocasecmp_to(y) < 0)
    print("Project P: found %d PNG frames in %s, e.g. %s" % [files.size(), folder,
        files[0].get_file()])

# Style A: files named by animation (egg_idle_00.png ...)
var groups := {}
var unmatched := 0
for path in files:
    var anim := _anim_for_file(path.get_file())
    if anim == "":
        unmatched += 1
    else:
        if not groups.has(anim):
            groups[anim] = []
        groups[anim].append(path)
if not groups.is_empty():
    print("Project P: matched frames by animation name.")
    if unmatched > 0:
        push_warning("Project P: %d files didn't match any animation name in ANIMS "
            % unmatched + "(SECTION 0) and were skipped.")
    return groups

# Style B: files only numbered -> hand them out in ANIMS order
print("Project P: matched frames by number (first %d -> %s, ...)." % [ANIMS[0][1],
    ANIMS[0][0]])
var next := 0
for a in ANIMS:
    var count: int = a[1]
    groups[a[0]] = files.slice(next, next + count)
    next += count
if files.size() != next:
    push_warning("Project P: found %d frames, ANIMS expects %d." % [files.size(), next])
return groups

```

```

# Which animation a file belongs to, from its name (" " if none).
# "egg_idle_03.png" -> "egg_idle" (exact name in ANIMS)
# "dragon_idle_03.png" -> "pet_idle" (pet name in front + "idle")
# "jump_land_a_05.png" -> "jump" (exact name in ANIMS)
# [EDIT] If a file isn't recognised, add its name (without the number) to ANIMS.
func _anim_for_file(file_name: String) -> String:
    var label := file_name.get_basename().to_lower()
    var re := RegEx.new()
    re.compile("[_-]*\\d+$") # remove the frame number at the end
    label = re.sub(label, "")
    # 1) exact match, e.g. "egg_idle", "jump_land_a"
    for a in ANIMS:
        if label in a[4]:
            return a[0]
    # 2) pet name in front, e.g. "dragon_idle" ends with "_idle"
    for a in ANIMS:
        for n in a[4]:
            if label.ends_with("_" + n):
                return a[0]
    # 3) extra rows from the slicer, e.g. "idle_b" -> try "idle"
    var row := RegEx.new()
    row.compile("[a-z]$")
    if row.search(label) != null:
        return _anim_for_file(row.sub(label, "") + "_00.png")
    return ""

# All PNG files in a folder and its subfolders (full paths).
# Works in the editor and in exported games, where Godot keeps only "<name>.png.import".
func _png_files(folder: String) -> Array:
    var result := []
    var dir := DirAccess.open(folder)
    if dir == null:
        return result
    for f in dir.get_files():
        var fname := f.trim_suffix(".import").trim_suffix(".remap")
        if fname.get_extension().to_lower() == "png":
            var path := folder.path_join(fname)
            if not result.has(path):
                result.append(path)
    for d in dir.get_directories():
        if not d.begins_with("."):
            result.append_array(_png_files(folder.path_join(d)))
    return result

# Searches the project for a folder called `wanted` that contains PNGs (" " if none).
func _find_folder(start: String, wanted: String) -> String:
    var dir := DirAccess.open(start)
    if dir == null:
        return ""
    for d in dir.get_directories():
        if d.begins_with(".") or d == "addons":
            continue
        var path := start.path_join(d)
        if d.to_lower() == wanted and not _png_files(path).is_empty():
            return path
        var deeper := _find_folder(path, wanted)
        if deeper != "":
            return deeper
    return ""

# ===== SECTION 9: EDITOR PREVIEW =====
# Only runs inside the Godot editor: draws the pet's first egg frame where the pet will
# be, so you can see and place the pet while building the scene. Nothing here runs in
# the game itself (the AnimatedSprite2D shows the real animations there).
# [FIX] No preview? Check the folder path in PETS, then use Scene > Reload Saved Scene.
func _draw() -> void:
    if not Engine.is_editor_hint():
        return
    var folder: String = PETS.get(pet_id, PETS.values()[0])
    var groups := _frame_groups(folder)
    for a in ANIMS:
        var paths: Array = groups.get(a[0], [])
        if not paths.is_empty():
            var tex: Texture2D = load(paths[0])
            draw_texture(tex, -tex.get_size() / 2.0) # centred like the AnimatedSprite2D
            return

```

A5. The pet selection grid (splash art)

`pet_picker.gd` shows the **Choose your pet** screen: a grid of pets, each shown with its own splash art, 2 per row (so 6 pets make 3 rows; more pets scroll). Tapping a pet's splash art chooses that pet, saves the choice and goes straight to the pet. While the grid is open, the pet and its egg are hidden. It reads the list from `PETS` in `pet.gd`, so new pets appear by themselves.

The grid is opened by the **Pet Selection** button of the main menu, which you add in step A6. Do this step first.

- 1 Add your pets to `PETS` in `pet.gd` and put each pet's frames in its folder (see "More pets"). For a grid of 6, `PETS` has 6 lines.
- 2 Drag `pet_picker.gd` from your computer into the **FileSystem** panel, next to `pet.gd`. If an older `pet_picker.gd` is already there, replace it.
- 3 Open `main.tscn`. Right-click **Main**, choose **Add Child Node**, search for **CanvasLayer**, and click **Create**. Rename it `PetPicker`. (Skip this if you already have a `PetPicker` node.)
- 4 Drag `pet_picker.gd` from the **FileSystem** panel onto the `PetPicker` node.
- 5 Save with **Ctrl+S**.

Check: the Scene panel shows `Main > Pet` and `PetPicker` side by side (PetPicker is not inside Pet).

With the main menu, steps 3 and 4 are optional. If the scene has no `PetPicker` node, the menu from step A6 makes one itself from `res://pet_picker.gd`; the file only has to be in the project's top folder. The grid works with a single pet too: it then shows one tile.

Add the splash art (one picture per pet):

- 1 In the **FileSystem** panel, right-click `res://`, choose **New > Folder**, and name it `menu`. Right-click `menu`, choose **New > Folder**, and name it `splash`. (If you unzip `ProjectP_Menu_Assets.zip` in step A6, these folders already exist.)
- 2 Name each picture after the pet's id in `PETS`, in small letters: `dragon.png` for "dragon", `cat.png` for "cat". PNG, JPG or WebP.
- 3 Draw them in the tile's shape: 600 x 660 pixels is best (the tile is 300 x 330). Other shapes are cut to fit, never stretched.
- 4 Drag the pictures into `res://menu/splash/`.

Check: press **F5**, open **Pet Selection** (step A6), and each pet shows its splash art. A pet without a picture shows its first idle frame on a plain tile instead, so the grid works before all art is ready. The pet chosen now has a thicker golden frame.

Settings you can change (SECTION 1 of `pet_picker.gd`):

- **Pets per row and tile size:** `columns` and `TILE_SIZE`. For 3 per row use `columns 3` and `TILE_SIZE Vector2(210, 260)`, and draw the splash art in that shape.
- **Names on the tiles:** `show_names`. Set it to `false` if the name is already painted into your splash art.
- **Texts:** `title_text` ("Choose your pet") and `back_button_text` ("Back").
- **Frame colours:** `TILE_EDGE_COLOR` and `CURRENT_EDGE_COLOR`.
- **Pixel-art splash art:** set `SMOOTH_PICTURES` to `false`.

If something is wrong:

- **A tile shows a small pet frame instead of the splash art:** the picture isn't in `res://menu/splash/`, or its name isn't exactly the pet's id (`dragon.png`, small letters).
- **"Script inherits from native type 'CanvasLayer'":** the node is the wrong type. Delete it and add a `CanvasLayer`.
- **"Node not found: ../Pet":** `PetPicker` is inside `Pet`. Drag it onto `Main`.

For reference, this is what `pet_picker.gd` contains (add the file itself to Godot):

```
# =====
# pet_picker.gd - THE PET SELECTION GRID IN THE GODOT GAME
# =====
# What this file does:
#   Shows the "Choose your pet" screen: a grid of pets, each shown with its own splash
#   art (2 per row by default, so 6 pets make 3 rows; it scrolls if there are more than
#   fit). Tapping a pet's splash art chooses that pet, saves the choice and goes
#   straight to the pet. While the grid is open the pet (or its egg) is hidden.
#
#   With the main menu (main_menu.gd on a "MainMenu" node): the grid is opened by the
#   menu's Pet Selection button, shows on top of the menu's animated background, and
#   its Back button returns to the menu.
#   Without a main menu: it adds its own "Change pet" and "Wallpaper" buttons in the
#   top-left corner and opens by itself on the very first start, as before.
#
# Where it goes:
#   On a CanvasLayer node called "PetPicker", added as a child of "Main" (next to "Pet").
#   It reads the list of pets from PETS in pet.gd, so new pets appear here by themselves.
#
# Splash art:
#   One picture per pet in res://menu/splash/, named after the pet's id in PETS:
#   dragon.png, cat.png, ... (PNG, JPG or WebP). A pet without splash art shows its
#   first idle frame instead, so the grid works before the art is ready.
#
# How to find things:
#   [EDIT] = a setting you can safely change (text, sizes, positions)
#   [LOGIC] = what leads to what
#   [FIX] = places to look first if something goes wrong
#
# Sections in this file:
#   1. Settings                                -> search "SECTION 1"
#   2. Start-up                                -> search "SECTION 2"
#   3. Its own buttons (only without a main menu) -> search "SECTION 3"
#   4. The pet grid                            -> search "SECTION 4"
#   5. One pet's tile (splash art)             -> search "SECTION 5"
#   6. Helpers                                -> search "SECTION 6"
# =====

extends CanvasLayer
# The main menu listens to these to know what the player did.
signal pet_chosen(id: String)      # a pet was picked
signal cancelled                   # the grid was closed without picking

# ===== SECTION 1: SETTINGS =====
# [FIX] Where the Pet node is, seen from this node. "../Pet" = the Pet next to it.
@export var pet_path: NodePath = ^"../Pet"
# [EDIT] Without a main menu: open the grid by itself the first time the game starts
#   (only if there's more than one pet to choose from).
@export var ask_on_first_start := true
# [EDIT] Text on the screens and buttons.
@export var title_text := "Choose your pet"
@export var back_button_text := "Back"
@export var change_button_text := "Change pet"
@export var wallpaper_button_text := "Wallpaper"
# [EDIT] The folder with the pets' splash art: one picture per pet, named after the
#   pet's id in PETS (pet.gd), e.g. dragon.png for "dragon".
const SPLASH_FOLDER := "res://menu/splash"
# [EDIT] Grid layout: how many pets per row, and the size of each pet's tile (pixels of
#   the 720-wide screen). 2 x 300px with 6 pets = 3 rows, no scrolling needed.
```

```

# For 3 per row use columns 3 and TILE_SIZE Vector2(210, 260).
# Splash art drawn in the tile's shape (here 300 x 330, or a multiple such as
# 600 x 660) fills its tile exactly; other shapes are cut to fit, never stretched.
@export var columns := 2
const TILE_SIZE := Vector2(300, 330)
const GAP := 16
const FONT_SIZE := 28
# [EDIT] Show each pet's name over the bottom of its splash art. Set to false if the
# name is already part of your splash art.
@export var show_names := true
# [EDIT] The frame around every tile, and around the pet that is chosen now.
const TILE_EDGE_COLOR := Color(1, 1, 1, 0.55)
const CURRENT_EDGE_COLOR := Color(1.0, 0.78, 0.25)
const TILE_EDGE_WIDTH := 4
const CURRENT_EDGE_WIDTH := 8
# [EDIT] Without a main menu: the picture behind the grid. It is the picture the
# wallpaper uses as the Project P wallpaper. With a main menu, the menu's own
# animated background is behind the grid instead.
const BACKGROUND_PICTURE := "res://addons/projectp_wallpaper/default_wallpaper.png"
# [EDIT] How much the background is darkened so the pets and text stand out:
# 0 = not at all, 1 = black.
const BACKGROUND_DARKEN := 0.3
# [EDIT] Colour of the dark edge around the title and names, which keeps them readable.
const TITLE_EDGE_COLOR := Color(0.23, 0.15, 0.10)
# [EDIT] true = splash art and the background are drawn smooth (right for painted
# art). Set to false if your splash art is pixel art.
const SMOOTH_PICTURES := true
# The part of the Android app that opens the wallpaper setup (WallpaperBridge.kt).
# It only exists in the app exported with the Project P Wallpaper add-on.
const WALLPAPER_PLUGIN := "ProjectPWallpaper"
# Kinds of picture files that are accepted as splash art.
const PICTURE_TYPES := ["png", "jpg", "jpeg", "webp"]
# Same save file as pet.gd; the chosen pet is stored under [game] chosen_pet.
const SAVE_PATH := "user://pet.cfg"

@onready var pet: Node2D = get_node(pet_path)
var _panel: Control = null # the open grid (null = closed)
var _own_buttons: Control = null # "Change pet" and "Wallpaper" (only without a menu)
var _menu_mode := false # true when a main menu is in charge
# ===== SECTION 2: START-UP =====
func _ready() -> void:
    layer = 10 # above the main menu (layer 5)
    _add_own_buttons()
    # Wait until every node is ready (so pet.gd has loaded its first pet and the main
    # menu, if there is one, has taken charge), then switch to the saved pet.
    _apply_saved_choice.call_deferred()

# Called by main_menu.gd: the menu takes charge, so this script's own buttons and its
# opening by itself on the first start are switched off.
func set_menu_mode(on: bool) -> void:
    _menu_mode = on
    if _own_buttons != null:
        _own_buttons.visible = not on

# Switches to the pet the player chose last time. Without a main menu it also opens
# the grid on the very first start.
func _apply_saved_choice() -> void:
    var chosen := ""
    var cfg := ConfigFile.new()

```

```

if cfg.load(SAVE_PATH) == OK:
    chosen = cfg.get_value("game", "chosen_pet", "")
    if chosen != "" and _pets().has(chosen):
        if chosen != pet.get("pet_id"):
            pet.set_pet(chosen)
    elif ask_on_first_start and not _menu_mode and _pets().size() > 1:
        open_picker()

# ===== SECTION 3: ITS OWN BUTTONS (ONLY WITHOUT A MAIN MENU) =====
# [EDIT] Change position (top-left corner, 16px in), spacing or font size here.
func _add_own_buttons() -> void:
    var column := VBoxContainer.new() # stacks the buttons under each other
    column.position = Vector2(16, 16)
    column.add_theme_constant_override("separation", 12)
    add_child(column)
    _own_buttons = column

    var b := Button.new()
    b.text = change_button_text
    b.add_theme_font_size_override("font_size", 28)
    b.pressed.connect(open_picker)
    b.visible = _pets().size() > 1 # no point showing it with only one pet
    column.add_child(b)

# "Wallpaper": opens the wallpaper setup. Only shown where it can work (the phone).
var w := Button.new()
w.text = wallpaper_button_text
w.add_theme_font_size_override("font_size", 28)
w.pressed.connect(open_wallpaper_setup)
w.visible = Engine.has_singleton(WALLPAPER_PLUGIN)
column.add_child(w)

# Opens the wallpaper setup screens (pet on the home screen and lock screen).
# Other scripts can call it too, e.g. $PetPicker.open_wallpaper_setup()
# [FIX] Nothing happens on the phone: export with the add-on enabled and Use Gradle
# Build on (see the guide, Part C).
func open_wallpaper_setup() -> void:
    if Engine.has_singleton(WALLPAPER_PLUGIN):
        Engine.get_singleton(WALLPAPER_PLUGIN).openWallpaperSetup()

# ===== SECTION 4: THE PET GRID =====
# Opens the grid: a title, one splash-art tile per pet, and a Back button.
# Other scripts can open it too, e.g. $PetPicker.open_picker()
func open_picker() -> void:
    if _panel != null:
        return # already open
    _set_pet_shown(false) # the pet (or its egg) is hidden during selection
    # Overlay over the whole screen; it also stops taps reaching anything behind it.
    # With a main menu it only darkens the menu's background a little. Without one it
    # shows its own picture (or a dark screen if that picture is missing).
    var picture: Texture2D = null if _menu_mode else _background_picture()
    var overlay := ColorRect.new()
    overlay.color = Color(0, 0, 0, BACKGROUND_DARKEN if _menu_mode else 0.85)
    overlay.set_anchors_preset(Control.PRESET_FULL_RECT)
    overlay.mouse_filter = Control.MOUSE_FILTER_STOP
    add_child(overlay)
    _panel = overlay
    if picture != null:
        # The picture fills the screen without stretching (edges that don't fit are cut)
        var bg := TextureRect.new()

```

```

        bg.texture = picture
        bg.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
        bg.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_COVERED
        bg.set_anchors_preset(Control.PRESET_FULL_RECT)
        bg.mouse_filter = Control.MOUSE_FILTER_IGNORE
        if SMOOTH_PICTURES:
            bg.texture_filter = CanvasItem.TEXTURE_FILTER_LINEAR
        overlay.add_child(bg)
        var shade := ColorRect.new()
        shade.color = Color(0, 0, 0, BACKGROUND_DARKEN)
        shade.set_anchors_preset(Control.PRESET_FULL_RECT)
        shade.mouse_filter = Control.MOUSE_FILTER_IGNORE
        overlay.add_child(shade)

var center := CenterContainer.new()
center.set_anchors_preset(Control.PRESET_FULL_RECT)
overlay.add_child(center)
var box := VBoxContainer.new()
box.add_theme_constant_override("separation", 16)
center.add_child(box)
var title := Label.new()
title.text = title_text
title.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
title.add_theme_font_size_override("font_size", 40)
# A dark edge around the letters so the title reads on any background
title.add_theme_color_override("font_outline_color", TITLE_EDGE_COLOR)
title.add_theme_constant_override("outline_size", 10)
box.add_child(title)

# The grid of pets: one tile per pet in PETS (pet.gd, SECTION 0), so it grows
# by itself when pets are added.
var pets := _pets()
var cols := clampi(pets.size(), 1, columns) # fewer pets than columns: stay centred
var grid := GridContainer.new()
grid.columns = cols
grid.add_theme_constant_override("h_separation", GAP)
grid.add_theme_constant_override("v_separation", GAP)
for id in pets:
    grid.add_child(_make_tile(String(id), String(pets[id])))
if pets.is_empty():
    # Nothing to choose from: say why on the screen instead of an empty grid.
    # [FIX] This shows when pet.gd has an error (red lines in Godot's Output panel,
    #       the Pet node then has no working script) or its PETS list is empty.
    var problem := Label.new()
    problem.text = "No pets found.\npet.gd has an error or its PETS list is empty.\n" \
        + "Look for red lines in Godot's Output panel."
    problem.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    problem.add_theme_font_size_override("font_size", FONT_SIZE)
    problem.add_theme_color_override("font_outline_color", TITLE_EDGE_COLOR)
    problem.add_theme_constant_override("outline_size", 8)
    box.add_child(problem)

# Put the grid in a scroll area. It's only as tall as the grid needs, up to the
# screen height minus room for the title and Back; beyond that it scrolls.
# [EDIT] 220 = space kept free for the title and the Back button.
var rows := ceil(pets.size() / float(cols))
var grid_height := rows * TILE_SIZE.y + maxi(rows - 1, 0) * GAP
var max_height := get_viewport().get_visible_rect().size.y - 220
var scroll := ScrollContainer.new()

```

```

scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
scroll.custom_minimum_size = Vector2(
    cols * TILE_SIZE.x + (cols - 1) * GAP, minf(grid_height, max_height))
scroll.add_child(grid)
box.add_child(scroll)

var back := Button.new()
back.text = back_button_text
back.add_theme_font_size_override("font_size", FONT_SIZE)
back.pressed.connect(close_picker)
box.add_child(back)

# Closes the grid without changing the pet.
# [LOGIC] With a main menu: back to the menu (the pet stays hidden).
#           Without one: back to the pet.
func close_picker() -> void:
    if _panel == null:
        return
    _panel.queue_free()
    _panel = null
    if not _menu_mode:
        _set_pet_shown(true)
    cancelled.emit()

func is_open() -> bool:
    return _panel != null

# [LOGIC] The player tapped a pet's splash art: switch to that pet, remember the
#           choice, close the grid and go to the pet.
func _choose(id: String) -> void:
    pet.set_pet(id)
    var cfg := ConfigFile.new()
    cfg.load(SAVE_PATH) # keep the pets' data already in the file
    cfg.set_value("game", "chosen_pet", id)
    cfg.save(SAVE_PATH)
    if _panel != null:
        _panel.queue_free()
        _panel = null
    _set_pet_shown(true)
    pet_chosen.emit(id) # the main menu closes itself when it hears this
# ===== SECTION 5: ONE PET'S TILE =====
# A tile is a button filled with the pet's splash art, a frame around it, and the
# pet's name over the bottom (if show_names is on). The pet chosen now gets a thicker
# frame in another colour.
# [FIX] A tile shows a small pet frame instead of your splash art: the picture isn't in
#       SPLASH_FOLDER, or its name isn't exactly the pet's id in PETS (dragon.png for
#       "dragon", same small letters).
func _make_tile(id: String, folder: String) -> Control:
    var tile := Button.new()
    tile.flat = true # no grey button look
    tile.custom_minimum_size = TILE_SIZE
    tile.clip_contents = true
    tile.pressed.connect(_choose.bind(id))
    # Darken the tile a little while a finger is on it
    tile.button_down.connect(func(): tile.modulate = Color(0.75, 0.75, 0.75))
    tile.button_up.connect(func(): tile.modulate = Color.WHITE)

    var splash := _splash_for(id)
    var art := TextureRect.new()

```

```

art.set_anchors_preset(Control.PRESET_FULL_RECT)
art.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
art.mouse_filter = Control.MOUSE_FILTER_IGNORE
if splash != null:
    art.texture = splash
    art.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_COVERED # fill the tile
    if SMOOTH_PICTURES:
        art.texture_filter = CanvasItem.TEXTURE_FILTER_LINEAR
else:
    # No splash art yet: a soft panel with the pet's first idle frame in the middle
    var plain := ColorRect.new()
    plain.color = Color(0, 0, 0, 0.35)
    plain.set_anchors_preset(Control.PRESET_FULL_RECT)
    plain.mouse_filter = Control.MOUSE_FILTER_IGNORE
    tile.add_child(plain)
    art.texture = _picture_for(folder)
    art.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
tile.add_child(art)

# The frame around the tile (thicker and coloured for the pet chosen now)
var is_current: bool = (id == pet.get("pet_id"))
var edge := StyleBoxFlat.new()
edge.bg_color = Color(0, 0, 0, 0)
edge.border_color = CURRENT_EDGE_COLOR if is_current else TILE_EDGE_COLOR
edge.set_border_width_all(CURRENT_EDGE_WIDTH if is_current else TILE_EDGE_WIDTH)
var frame := Panel.new()
frame.add_theme_stylebox_override("panel", edge)
frame.set_anchors_preset(Control.PRESET_FULL_RECT)
frame.mouse_filter = Control.MOUSE_FILTER_IGNORE
tile.add_child(frame)

if show_names or splash == null:
    var name_label := Label.new()
    name_label.text = id.capitalize()
    name_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    name_label.add_theme_font_size_override("font_size", FONT_SIZE)
    name_label.add_theme_color_override("font_outline_color", TILE_EDGE_COLOR)
    name_label.add_theme_constant_override("outline_size", 8)
    name_label.set_anchors_preset(Control.PRESET_BOTTOM_WIDE)
    name_label.grow_vertical = Control.GROW_DIRECTION_BEGIN
    name_label.offset_bottom = -14
    name_label.mouse_filter = Control.MOUSE_FILTER_IGNORE
    tile.add_child(name_label)
return tile

# ===== SECTION 6: HELPERS =====
# The pet list (PETS) from pet.gd, so this script never needs its own copy.
# Empty if pet.gd isn't on the Pet node or has an error.
func _pets() -> Dictionary:
    var script = pet.get_script()
    if not (script is Script) or not pet.has_method("set_pet"):
        return {}
    var pets = script.get_script_constant_map().get("PETS", {})
    return pets if pets is Dictionary else {}

# Shows or hides the pet. A hidden pet is also paused, so it doesn't wander off or
# react to taps while the grid is in front of it.
func _set_pet_shown(shown: bool) -> void:
    pet.visible = shown
    pet.process_mode = Node.PROCESS_MODE_INHERIT if shown else Node.PROCESS_MODE_DISABLED

```

```
# A pet's splash art: SPLASH_FOLDER/<id>.png (or .jpg, .jpeg, .webp); null if none.
func _splash_for(id: String) -> Texture2D:
    for type in PICTURE_TYPES:
        var path := "%s/%s.%s" % [SPLASH_FOLDER, id, type]
        if ResourceLoader.exists(path):
            return load(path) as Texture2D
    return null

# The picture behind the grid when there is no main menu (null if the file isn't there).
func _background_picture() -> Texture2D:
    if ResourceLoader.exists(BACKGROUND_PICTURE):
        return load(BACKGROUND_PICTURE) as Texture2D
    return null

# The stand-in for missing splash art: the pet's first idle frame (or its egg).
# [FIX] No picture at all? Check that pet's frames load in the game (Output panel).
func _picture_for(folder: String) -> Texture2D:
    var groups: Dictionary = pet._frame_groups(folder)
    for anim in ["pet_idle", "egg_idle"]:
        var paths: Array = groups.get(anim, [])
        if not paths.is_empty():
            return load(paths[0])
    return null
```

A6. The main menu

`main_menu.gd` adds the screen the app opens on:

- **Your animated background**, 16 frames played in a loop.
- **A Pet Selection button**. It opens the pet grid from step A5. Choosing a pet's splash art closes the menu and shows that pet.
- **A Wallpaper Selection button**. On the phone it opens the wallpaper setup (Part C). On the computer it shows a short message instead, because the wallpaper only exists in the Android app.
- **Each button plays its 3 frames** when it is tapped or clicked, then does its job.
- **The pet and its egg are hidden** while the menu or the pet grid is open.
- **On the pet's screen** a small **Menu** button (top left) brings the menu back. The phone's Back button does the same: pet to menu, pet grid to menu, menu to exit.

Step 1 - Add the menu to the scene

- 1 Drag `main_menu.gd` from your computer into the **FileSystem** panel, next to `pet.gd`.
- 2 Open `main.tscn`. Right-click **Main**, choose **Add Child Node**, search for **CanvasLayer**, and click **Create**. Rename it **MainMenu**.
- 3 Drag `main_menu.gd` from the FileSystem panel onto the **MainMenu** node.
- 4 Save with **Ctrl+S** and press **F5**.

Check: the Scene panel shows **Main > Pet**, **PetPicker** and **MainMenu** side by side. The game opens on a plain-coloured menu with two grey buttons, **Pet Selection** and **Wallpaper Selection**, and no egg in sight. The buttons already work; the next steps give them your art. The Output panel shows **Project P: main menu ready, 0 background frames**.

Step 2 - Try it with the sample art (optional)

- 1 Unzip `ProjectP_Menu_Assets.zip`. It contains one folder named `menu`.
- 2 In your computer's file explorer, copy that `menu` folder into your Godot project folder, next to `project.godot`.
- 3 Go back to Godot, wait for the import to finish, and press **F5**.

Check: the menu shows the living room with gently moving light and two brown buttons that sink in when clicked. These are placeholders with the right names and sizes; the next steps replace them with yours.

Step 3 - Insert your own background with 16 frames

- 1 Save your 16 frames as `bg_00.jpg`, `bg_01.jpg`, ... `bg_15.jpg` (two digits, so `bg_02` comes before `bg_10`). JPG, PNG or WebP all work.
- 2 Make them all the same size, portrait: **720 x 1280 pixels**. The last frame should lead back into the first, because they play in a loop.
- 3 In the **FileSystem** panel, open `res://menu/background/` (create the folders `menu` and then `background` with **New > Folder** if you skipped Step 2).
- 4 If the sample frames are there, select them all (click the first, Shift+click the last), right-click, and choose **Delete**.
- 5 Drag your 16 frames from your computer into `res://menu/background/`.
- 6 Keep the app small: select all 16 frames in the FileSystem panel, open the **Import** tab (next to the Scene tab, top left), set **Compress > Mode** to **Lossy**, and click **Reimport**. This makes each frame about a seventh of the size, with no visible difference for a background.
- 7 Press **F5**.

Check: the Output panel shows `Project P: main menu ready, 16 background frames.` and the menu plays your background.

- **Speed:** select the **MainMenu** node and change **Background Fps** in the Inspector. 16 frames at 8 make a 2-second loop; at 4, a 4-second loop.
- **More or fewer frames:** any number works; every picture in the folder is played in the order of its name. One picture gives a still background.
- **Bigger frames:** 1080 x 1920 looks sharper on large phones but uses more than twice the memory (about 130 MB instead of 60 MB while the menu is open). Stay with 720 x 1280 unless it looks soft on your phone.
- **Pixel-art menu:** set `SMOOTH_PICTURES` to `false` in SECTION 1 of `main_menu.gd`.

Step 4 - Insert your own two buttons (3 frames each)

- 1 Draw each button in 3 frames: `_00` is the button at rest (the way it looks on the menu), `_01` is the button going down, and `_02` is the button fully pressed.
- 2 Save them as PNG with a transparent background, all 3 frames of a button the same size (for example 480 x 150 pixels).
- 3 Name the Pet Selection frames exactly `pet_selection_00.png`, `pet_selection_01.png`, `pet_selection_02.png`.
- 4 Name the Wallpaper Selection frames exactly `wallpaper_selection_00.png`, `wallpaper_selection_01.png`, `wallpaper_selection_02.png`.
- 5 In the **FileSystem** panel, open `res://menu/buttons/` (create it with **New > Folder** if needed). Delete the sample buttons with the same names if they are there.
- 6 Drag your 6 pictures into `res://menu/buttons/`.
- 7 Press **F5** and click each button.

Check: each button shows its `_00` picture, plays `_01` then `_02` when clicked, and then opens its screen: the pet grid for Pet Selection, the message "The wallpaper setup opens on the phone." for Wallpaper Selection (on the phone, the wallpaper setup).

- **Size on screen:** select **MainMenu** and change **Button Width** (440 of the 720-wide screen). The height follows your picture's shape.
- **Position: Buttons Center Y** is where the pair sits: 0.5 is the middle of the screen, 0.68 the lower part, 0.8 near the bottom. **Button Gap** is the space between them.
- **Animation speed: Press Fps.** 3 frames at 12 take a quarter of a second before the button acts.
- **More frames:** add `_03`, `_04`, ... (up to `_09`); they are played in order.
- **The small Menu button** on the pet's screen works the same way: `menu_00.png`, `menu_01.png`, `menu_02.png` in the same folder (for example 240 x 96 pixels), with **Menu Button Width** for its size. Without pictures it is a grey button with the text "Menu".

Step 5 - Check the whole flow

- 1 Press **F5**. The menu opens with your background; no pet or egg is visible.
- 2 Click **Pet Selection**. The button animates, then the grid of your 6 pets opens over the moving background.
- 3 Click a pet's splash art. The grid and the menu close and that pet appears (as an egg if it hasn't hatched yet).
- 4 Click **Menu** (top left). The menu returns and the pet is hidden again.
- 5 Click **Pet Selection**, then **Back**. You are on the menu again with both buttons.

To see the first start again while testing: **Project > Open User Data Folder**, then delete `pet.cfg`.

If something is wrong:

- **The egg shows through the menu, or there is no menu at all:** the script isn't on a node named `MainMenu` under `Main`, or the node isn't a `CanvasLayer`. Repeat Step 1.
- **Pet Selection shows "Pet grid not found":** `pet_picker.gd` isn't in the project's top folder next to `pet.gd`. Put it there, or change `PICKER_SCRIPT` in SECTION 1 of `main_menu.gd` to where it is.
- **Pet Selection does nothing or shows a message:** a message on the menu says what is missing (the `pet_picker.gd` file, or the new version of it on the `PetPicker` node). If the grid opens but says "No pets found", `pet.gd` has an error or an empty `PETS` list: look for red lines in the Output panel.
- **Plain colour instead of the background:** the frames aren't in `res://menu/background/`. The Output line says how many were found.
- **A grey text button instead of your button art:** the file names don't match. They must be exactly `pet_selection_00.png` and `wallpaper_selection_00.png` (and `_01`, `_02`), in `res://menu/buttons/`.
- **The old "Change pet" button is still at the top left:** the project still has the old `pet_picker.gd`. Replace it with the new file (step A5).
- **The button art looks blocky or blurry:** draw it at about the size it is shown (440 wide on a 720-wide screen), or twice that for extra sharpness.

For reference, this is what `main_menu.gd` contains (add the file itself to Godot):

```

# =====
# main_menu.gd - THE MAIN MENU (the first screen when the app opens)
# =====
# What this file does:
#   Shows the main menu as soon as the game starts:
#   - your animated background (16 frames, or however many you put in the folder),
#   - a "Pet Selection" button -> opens the pet grid (pet_picker.gd); choosing a
#       pet closes the menu and shows that pet,
#   - a "Wallpaper Selection" button -> opens the wallpaper setup on the phone.
# Each button plays its own frames (3 by default) when it is tapped or clicked, and
# then does its job.
# While the menu or the pet grid is open, the pet (or its egg) is hidden and paused.
# On the pet's screen a small "Menu" button (top left) brings the menu back. On the
# phone, the Back button does the same: pet -> menu, pet grid -> menu, menu -> exit.
#
# Where it goes:
#   On a CanvasLayer node called "MainMenu", added as a child of "Main" (next to "Pet"
#   and "PetPicker"). If the scene has no "PetPicker" node, the menu makes one itself
#   from pet_picker.gd, so that file only has to be in the project.
#
# Your pictures (all optional; without them plain buttons and a plain colour are used):
#   res://menu/background/   bg_00.jpg ... bg_15.jpg           the animated background
#   res://menu/buttons/      pet_selection_00.png ... _02.png    Pet Selection
#                           wallpaper_selection_00.png ... _02.png Wallpaper Selection
#                           menu_00.png ... _02.png              the small Menu button
#   res://menu/splash/       dragon.png, cat.png ...            see pet_picker.gd
#
# How to find things:
#   [EDIT] = a setting you can safely change (folders, sizes, speeds, texts)
#   [LOGIC] = what leads to what
#   [FIX] = places to look first if something goes wrong
#
# Sections in this file:
#   1. Settings -> search "SECTION 1"
#   2. Start-up -> search "SECTION 2"
#   3. Opening and closing the menu -> search "SECTION 3"
#   4. The animated background -> search "SECTION 4"
#   5. The buttons and their press animation -> search "SECTION 5"
#   6. What the buttons do -> search "SECTION 6"
#   7. The phone's Back button -> search "SECTION 7"
#   8. Helpers -> search "SECTION 8"
# =====

extends CanvasLayer

# ===== SECTION 1: SETTINGS =====
# [FIX] Where the Pet and PetPicker nodes are, seen from this node.
@export var pet_path: NodePath = ^"../Pet"
@export var picker_path: NodePath = ^"../PetPicker"
# [EDIT] The pet grid's script. If the scene has no PetPicker node, the menu makes one
#         itself from this file, so the Pet Selection button always has a grid to open.
const PICKER_SCRIPT := "res://pet_picker.gd"

# --- Background ---
# [EDIT] The folder with the background frames. They are played in the order of their
#         names (bg_00, bg_01, ... bg_15). JPG, PNG or WebP. One picture = a still
#         background.
const BACKGROUND_FOLDER := "res://menu/background"
# [EDIT] Speed of the background, in frames per second. 16 frames at 8 = a 2-second loop.
@export var background_fps := 8.0

```

```

# [EDIT] Colour shown if there are no background pictures.
const BACKGROUND_COLOR := Color(0.95, 0.89, 0.81)

# --- Buttons ---
# [EDIT] The folder with the button pictures, and each button's file name. A button's
#       frames are <name>_00.png (how it looks at rest), <name>_01.png, <name>_02.png
#       (being pressed). More or fewer frames work too: _00 up to _09.
const BUTTON_FOLDER := "res://menu/buttons"
const PET_BUTTON := "pet_selection"
const WALLPAPER_BUTTON := "wallpaper_selection"
const MENU_BUTTON := "menu"
# [EDIT] How wide the two big buttons are drawn, in pixels of the 720-wide screen. The
#       height follows from your picture's shape.
@export var button_width := 440.0
# [EDIT] Space between the two buttons.
@export var button_gap := 28
# [EDIT] Where the pair of buttons sits: 0.5 = middle of the screen, 0.68 = lower part,
#       0.8 = near the bottom.
@export var buttons_center_y := 0.68
# [EDIT] Speed of the press animation, in frames per second. 3 frames at 12 = a quarter
#       of a second before the button does its job.
@export var press_fps := 12.0
# [EDIT] Width of the small "Menu" button on the pet's screen.
@export var menu_button_width := 150.0
# [EDIT] Text on the buttons when they have no pictures yet.
@export var pet_button_text := "Pet Selection"
@export var wallpaper_button_text := "Wallpaper Selection"
@export var menu_button_text := "Menu"
# [EDIT] Message shown when Wallpaper Selection is used on the computer.
@export var computer_notice := "The wallpaper setup opens on the phone."

# [EDIT] true = the menu's pictures are drawn smooth (right for painted art). Set to
#       false if your menu art is pixel art and should keep hard pixel edges.
const SMOOTH_PICTURES := true

# The part of the Android app that opens the wallpaper setup (WallpaperBridge.kt). It
# only exists in the app exported with the Project P Wallpaper add-on.
const WALLPAPER_PLUGIN := "ProjectPWallpaper"
# Kinds of picture files that are accepted.
const PICTURE_TYPES := ["png", "jpg", "jpeg", "webp"]
@onready var pet: Node2D = get_node_or_null(pet_path)
@onready var picker: Node = get_node_or_null(picker_path)

var _root: Control = null          # everything of the menu (background + buttons)
var _background: TextureRect = null # shows the current background frame
var _buttons: Control = null       # holds the two big buttons
var _menu_button: BaseButton = null # the small "Menu" button on the pet's screen
var _notice: Label = null          # the message for computer users
var _open := false                # is the menu on screen?
var _bg_paths: Array = []          # the background frame files, in order
var _bg_frames: Array = []         # the frames loaded so far
var _bg_time := 0.0               # seconds the background has been playing
var _button_busy := false          # a button's press animation is playing

# ===== SECTION 2: START-UP =====
func _ready() -> void:
    layer = 5                      # above the game, below the pet grid (layer 10)
    _bg_paths = _picture_files(BACKGROUND_FOLDER)
    _build_menu()

```

```

# The pet grid belongs to the menu. No PetPicker node in the scene: make one.
if picker == null:
    _make_picker()
else:
    _connect_picker()
# On the phone, the Back button is handled in SECTION 7 instead of closing the app.
get_tree().quit_on_go_back = false
open_menu() # the app starts on the menu
print("Project P: main menu ready, %d background frames." % _bg_paths.size())

# Makes the PetPicker node (a CanvasLayer with pet_picker.gd) next to this node.
# It is added a moment later, because the scene is still being set up right now.
# [FIX] "pet_picker.gd not found": put pet_picker.gd in the project's top folder, next
# to pet.gd, or change PICKER_SCRIPT (SECTION 1) to where it is.
func _make_picker() -> void:
    if not ResourceLoader.exists(PICKER_SCRIPT):
        push_warning("Project P: no PetPicker node, and %s not found." % PICKER_SCRIPT)
        return
    var made := CanvasLayer.new()
    made.name = "PetPicker"
    made.set_script(load(PICKER_SCRIPT))
    picker = made
    get_parent().add_child.call_deferred(made)
    _connect_picker.call_deferred()
    print("Project P: main menu made its own PetPicker node from %s." % PICKER_SCRIPT)

# Tells the pet grid that the menu is in charge, and listens to what the player does.
func _connect_picker() -> void:
    if picker.has_method("set_menu_mode"):
        picker.set_menu_mode(true)
    if picker.has_signal("pet_chosen") and not picker.pet_chosen.is_connected(_on_pet_chosen):
        picker.pet_chosen.connect(_on_pet_chosen)
    if picker.has_signal("cancelled") and not picker.cancelled.is_connected(_on_picker_cancelled):
        picker.cancelled.connect(_on_picker_cancelled)

# Builds the menu's parts once. They are shown and hidden later.
func _build_menu() -> void:
    # Covers the whole screen and stops taps from reaching the pet
    _root = ColorRect.new()
    _root.color = BACKGROUND_COLOR
    _root.set_anchors_preset(Control.PRESET_FULL_RECT)
    _root.mouse_filter = Control.MOUSE_FILTER_STOP
    if SMOOTH_PICTURES:
        _root.texture_filter = CanvasItem.TEXTURE_FILTER_LINEAR # also for everything in it
    add_child(_root)

    # The background picture: fills the screen without stretching (edges that don't
    # fit are cut off evenly)
    _background = TextureRect.new()
    _background.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    _background.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_COVERED
    _background.set_anchors_preset(Control.PRESET_FULL_RECT)
    _background.mouse_filter = Control.MOUSE_FILTER_IGNORE
    _root.add_child(_background)

    # The two big buttons, one under the other, centred sideways
    var column := VBoxContainer.new()
    column.add_theme_constant_override("separation", button_gap)
    column.add_child(_make_button(PET_BUTTON, pet_button_text, button_width,
        _open_pet_selection))

```

```

column.add_child(_make_button(WALLPAPER_BUTTON, wallpaper_button_text, button_width,
    _open_wallpaper_selection))
# A thin strip across the screen at the chosen height; the buttons are centred on it
var holder := CenterContainer.new()
holder.anchor_left = 0.0
holder.anchor_right = 1.0
holder.anchor_top = buttons_center_y
holder.anchor_bottom = buttons_center_y
holder.mouse_filter = Control.MOUSE_FILTER_IGNORE
holder.add_child(column)
_root.add_child(holder)
_buttons = holder

# The message for computer users (hidden until needed)
_notice = Label.new()
_notice.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
_notice.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART # long messages wrap
_notice.add_theme_font_size_override("font_size", 28)
_notice.add_theme_color_override("font_outline_color", Color(0.2, 0.13, 0.09))
_notice.add_theme_constant_override("outline_size", 10)
_notice.anchor_left = 0.0
_notice.anchor_right = 1.0
_notice.anchor_top = 0.9
_notice.anchor_bottom = 0.9
_notice.mouse_filter = Control.MOUSE_FILTER_IGNORE
_notice.visible = false
_root.add_child(_notice)

# The small "Menu" button on the pet's screen (top left)
# [EDIT] Change its position here (16 pixels in from the top-left corner).
_menu_button = _make_button(MENU_BUTTON, menu_button_text, menu_button_width, open_menu)
_menu_button.position = Vector2(16, 16)
if SMOOTH_PICTURES:
    _menu_button.texture_filter = CanvasItem.TEXTURE_FILTER_LINEAR
add_child(_menu_button)

# ===== SECTION 3: OPENING AND CLOSING THE MENU =====
# Shows the menu and hides the pet. Other scripts can call it: $MainMenu.open_menu()
func open_menu() -> void:
    _open = true
    _root.visible = true
    _buttons.visible = true
    _menu_button.visible = false
    _set_pet_shown(false) # the pet (or its egg) is never visible on the menu
    _start_background()

# Hides the menu and shows the pet.
func close_menu() -> void:
    _open = false
    _root.visible = false
    _menu_button.visible = true
    _set_pet_shown(true)
    # Give the background's memory back while the player is with the pet
    _bg_frames.clear()
    _background.texture = null

func is_open() -> bool:
    return _open

# Shows or hides the pet. A hidden pet is also paused, so it doesn't wander off or

```

```

# react to taps while the menu is in front of it.
func _set_pet_shown(shown: bool) -> void:
    if pet == null:
        return
    pet.visible = shown
    pet.process_mode = Node.PROCESS_MODE_INHERIT if shown else Node.PROCESS_MODE_DISABLED

# ===== SECTION 4: THE ANIMATED BACKGROUND =====
# [LOGIC] The first frame is shown at once. The others are loaded one per moment
#         while the menu is already on screen, so opening the menu never stalls. When
#         all are loaded, they play in a loop at background_fps.
# [FIX] Plain colour instead of your pictures: check they are in BACKGROUND_FOLDER
#         and read the "Project P: main menu ready" line in the Output panel.
func _start_background() -> void:
    _bg_time = 0.0
    if _bg_frames.is_empty() and not _bg_paths.is_empty():
        _bg_frames.append(load(_bg_paths[0]))
    if not _bg_frames.is_empty():
        _background.texture = _bg_frames[0]
func _process(delta: float) -> void:
    if not _root.visible or _bg_paths.is_empty():
        return
    if _bg_frames.size() < _bg_paths.size():
        _bg_frames.append(load(_bg_paths[_bg_frames.size()])) # one more frame
    return
    if _bg_frames.size() > 1:
        _bg_time += delta
        var index := int(_bg_time * background_fps) % _bg_frames.size()
        _background.texture = _bg_frames[index]

# ===== SECTION 5: THE BUTTONS AND THEIR PRESS ANIMATION =====
# Makes one button. With pictures (<name>_00.png, _01, _02 in BUTTON_FOLDER) it is a
# picture button that plays its frames when pressed; without them it is a plain button
# with text, so the menu works before your art is ready.
# `action` is what the button does after its animation.
func _make_button(file_name: String, text: String, width: float,
    action: Callable) -> BaseButton:
    var frames := _button_frames(file_name)
    if frames.is_empty():
        var plain := Button.new()
        plain.text = text
        plain.add_theme_font_size_override("font_size", 30)
        plain.custom_minimum_size = Vector2(width, 0)
        plain.pressed.connect(action)
        return plain
    var b := TextureButton.new()
    b.texture_normal = frames[0]
    b.ignore_texture_size = true
    b.stretch_mode = TextureButton.STRETCH_KEEP_ASPECT_CENTERED
    var picture_size: Vector2 = frames[0].get_size()
    b.custom_minimum_size = Vector2(width, width * picture_size.y / picture_size.x)
    b.size = b.custom_minimum_size
    b.pressed.connect(_play_button.bind(b, frames, action))
    return b

# [LOGIC] A tap or click plays the button's frames in order (_01, _02, ...), goes back
#         to _00, and then does the button's job. Other taps are ignored meanwhile.
func _play_button(b: TextureButton, frames: Array, action: Callable) -> void:
    if _button_busy:

```

```

        return
        _button_busy = true
    for i in range(1, frames.size()):
        b.texture_normal = frames[i]
        await get_tree().create_timer(1.0 / press_fps).timeout
    b.texture_normal = frames[0]
    _button_busy = false
    action.call()

# A button's frames: <name>_00, <name>_01, ... as far as they exist (empty if none).
func _button_frames(file_name: String) -> Array:
    var frames := []
    for i in range(10):
        var found := _find_picture(BUTTON_FOLDER.path_join("%s_%02d" % [file_name, i]))
        if found == "":
            break
        frames.append(load(found))
    return frames

# ===== SECTION 6: WHAT THE BUTTONS DO =====
# [LOGIC] Pet Selection: the buttons step aside and the pet grid opens on top of the
#     animated background. Choosing a pet -> the menu closes and that pet is shown.
#     "Back" in the grid -> the menu's buttons return.
# [FIX] The button shows a message instead of the grid: the message says what is
#     missing. The grid is the "PetPicker" node with pet_picker.gd on it; if the scene
#     has none, the menu makes it from PICKER_SCRIPT (SECTION 1).
func _open_pet_selection() -> void:
    if picker == null:
        _show_notice("Pet grid not found: put pet_picker.gd next to pet.gd (step A5).")
        return
    if not picker.has_method("open_picker") or not picker.has_signal("pet_chosen"):
        _show_notice("The PetPicker node needs the new pet_picker.gd on it (step A5).")
        return
    _buttons.visible = false
    picker.open_picker()

func _on_pet_chosen(_id: String) -> void:
    close_menu() # straight to the chosen pet

func _on_picker_cancelled() -> void:
    if _open:
        _buttons.visible = true # back to the two buttons

# [LOGIC] Wallpaper Selection: opens the wallpaper setup screens on the phone (keep
#     your wallpaper or use the Project P wallpaper). The menu stays open behind.
# [FIX] Nothing happens on the phone: export with the Project P Wallpaper add-on
#     enabled and Use Gradle Build on (see the guide, Part C).
func _open_wallpaper_selection() -> void:
    if Engine.has_singleton(WALLPAPER_PLUGIN):
        Engine.get_singleton(WALLPAPER_PLUGIN).openWallpaperSetup()
    else:
        _show_notice(computer_notice)

# Shows a short message near the bottom of the menu for a few seconds.
func _show_notice(text: String) -> void:
    _notice.text = text
    _notice.visible = true
    await get_tree().create_timer(4.0).timeout
    if _notice.text == text: # (not replaced by a newer message meanwhile)
        _notice.visible = false

```

```
# ===== SECTION 7: THE PHONE'S BACK BUTTON =====
# [LOGIC] Back on the pet grid -> the menu. Back on the pet's screen -> the menu.
# Back on the menu -> closes the app.
func _notification(what: int) -> void:
    if what != NOTIFICATION_WM_GO_BACK_REQUEST:
        return
    if picker != null and picker.has_method("is_open") and picker.is_open():
        picker.close_picker()
    elif _open:
        get_tree().quit()
    else:
        open_menu()

# ===== SECTION 8: HELPERS =====
# All picture files in a folder, in the order of their names (full paths).
# Works in the editor and in exported games, where Godot keeps only "<name>.import".
func _picture_files(folder: String) -> Array:
    var result := []
    var dir := DirAccess.open(folder)
    if dir == null:
        return result
    for f in dir.get_files():
        var fname := f.trim_suffix(".import").trim_suffix(".remap")
        if fname.get_extension().to_lower() in PICTURE_TYPES:
            var path := folder.path_join(fname)
            if not result.has(path) and ResourceLoader.exists(path):
                result.append(path)
    result.sort_custom(func(a, b): return a.naturalnocasecmp_to(b) < 0)
    return result

# The picture file with this name, whatever its type (" " if there is none).
# `path_without_type` is e.g. "res://menu/buttons/menu_00".
func _find_picture(path_without_type: String) -> String:
    for type in PICTURE_TYPES:
        var path := "%s.%s" % [path_without_type, type]
        if ResourceLoader.exists(path):
            return path
    return ""
```

A7. Get your computer ready for Android export (one-time)

- 1 Install **JDK 17** (for example Temurin from adoptium.net). It usually ends up in **C:\Program Files\Eclipse Adoptium\jdk-17...-hotspot**.
- 2 Install **Android Studio**. Open its **SDK Manager** (Welcome screen > **More Actions** > **SDK Manager**, or **Tools** > **SDK Manager**) and install: a recent **Android SDK Platform**, **Android SDK Build-Tools**, **Android SDK Platform-Tools**, and **Android SDK Command-line Tools**. The box **Android SDK Location** at the top of that window is your SDK path.
- 3 In Godot, go to **Editor > Editor Settings > Export > Android** and set **Java SDK Path** to the JDK's main folder, the one that contains a **bin** folder. Not the **bin** folder itself, and not **C:\Program Files\Eclipse Adoptium**.
- 4 In the same window, set **Android SDK Path**. It's usually **C:\Users\<you>\AppData\Local\Android\Sdk**. **AppData** is a hidden folder, so paste the path into the box instead of browsing to it. To find it, press **Windows + R**, type **%LOCALAPPDATA%\Android\Sdk**, press Enter, and copy the path from the address bar.
- 5 Go to **Editor > Manage Export Templates** and install the Android templates. In older Godot versions, click **Download and Install**. In newer versions the window has two lists: under **Available Templates**, tick **Android** (everything under it) and click the install button; Android then appears under **Installed Templates**. If a download fails, tick the remaining files and install again, or pick another mirror.
- 6 Go to **Project > Project Settings > Rendering > Textures > VRAM Compression** and tick **Import ETC2 ASTC**. Android needs this texture format.

Red messages in the export window and what they mean:

- "Target platform requires 'ETC2/ASTC' texture compression": step 6 isn't done. Click **Show Project Setting** next to the message and tick **Import ETC2 ASTC**.
- "Invalid Java SDK path... Missing 'bin' directory": the Java SDK Path is empty or points to the wrong folder (step 3).
- "Export templates missing": step 5 isn't finished for Android.

A8. Export and install on your phone

- 1 Go to **Project > Export... > Add... > Android**.
- 2 Set: **Architectures** - check **arm64-v8a**; **Package > Unique Name** - **com.yourname.projectp** (lowercase, no dashes); **Package > Name** - **Project P**; **Keystore** - leave empty for now (Godot uses its own debug key, fine for testing).
- 3 Click **Export Project...**, name the file **ProjectP.apk**, and leave **Export With Debug** checked.
- 4 Get the APK onto your phone with a USB cable: plug in the phone, choose **File transfer** in the phone's USB notification, and copy **ProjectP.apk** into the phone's **Download** folder. (Google Drive also works, but tap the three dots next to the file and choose **Download**; tapping the file itself often does nothing.)
- 5 On the phone, open the **Files** app, go to **Downloads**, tap the APK, and allow **Install unknown apps** when Android asks. If you see "Blocked by Play Protect", tap **More details > Install anyway**; that's normal for a test build.

Faster option: turn on Developer Options on your phone (Settings > About phone > tap **Build number** 7 times), then turn on **USB debugging** and plug in the phone. An Android icon appears at the top right of Godot. Click it to install and run the game in one step. This is worth setting up, because you'll reinstall many times while testing.

Part B - The live wallpaper as its own app (optional)

You can skip Part B. Part C packages the same wallpaper into the Godot game and doesn't need this Android Studio project. Do Part B only if you want the wallpaper as a separate small app, or want to try wallpaper changes quickly in Android Studio. The code files it uses are in **ProjectP_Wallpaper_Addon.zip**, in **addons/projectp_wallpaper/android/src/**. Leave out **WallpaperBridge.kt**: it belongs to the Godot game and doesn't compile in this project.

B1. Create the project

- 1 In Android Studio, go to **File > New > New Project** and choose **No Activity**.
- 2 Fill in: **Name** **Project P Wallpaper**; **Package name** **com.yourname.projectp.wallpaper** (must be different from the game's); **Language** Kotlin; **Minimum SDK API** 26.
- 3 Click **Finish** and wait for Gradle to finish loading.

B2. Add the pet's frames and default background

- 1 In the left panel, switch the view to **Project**. Right-click **app/src/main**, choose **New > Folder > Assets Folder**, and click **Finish**. This creates **app/src/main/assets**.
- 2 Right-click **assets**, choose **New > Directory**, and type **pets/dragon**. Copy the dragon's frame PNGs into **assets/pets/dragon/**, the same files as in the Godot app (no subfolder here). (Android's **res/drawable** folder can't hold a folder per pet, which is why the frames go in **assets**.)

- 3 Right-click `app/src/main/res`, choose **New > Android Resource Directory**, set **Resource type** to **drawable**, add the **Density** qualifier, and choose **No Density**. This creates `drawable-nodpi`.
- 4 Copy `pet_default_bg.png` (included with this guide) into `drawable-nodpi`. This is the Project P wallpaper: the living-room picture. It is the default background behind the pet and the picture behind the setup screens.

To use another picture, save it as a PNG with the same file name and replace this file. Keep it tall like a phone screen. The current picture is 941x1672 pixels; a larger version (about 1440x2560) looks sharper on big phones. For a pixel-art background, also set `defaultBackgroundIsPixelArt` to `true` in `PetWallpaperService.kt` (SECTION 2). For an animated Project P wallpaper, create the folder `app/src/main/assets/wallpaper_bg_frames/` and copy your frames into it; the rules for the frames are in step C10.

B3. Wallpaper description file

Right-click `res`, choose **New > Android Resource Directory**, and set the type to `xml`. Then right-click `xml`, choose **New > File**, name it `pet_wallpaper.xml`, and paste:

```
<?xml version="1.0" encoding="utf-8"?>
<wallpaper xmlns:android="http://schemas.android.com/apk/res/android"
    android:description="@string/wallpaper_description"
    android:thumbnail="@drawable/pet_default_bg" />
```

Open `res/values/strings.xml` and add this line inside `<resources>`:

```
<string name="wallpaper_description">Your Project P pet lives here.</string>
```

B4. The pet list and the wallpaper code

First, the list of pets. Right-click `app/src/main/java/com/yourname/projectp/wallpaper`, choose **New > Kotlin Class/File**, name it `PetCatalog`, and replace its contents with the file `PetCatalog.kt` (copy it into that folder, or open the file in a text editor and copy from there rather than from this PDF). This is the one place to add pets on Android.

```
// =====
// PetCatalog.kt - THE LIST OF ALL PETS (the one place to add a new pet)
// =====
// What this file does:
//   Lists every pet the wallpaper can show, the animations every pet's frames are split
//   into, and the things a pet does by itself (ACTIONS). PetWallpaperService.kt draws
//   the chosen pet; SetWallpaperActivity.kt lets the user pick which one. It also reads
//   the Godot game's save file when the wallpaper is packaged in the same app as the
//   game (see GameSave at the bottom).
//
// Where the pets, their frames, animations and actions come from:
//   - One app with the Godot game (Part C): AUTOMATIC, all from pet.gd. On every export
//   the Godot add-on reads PETS, ANIMS and ACTIONS from pet.gd, sorts every pet's PNGs
//   into animations exactly like the game does, and packs them into
//   assets/wallpaper_frames/. Nothing to edit here; the PETS, ANIMS and ACTIONS lists
//   below are only the stand-ins used when that information is missing.
//   - Wallpaper-only app (Part B): the PETS, ANIMS and ACTIONS lists below, with the
//   frames copied by hand into app/src/main/assets/pets/<name>/.
//
// >>> HOW TO ADD A NEW PET <<<
```

```

// One app with the game: add it to PETS in pet.gd (search "ADD PET" there). Done.
// Wallpaper-only app:
// 1. Export its frames as separate PNGs, named like the dragon's (see ANIMS below):
//    egg_idle_00.png, egg_hatch_00.png, cat_emerge_00.png, idle_a_00.png, walk_a_00.png
// 2. Put them in their own folder: assets/pets/<name>/, e.g. pets/cat/
// 3. Add one line to PETS below, e.g. PetType("cat", "Cat", "pets/cat"),
// The setup screen shows a "Which pet?" question as soon as there is more than one.
//
// How to find things:
// [ADD PET] = where new pets go
// [ADD ACTION] = where new movements the pet does by itself go
// [EDIT] = a setting you can safely change
// =====

package com.yourname.projecttp.wallpaper

import java.io.File

/**
 * One pet.
 * id = short name used in saved data, e.g. "dragon". Never change it after release,
 * or users lose their saved progress for that pet. Use the same id as in pet.gd.
 * name = name shown to the user on the "Which pet?" screen.
 * folder = its frame folder inside the app's assets, e.g. "pets/dragon".
 */
data class PetType(val id: String, val name: String, val folder: String)

/**
 * One action.
 * frames = how many frame files it uses when files are only numbered
 * fps = frames per second (higher = faster)
 * loop = true repeats forever, false plays once and then calls onAnimationFinished()
 * names = what its files are called, without the frame number. The pet's name in
 * front is allowed: "idle" also matches dragon_idle_00.png, cat_idle_00.png...
 */
data class Anim(val frames: Int, val fps: Int, val loop: Boolean, val names: List<String>)

/**
 * One thing the pet may do by itself between blinks.
 * kind = "blink" : nothing new, it keeps resting and blinking (a longer idle)
 * "travel" : goes to a random spot on the left or right with this animation
 * "hop" : jumps up on the spot with this animation, then plays "landing"
 * "play" : plays this animation once where it stands
 * anim = the animation it uses (a name from ANIMS; empty for "blink")
 * weight = how often it is picked compared with the others (4 = four times as often
 * as 1; 0 = switched off)
 */
data class Action(val kind: String, val anim: String, val weight: Float)

/**
 * Everything about how the pets animate and behave, in one bundle. Comes from pet.gd
 * (one app with the game) or from the lists in PetCatalog below.
 */
class PetTable(
    val anims: Map<String, Anim>, // the animations
    val actions: List<Action>, // what the pet does by itself
    val waitMin: Float, // seconds of blinking between two actions
    val waitMax: Float,
    val blinkFrames: Int, // frames in one blink
    val fromGame: Boolean, // true = read from the Godot game's export

```

```

// Movements with several versions: animation -> frames per version. Playing such an
// animation plays ONE version, picked at random ("trick#1", "trick#2", ...).
val sets: Map<String, Int> = PetCatalog.ANIM_SETS,
// Low energy: at or below this battery percent (and not charging) the pet picks from
// lowActions instead of actions.
val lowBatteryPercent: Int = PetCatalog.LOW_BATTERY_PERCENT,
val lowActions: List<Action> = PetCatalog.LOW_ENERGY_ACTIONS,
// Sleep: between these hours (24-hour clock) the pet plays "sleep" instead of
// pet_idle; a tap or a shake wakes it for awakeMinutes.
val sleepFrom: Int = PetCatalog.SLEEP_FROM_HOUR,
val sleepUntil: Int = PetCatalog.SLEEP_UNTIL_HOUR,
val awakeMinutes: Float = PetCatalog.AWAKE_MINUTES,
// The game's screen as set in Godot (Project Settings > Display > Window): its width
// and height in the game's own units, whether Stretch is on, and the Stretch Scale.
// With these the wallpaper shows the pet at the same size on screen as the game does.
val gameWidth: Float = 720f,
val gameHeight: Float = 1280f,
val gameStretches: Boolean = true,
val gameScale: Float = 1f,
// pet_size in pet.gd: the pet's size in the game until the player resizes it (0 =
// not known, the wallpaper's own drawScale is used).
val defaultSize: Float = 0f
) {
    /**
     * How many screen pixels one unit of the game's screen is on a screen of this size.
     * (Godot fits the game's screen into the phone's screen: the smaller of the two
     * ratios, the same for every Stretch Aspect except "ignore", which is close to it.)
     */
    fun gamePixel(screenW: Float, screenH: Float): Float {
        val fit = if (gameStretches) minOf(screenW / gameWidth, screenH / gameHeight) else 1f
        return fit * gameScale
    }
}

object PetCatalog {

    // [ADD PET] Used by the wallpaper-only app (Part B). Inside the Godot game's app the
    // list comes from pet.gd instead (see fromGameList below).
    // One line per pet: PetType(id, name shown to the user, assets folder).
    // The FIRST pet is the default.
    val PETS = listOf(
        PetType("dragon", "Dragon", "pets/dragon"),
        // PetType("cat", "Cat", "pets/cat"),
        // PetType("fox", "Fox", "pets/fox"),
    )

    // [EDIT] The animations: Anim(frames, fps, loop, file names it accepts).
    // Files are matched by name first (egg_idle_03.png -> egg_idle,
    // idle_b_03.png -> pet_idle, fly_a_00.png -> move). If no names match, files
    // are taken in number order: the first 8 -> egg_idle, the next 8 -> egg_hatch...
    // The second number is the speed. Keep in sync with ANIMS in pet.gd (Godot).
    // "move" is played while the pet travels sideways (by itself, sent by a tap,
    // or the phone tilted on the lock screen). fly_ frames make it bob in the air.
    // "move_left" / "move_right": if a pet has these, they are used when it travels
    // that way (no mirroring). With only "move", one set serves both ways.
    // "trick" holds the pet's tricks, 8 frames each; one is played at a time.
    // "low_energy" is played on a low battery, "sleep" is its resting animation
    // at night (see the settings below ACTIONS).
    // "greet" is optional: frames named greet_00.png, hello_00.png or wave_00.png
    // are played when the lock screen appears. Without them the pet hops instead.

```

```

val ANIMS = linkedMapOf(
    "egg_idle" to Anim(8, 6, true, listOf("egg_idle")),
    "egg_hatch" to Anim(8, 10, false, listOf("egg_hatch", "egg_hatch_emerge")),
    "pet_emerge" to Anim(8, 10, false, listOf("pet_emerge", "emerge")),
    "pet_idle" to Anim(8, 8, true, listOf("pet_idle", "idle")),
    "jump" to Anim(8, 12, false, listOf("jump", "jump_land_a", "jump_landing_a")),
    "landing" to Anim(8, 12, false, listOf("landing", "jump_land_b", "jump_landing_b")),
    "ticklish" to Anim(8, 12, false, listOf("ticklish", "ticklish_recovery_a")),
    "recovery" to Anim(8, 10, false, listOf("recovery", "ticklish_recovery_b")),
    "move" to Anim(32, 12, true, listOf("move", "fly", "walk", "run", "hop")),
    "move_left" to Anim(8, 12, true,
        listOf("move_left", "fly_left", "walk_left", "run_left")),
    "move_right" to Anim(8, 12, true,
        listOf("move_right", "fly_right", "walk_right", "run_right")),
    "trick" to Anim(16, 10, false, listOf("trick", "extra", "special", "dance")),
    "low_energy" to Anim(8, 8, false, listOf("low_energy", "low_battery", "tired")),
    "sleep" to Anim(8, 6, true, listOf("sleep", "sleeping", "asleep")),
    "greet" to Anim(8, 10, false, listOf("greet", "hello", "wave")),
)
// [EDIT] WHAT THE PET DOES BY ITSELF (wallpaper-only app; inside the Godot game's app
//      this comes from ACTIONS in pet.gd). Most of the time the hatched pet rests
//      and blinks (pet_idle). After ACTION_WAIT_MIN..ACTION_WAIT_MAX seconds it
//      picks one line below at random, does it, and goes back to resting.
//      Action(what it does, animation, weight) - see "Action" above.
//      An action whose animation has no frames for the pet is skipped.
// [ADD ACTION] Add the animation to ANIMS above (loop = false), then a line here,
//      e.g. Action("play", "spin", 2f),
val ACTIONS = listOf(
    Action("blink", "", 4f),          // keep blinking a while longer
    Action("travel", "move", 3f),     // fly / walk / run to the left or right
    Action("hop", "jump", 2f),        // jump on the spot (jump, then landing)
    Action("play", "trick", 2f),      // one of its tricks (see ANIM_SETS)
)
// [EDIT] Seconds of resting and blinking between two actions (a random time in this
//      range), and how many frames one blink has (an action starts between blinks).
const val ACTION_WAIT_MIN = 3f
const val ACTION_WAIT_MAX = 7f
const val BLINK_FRAMES = 8
// The settings below are the stand-ins for the wallpaper-only app. Inside the Godot
// game's app they all come from pet.gd (SECTION 0), where they have the same names.

// [EDIT] MOVEMENTS WITH SEVERAL VERSIONS: animation -> frames per version. With
//      "trick" to 8, trick_a_00..07 is one trick and trick_b_00..07 another; when
//      the pet plays "trick" it picks ONE of them at random.
val ANIM_SETS = mapOf("trick" to 8, "low_energy" to 8)

// [EDIT] LOW ENERGY: at or below this battery percent, and not charging, the pet
//      picks from this list instead of ACTIONS.
const val LOW_BATTERY_PERCENT = 15
val LOW_ENERGY_ACTIONS = listOf(
    Action("blink", "", 3f),          // keep blinking a while longer
    Action("play", "low_energy", 4f), // one of its low-energy movements
)

// [EDIT] SLEEP: between these hours of the phone's clock (24-hour: 23 = 11 PM) the
//      pet sleeps; a tap or a shake wakes it for AWAKE_MINUTES.
const val SLEEP_FROM_HOUR = 23
const val SLEEP_UNTIL_HOUR = 6
const val AWAKE_MINUTES = 5f

/** The lists above as one bundle (used when pet.gd's lists aren't in the app). */
fun defaultTable() =
    PetTable(ANIMS, ACTIONS, ACTION_WAIT_MIN, ACTION_WAIT_MAX, BLINK_FRAMES, false)

/**
 * Which animation a frame file belongs to, from its name (null if none).
 * "egg_idle_03.png" -> "egg_idle" (exact name)
 * "dragon_idle_03.png" -> "pet_idle" (pet name in front + "idle")
 * "idle_b_03.png" -> "pet_idle" (extra rows a, b, c, d from the slicer)
 * [EDIT] If a file isn't recognised, add its name (without the number) to ANIMS.
 */
fun animForFile(fileName: String): String? {
    val label = fileName.substringBeforeLast('.').lowercase()
        .replace(Regex("[ _]*\\d+$"), "") // remove the frame number
    return animForLabel(label)
}

private fun animForLabel(label: String): String? {

```

```

// 1) exact match, e.g. "egg_idle", "jump_land_a"
ANIMS.entries.firstOrNull { label in it.value.names }?.let { return it.key }
// 2) pet name in front, e.g. "dragon_idle" ends with "_idle"
ANIMS.entries.firstOrNull { e -> e.value.names.any { label.endsWith("_$it") } }
    ?.let { return it.key }
// 3) extra rows from the slicer, e.g. "idle_b" -> try "idle"
if (Regex("[a-z]$").containsMatchIn(label)) return animForLabel(label.dropLast(2))
return null
}

/**
 * Sorts a pet's frame file names into animations: by name if they match ANIMS,
 * otherwise by number order. Returns e.g. { "egg_idle": [file names...], ... }.
 */
fun groupFrames(fileNames: List<String>): Map<String, List<String>> {
    val sorted = fileNames.sortedWith(FRAME_ORDER)
    val byName = LinkedHashMap<String, MutableList<String>>()
    for (name in sorted) {
        val anim = animForFile(name) ?: continue
        byName.getOrPut(anim) { mutableListOf() }.add(name)
    }
    if (byName.isEmpty()) return byName
    val result = LinkedHashMap<String, List<String>>()
    var next = 0
    for ((name, anim) in ANIMS) {
        result[name] = sorted.drop(next).take(anim.frames)
        next += anim.frames
    }
    return result
}

// Saved settings shared by the setup screen and the wallpaper.
const val PREFS = "pet_wallpaper" // name of the saved-settings file
const val KEY_PET = "pet_id" // which pet the user chose

// Where the Godot add-on puts things inside the app's assets (one app with the game).
const val GAME_FRAMES = "wallpaper_frames" // + "<pet id>/<frame>.png"
const val GAME_PET_LIST = "wallpaper_frames/pets.txt" // one pet id per line
// ANIMS and ACTIONS from pet.gd, written by the add-on (see tableFromGame below).
const val GAME_TABLE = "wallpaper_frames/anim.txt"
// In each pet's folder: which frame files belong to which animation, sorted by the
// add-on with pet.gd's own rules (see framesFromList below).
const val FRAME_LIST = "frames.txt"

// The animated Project P wallpaper: the assets folder with its frames, and the kinds
// of picture files it accepts. With the Godot add-on the frames are packed there from
// addons/projectp_wallpaper/default_wallpaper_frames/ on every export. In the
// wallpaper-only app, copy them into app/src/main/assets/wallpaper_bg_frames/.
// No frames = the still picture (pet_default_bg.png) is used.
const val DEFAULT_BG_FRAMES = "wallpaper_bg_frames"
val PICTURE_TYPES = setOf("png", "jpg", "jpeg", "webp")

/**
 * Turns the add-on's pet list (one id per line, same order as PETS in pet.gd) into
 * pets. The name shown to the user is the id with capitals: "ice_dragon" -> "Ice Dragon".
 */
fun fromGameList(text: String): List<PetType> =
    text.lines().map { it.trim() }.filter { it.isNotEmpty() }.map { id ->
        val name = id.split('_', '-', ' ').filter { it.isNotEmpty() }
            .joinToString(" ") { it.replaceFirstChar { c -> c.uppercase() } }
    }

```

```

        PetType(id, name, "$GAME_FRAMES/$id")
    }

/**
 * Reads the animations and actions the add-on copied from pet.gd. One entry per line:
 * anim|pet_idle|8.0|true          name, frames per second, loops?
 * actions|1                      pet.gd has an ACTIONS list (the lines below)
 * action|travel|move|3           what it does, animation, weight
 * wait|3.0|7.0                   seconds between actions (min, max)
 * blink|8                        frames per blink
 * sets|1 and set|trick|8         movements with several versions: frames each
 * lowbattery|15                  low energy at or below this battery percent
 * lowactions|1 and lowaction|play|low_energy|4 what it does then
 * sleep|23|6|5.0                 asleep from, until (hours), minutes awake after a tap
 * gamescreen|720|1280|canvas_items|1.0 the game's screen: width, height,
 *                                Stretch Mode, Stretch Scale
 * petsize|2.0                    pet_size in pet.gd (the size before any resizing)
 * Animations pet.gd doesn't list (e.g. "greet") keep their values from ANIMS above.
 * Anything missing or unreadable falls back to the lists above.
 */
fun tableFromGame(text: String): PetTable {
    val anims = LinkedHashMap<ANIMS>()
    val actions = ArrayList<Action>()
    var waitMin = ACTION_WAIT_MIN
    var waitMax = ACTION_WAIT_MAX
    var blink = BLINK_FRAMES
    var listsActions = false // pet.gd has an ACTIONS list (even an empty one)
    val sets = LinkedHashMap<String, Int>()
    var listsSets = false // pet.gd has ANIM_SETS (even an empty one)
    val lowActions = ArrayList<Action>()
    var listsLowActions = false
    var lowBattery = LOW_BATTERY_PERCENT
    var sleepFrom = SLEEP_FROM_HOUR
    var sleepUntil = SLEEP_UNTIL_HOUR
    var awakeMinutes = AWAKE_MINUTES
    var gameWidth = 720f
    var gameHeight = 1280f
    var gameStretches = true
    var gameScale = 1f
    var defaultSize = 0f
    for (raw in text.lines()) {
        val part = raw.trim().split('|').map { it.trim() }
        try {
            when (part[0]) {
                "anim" -> {
                    val fps = part[2].toFloat().toInt().coerceIn(1, 60)
                    val old = anims[part[1]]
                    anims[part[1]] = Anim(
                        old?.frames ?: 8, fps, part[3] == "true", old?.names ?: emptyList()
                    )
                }
                "actions" -> listsActions = true
                "action" -> actions += Action(part[1], part[2], part[3].toFloat())
                "wait" -> {
                    waitMin = part[1].toFloat()
                    waitMax = part[2].toFloat()
                }
                "blink" -> blink = part[1].toFloat().toInt().coerceAtLeast(1)
                "sets" -> listsSets = true
                "set" -> {
                    val per = part[2].toFloat().toInt()

```

```

        if (per > 0) sets[part[1]] = per
    }
    "lowbattery" -> lowBattery = part[1].toFloat().toInt()
    "lowactions" -> listsLowActions = true
    "lowaction" -> lowActions += Action(part[1], part[2], part[3].toFloat())
    "sleep" -> {
        sleepFrom = part[1].toFloat().toInt()
        sleepUntil = part[2].toFloat().toInt()
        awakeMinutes = part[3].toFloat().coerceAtLeast(0f)
    }
    "gamescreen" -> {
        gameWidth = part[1].toFloat().coerceAtLeast(1f)
        gameHeight = part[2].toFloat().coerceAtLeast(1f)
        gameStretches = part[3] != "disabled"
        gameScale = part[4].toFloat().takeIf { it > 0f } ?: 1f
    }
    "petsize" -> defaultSize = part[1].toFloat().coerceAtLeast(0f)
}
} catch (e: Exception) {
    // a line that can't be read is skipped
}
}
if (waitMax < waitMin) waitMax = waitMin
// No ACTIONS in pet.gd (an older pet.gd): the list above. An empty ACTIONS list
// in pet.gd is respected: the pet then only rests and blinks.
val chosen = if (listsActions) actions else actions.isEmpty { ACTIONS }
return PetTable(
    anims, chosen, waitMin, waitMax, blink, true,
    sets = if (listsSets) sets else ANIM_SETS,
    lowBatteryPercent = lowBattery,
    lowActions = if (listsLowActions) lowActions else LOW_ENERGY_ACTIONS,
    sleepFrom = sleepFrom, sleepUntil = sleepUntil, awakeMinutes = awakeMinutes,
    gameWidth = gameWidth, gameHeight = gameHeight,
    gameStretches = gameStretches, gameScale = gameScale, defaultSize = defaultSize
)
}

/**
 * Reads a pet's frame list written by the add-on. One line per animation:
 *   pet_idle=idle_a_00.png/idle_a_01.png/...
 * Returns { "pet_idle": [file names in playing order], ... }.
 */
fun framesFromList(text: String): Map<String, List<String>> {
    val result = LinkedHashMap<String, List<String>>()
    for (raw in text.lines()) {
        val line = raw.trim()
        if (!line.contains('=')) continue
        val files = line.substringAfter('=').split('/').map { it.trim() }
            .filter { it.isNotEmpty() }
        if (files.isNotEmpty()) result[line.substringBefore('=').trim()] = files
    }
    return result
}

/** Finds a pet by id in a list; falls back to the first pet if the id is unknown. */
fun byId(pets: List<PetType>, id: String?): PetType =
    pets.firstOrNull { it.id == id } ?: pets.first()

/**
 * Sorts frame file names by their numbers, so "frame_2.png" comes before

```

```

    * "frame_10.png". Works with names like 01.png, frame_1.png or idle_b_03.png.
    */
    val FRAME_ORDER = Comparator<String> { a, b ->
        val chunksA = Regex("\\d+|\\D+").findAll(a.lowercase()).map { it.value }.toList()
        val chunksB = Regex("\\d+|\\D+").findAll(b.lowercase()).map { it.value }.toList()
        for (i in 0 until minOf(chunksA.size, chunksB.size)) {
            val x = chunksA[i]
            val y = chunksB[i]
            val cmp = if (x[0].isDigit() && y[0].isDigit()) {
                x.toBigInteger().compareTo(y.toBigInteger()) // compare as numbers
            } else {
                x.compareTo(y) // compare as text
            }
            if (cmp != 0) return@Comparator cmp
        }
        chunksA.size - chunksB.size
    }
}

// =====
// GameSave - READS THE GODOT GAME'S SAVE FILE (only when both are in one app)
// =====
// When the wallpaper is packaged inside the Godot game's APK, both share the same
// private storage. The game saves to "user://pet.cfg", which is the file "pet.cfg" in
// that storage, so the wallpaper can read it to follow the game:
// - which pet the player chose -> [game] chosen_pet="dragon"
// - whether that pet has hatched -> [dragon] hatched=true
// - how big the player made it -> [dragon] size=2.0
// In the wallpaper-only app this file doesn't exist, and everything here returns
// "nothing", so the wallpaper uses its own settings instead.
// The wallpaper only READS this file; it never changes the game's save.
// [FIX] If the wallpaper doesn't follow the game, check the names above still match
// what pet.gd and pet_picker.gd save (SAVE_PATH, "hatched", "chosen_pet").
// The pet is chosen on the pet grid, opened from the main menu (main_menu.gd).
object GameSave {

    const val FILE_NAME = "pet.cfg"

    /** The save file as { section -> { key -> value } }. Empty if there is no file. */
    fun read(filesDir: File): Map<String, Map<String, String>> {
        val file = File(filesDir, FILE_NAME)
        if (!file.exists()) return emptyMap()
        val result = LinkedHashMap<String, MutableMap<String, String>>()
        var section = ""
        try {
            for (raw in file.readLines()) {
                val line = raw.trim()
                if (line.startsWith("[") && line.endsWith("]")) {
                    section = line.substring(1, line.length - 1)
                } else if (line.contains("=") && !line.startsWith(";")) {
                    val key = line.substringBefore("=").trim()
                    val value = line.substringAfter("=").trim().removeSurrounding("\"")
                    result.getOrPut(section) { LinkedHashMap() }[key] = value
                }
            }
        } catch (e: Exception) {
            return emptyMap()
        }
        return result
    }

    /** The pet chosen in the game, or null if the game hasn't saved one. */
    fun chosenPet(filesDir: File): String? =
        read(filesDir)["game"]?.get("chosen_pet")?.takeIf { it.isNotEmpty() }

    /** Has this pet hatched in the game? */
    fun isHatched(filesDir: File, petId: String): Boolean =
        read(filesDir)[petId]?.get("hatched") == "true"

    /**
     * The size the player gave this pet in the game ([dragon] size=2.0): 1 = the frame's
     * own size on the game's screen, 2 = twice that. null if the game hasn't saved one.
     */
    fun size(filesDir: File, petId: String): Float? =
        read(filesDir)[petId]?.get("size")?.toFloatOrNull()?.takeIf { it > 0f }
}

```

Then the wallpaper itself: right-click `app/src/main/java/com/yourname/projectp/wallpaper`, choose **New > Kotlin Class/File**, name it `PetWallpaperService`, and replace its contents with the file `PetWallpaperService.kt` (code shown below for reference):

```
// =====
// PetWallpaperService.kt - THE PET ON THE HOME SCREEN AND LOCK SCREEN
// =====
// What this file does:
//   This is the live wallpaper. It draws the background (the Project P wallpaper, still
//   or animated, the user's own pictures, or their GIF) and the animated pet on top.
//   The user can have one picture for the home screen and another for the lock screen.
//   - Home screen: tap the pet to tickle it (it glows softly while it listens); then
//   tap any spot and it goes there (sideways it flies or walks, up or down it jumps).
//   No finger drag is needed, so the home screen's own swipes (app list, pages) don't
//   get mixed up with moving the pet.
//   - By itself, on both screens: the pet mostly rests and blinks. Now and then it
//   does one thing picked at random from the ACTIONS list (travels left or right,
//   hops, plays ONE of its tricks, or just keeps blinking), then rests again.
//   - Low battery (15% or less, not charging): it picks from the low-energy list
//   instead. At night (11 PM to 6 AM by the phone's clock) it sleeps; a tap on the
//   home screen or a shake on the lock screen wakes it for a few minutes.
//   - Its size is the size the player gave it in the game (one app with the game).
//   - Lock screen (no touch there): the pet greets you with a hop when the screen turns
//   on, giggles when you shake the phone, hops when you tilt the phone up, and flies
//   or walks towards the side you tilt it to. A GIF background is animated on the
//   lock screen and kept still on the home screen.
//   Android runs this whenever the home screen or lock screen is visible, and pauses it
//   when an app covers the screen.
//
// How to find things:
//   [EDIT] = a setting you can safely change (sizes, speeds, what is switched on...)
//   [LOGIC] = rules for how the pet behaves (what plays after what)
//   [FIX] = places to look first if something goes wrong
//
// Sections in this file:
//   1. Which pet is shown -> search "SECTION 1"
//   2. Settings (size, taps, what the pet does where) -> search "SECTION 2"
//   3. Loaded images and saved data -> search "SECTION 3"
//   4. Pet state (position, animation, moving, sensors) -> search "SECTION 4"
//   5. Wallpaper lifecycle (start, resize, show/hide) -> search "SECTION 5"
//   6. Loading a pet + animation rules (hatch, tickle...) -> search "SECTION 6"
//   7. Touch: tap, tap to move (home screen) -> search "SECTION 7"
//   8. Background (default, own pictures, GIF, animation) -> search "SECTION 8"
//   9. Drawing each frame -> search "SECTION 9"
//   10. Home screen or lock screen? -> search "SECTION 10"
//   11. What the pet does by itself; travelling, jumping -> search "SECTION 11"
//   12. Phone movement: shake and tilt -> search "SECTION 12"
//   13. Sleeping at night and low energy -> search "SECTION 13"
//
// Adding more pets: you don't need to change this file. See PetCatalog.kt.
//
// One app with the Godot game: this file works unchanged there. It then follows the
// game's save (chosen pet, hatched, size) through GameSave in PetCatalog.kt.
//
// Related files:
//   PetCatalog.kt - the list of pets and the animation layout ([ADD PET])
//   SetWallpaperActivity.kt - the setup screens (questions, pet choice, photo picker)
//   assets/wallpaper_frames/<id>/*.png - frames packed by the Godot add-on (one app)
//   assets/pets/<name>/*.png - frames copied by hand (wallpaper-only app)
//   res/drawable-nodpi/pet_default_bg.png - the Project P wallpaper (still picture)
//   assets/wallpaper_bg_frames/* - its animation frames, if you supply any
//   AndroidManifest.xml - registers this wallpaper with Android
// =====
```

```

package com.yourname.projectp.wallpaper

import android.app.KeyguardManager
import android.app.WallpaperManager
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.graphics.*
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.media.ExifInterface
import android.os.BatteryManager
import android.os.Build
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.os.PowerManager
import android.os.SystemClock
import android.service.wallpaper.WallpaperService
import android.view.MotionEvent
import android.view.SurfaceHolder
import android.view.ViewConfiguration
import java.io.File
import java.util.Calendar
import java.util.concurrent.Executors
import kotlin.math.PI
import kotlin.math.abs
import kotlin.math.asin
import kotlin.math.exp
import kotlin.math.hypot
import kotlin.math.max
import kotlin.math.min
import kotlin.math.sin
import kotlin.math.sqrt
import kotlin.random.Random

/**
 * The pets this app can show.
 * - Inside the Godot game's app: the list the add-on packed (same pets as pet.gd).
 * - Wallpaper-only app: the PETS list in PetCatalog.kt.
 * [FIX] If the wallpaper shows the wrong pets, check the Output line of the Godot export
 * ("Project P Wallpaper: ... pets: ...").
 */
fun availablePets(context: Context): List<PetType> {
    val fromGame = try {
        context.assets.open(PetCatalog.GAME_PET_LIST).bufferedReader().use { it.readText() }
    } catch (e: Exception) {
        null // no list from the game: wallpaper-only app
    }
    val pets = if (fromGame != null) PetCatalog.fromGameList(fromGame) else emptyList()
    return pets.ifEmpty { PetCatalog.PETS }
}

/**
 * How the pets animate and behave (animations, actions, waiting times).
 * - Inside the Godot game's app: ANIMS and ACTIONS from pet.gd, packed by the add-on.
 * - Wallpaper-only app: the lists in PetCatalog.kt.
 */

```

```

fun petTable(context: Context): PetTable {
    val fromGame = try {
        context.assets.open(PetCatalog.GAME_TABLE).bufferedReader().use { it.readText() }
    } catch (e: Exception) {
        null // not packed by the game: use PetCatalog's lists
    }
    return if (fromGame != null) PetCatalog.tableFromGame(fromGame) else PetCatalog.defaultTable()
}

/**
 * Which of a pet's frame files belong to which animation, in playing order:
 * { "pet_idle": ["idle_a_00.png", ...], "move": [...], ... }.
 * - Inside the Godot game's app: the list the add-on wrote (frames.txt in the pet's
 *   folder), sorted with pet.gd's own rules, so the wallpaper always agrees with the
 *   game about which file is which.
 * - Otherwise: sorted here by file name with PetCatalog.groupFrames().
 * [FIX] An animation is missing on the phone: open the wallpaper setup and tap
 *       "Check my pet". It shows how many frames each animation got.
 */
fun petFrameFiles(context: Context, folder: String): Map<String, List<String>> {
    val inFolder = try {
        (context.assets.list(folder) ?: emptyArray()).toList()
    } catch (e: Exception) {
        emptyList()
    }
    val listed = try {
        context.assets.open("$folder/${PetCatalog.FRAME_LIST}").bufferedReader()
            .use { it.readText() }
    } catch (e: Exception) {
        null
    }
    if (listed != null) {
        // Keep only files that really are in the app
        val groups = PetCatalog.framesFromList(listed)
            .mapValues { (_, files) -> files.filter { it in inFolder } }
            .filterValues { it.isNotEmpty() }
        if (groups.isNotEmpty()) return groups
    }
    return PetCatalog.groupFrames(inFolder.filter { it.lowercase().endsWith(".png") })
}

/**
 * The phone's battery: how full it is (0..100, or -1 if the phone doesn't say) and
 * whether it is plugged in. Android hands this out to every app; no permission needed.
 */
fun phoneBattery(context: Context): Pair<Int, Boolean> {
    val info = try {
        context.applicationContext.registerReceiver(
            null, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
    } catch (e: Exception) {
        null
    }
    if (info == null) return -1 to false
    val level = info.getIntExtra(BatteryManager.EXTRA_LEVEL, -1)
    val full = info.getIntExtra(BatteryManager.EXTRA_SCALE, -1)
    val percent = if (level >= 0 && full > 0) level * 100 / full else -1
    val plugged = info.getIntExtra(BatteryManager.EXTRA_PLUGGED, 0) != 0
    return percent to plugged
}

/**

```

```

* One frame's soft outline, for the glow around a listening pet: the frame's shape,
* blurred, made from a small copy of the frame (frameW x frameH pixels). `left` and
* `top` say where it sits compared with that copy (it is a little bigger on every side).
*/
private class Glow(
    val shape: Bitmap, val left: Int, val top: Int, val frameW: Int, val frameH: Int
)

/**
 * One of the user's own pictures: the one for the home screen or the one for the lock
 * screen. Filled in by loadPicture() (SECTION 8).
 */
private class UserPicture(val fileName: String) {
    var fitted: Bitmap? = null // the picture cut and scaled to the screen (null = none)
    @Suppress("DEPRECATION")
    var gif: Movie? = null // the same picture as a GIF, if it is an animated one
    var stamp = 0L // when the file was last changed (reload only if newer)
}

/** Android asks this service for an "engine" each time the wallpaper is shown. */
class PetWallpaperService : WallpaperService() {

    override fun onCreateEngine(): Engine = PetEngine()

    // The animations, the actions and the other rules for the pets: from pet.gd when the
    // wallpaper is inside the Godot game's app, otherwise from PetCatalog.kt. Read once.
    private val rules: PetTable by lazy { petTable(this) }

    // [LOGIC] A pet woken up at night stays awake until this moment (milliseconds since
    // the phone was switched on). Kept here so the home screen and the lock
    // screen agree.
    @Volatile private var awakeUntil = 0L

    // ===== THE LOADED PET'S PICTURES (shared) =====
    // Android can run several engines at once (home screen, lock screen, the preview).
    // They all show the same pet, so its pictures are loaded once here and shared, and
    // the old pet's pictures are thrown away BEFORE the new pet's are loaded. That keeps
    // memory low when the pet is changed, which matters with many frames per pet.
    // [EDIT] The most memory one pet's frames may use, in megabytes. Bigger pets are
    // loaded at half size (and drawn at the same size on screen) to fit.
    private val frameMemoryLimitMb = 96
    private var loadedPetId: String? = null
    private var loadedFrames: Map<String, List<Bitmap>> = emptyMap()
    // The animations whose frames are fly_ frames: the pet bobs in the air while it
    // travels with one of these.
    private var loadedBobAnims: Set<String> = emptySet()
    private var loadedFrameW = 128 // size of one frame file in pixels
    private var loadedFrameH = 128

    /** The pet's pictures, loading them first if another pet (or none) is loaded. */
    private fun framesFor(pet: PetType): Map<String, List<Bitmap>> {
        if (pet.id == loadedPetId) return loadedFrames
        freeFrames() // old pet out first, then the new one in
        loadedFrames = try {
            loadFrames(pet.folder)
        } catch (e: Throwable) {
            emptyMap() // no pictures: the background still shows
        }
        loadedPetId = pet.id
        return loadedFrames
    }
}

```

```

}

private fun freeFrames() {
    val old = loadedFrames
    loadedFrames = emptyMap()
    loadedPetId = null
    old.values.flatten().distinct().forEach { it.recycle() }
}

// Loads the pet's frames, sorted into animations by petFrameFiles() (top of this
// file): with the list from the Godot game, or by file name.
// [FIX] If an action shows the wrong pictures, check ANIMS in pet.gd (one app with
// the game) or the names listed in PetCatalog.ANIMS (wallpaper-only app).
private fun loadFrames(folder: String): Map<String, List<Bitmap>> {
    val grouped = petFrameFiles(this, folder)
    // Frames named fly_... mean the pet flies (and bobs in the air) with that animation
    loadedBobAnims = grouped.filterValues {
        it.firstOrNull()?.lowercase()?.startsWith("fly") == true
    }.keys
    // Measure one frame, then decide whether the frames must be loaded smaller.
    val firstName = grouped.values.flatten().firstOrNull() ?: return emptyMap()
    val bounds = BitmapFactory.Options().apply { inJustDecodeBounds = true }
    assets.open("$folder/$firstName").use { BitmapFactory.decodeStream(it, null, bounds) }
    loadedFrameW = bounds.outWidth.coerceAtLeast(1)
    loadedFrameH = bounds.outHeight.coerceAtLeast(1)
    val count = grouped.values.sumOf { it.size }
    var shrink = 1
    while (loadedFrameW.toLong() * loadedFrameH * 4L * count / (shrink * shrink) >
        frameMemoryLimitMb * 1024L * 1024L
    ) shrink *= 2
    val opts = BitmapFactory.Options().apply { inSampleSize = shrink }
    val loaded = LinkedHashMap<String, List<Bitmap>>()
    for ((anim, files) in grouped) {
        loaded[anim] = files.mapNotNull { name ->
            assets.open("$folder/$name").use { BitmapFactory.decodeStream(it, null, opts) }
        }
    }
    // [LOGIC] Movements with several versions (ANIM_SETS in pet.gd / PetCatalog.kt):
    // besides the whole animation, each version gets its own entry, named
    // "trick#1", "trick#2", ... (the same pictures, not loaded twice). The
    // pet plays ONE of them at a time, see oneVersion() in SECTION 11.
    for ((anim, perVersion) in rules.sets) {
        val all = loaded[anim] ?: continue
        if (perVersion <= 0) continue
        for (v in 0 until all.size / perVersion) {
            loaded["$anim#${v + 1}"] =
                all.subList(v * perVersion, (v + 1) * perVersion).toList()
        }
    }
    return loaded
}

// A helper thread that prepares the animated Project P wallpaper's next frame, so
// loading big pictures never makes the pet stutter (SECTION 8).
private val frameLoader = Executors.newSingleThreadExecutor()

// The wallpaper was removed or replaced: give the memory back.
override fun onDestroy() {
    frameLoader.shutdown()
    freeFrames()
}

```

```

        super.onDestroy()
    }

    /** The engine does all the work: drawing, animation, touch and phone movement. */
    @Suppress("DEPRECATION") // android.graphics.Movie (GIF) is old but works everywhere
    inner class PetEngine : Engine() {

        // ===== SECTION 1: WHICH PET IS SHOWN =====
        // The animation layout and the list of pets live in PetCatalog.kt.
        // [ADD PET] New pets are added there (or in pet.gd for the one-app setup).
        // The animations, the actions and the waiting times: from pet.gd when the
        // wallpaper is inside the Godot game's app, otherwise from PetCatalog.kt.
        private val table = rules
        private val anims = table.anims
        // Used for an animation name nobody listed, so nothing can go wrong with it.
        private val plainAnim = Anim(8, 8, false, emptyList())
        // One version of a movement ("trick#2") has the speed and looping of "trick".
        private fun animOf(name: String) =
            anims[name] ?: anims[name.substringBefore('#')] ?: plainAnim
        // ===== SECTION 2: SETTINGS =====
        // [EDIT] How many taps on the egg before it hatches.
        private val tapsToHatch = 3
        // [EDIT] true = the pet is as big as the player made it in the game (pinching it
        // there), so it looks the same in the app and on the home screen. Resize
        // it in the game and the wallpaper follows the next time it is shown.
        // false = always use drawScale below.
        private val sizeFollowsGame = true
        // [EDIT] Pet size on screen when the game's size isn't used (wallpaper-only app,
        // or sizeFollowsGame = false): 2f = each 128px frame is drawn at 256px.
        // Use 1.5f for smaller, 3f for bigger.
        private val drawScale = 2f
        // [EDIT] true if the pet frames are pixel art (keeps hard pixel edges);
        // false for smooth, painted frames (softens the edges when scaled).
        private val petIsPixelArt = false
        // [EDIT] Same choice for the Project P wallpaper picture (pet_default_bg.png).
        private val defaultBackgroundIsPixelArt = false

        // --- Moving the pet on the home screen ---
        // [EDIT] Tap to move: tap the pet (it giggles and glows softly while it listens),
        // then tap a spot and it goes there. false switches this off.
        private val tapToMove = true
        // [EDIT] How many seconds the pet keeps listening for the "go there" tap.
        private val listenSeconds = 4f
        // [EDIT] true = a tap mostly above or below the pet makes every pet jump there
        // (jump frames, then landing frames). false = pets with fly_ frames fly
        // there instead and only walking pets jump.
        private val flyersJumpUpAndDown = true
        // [EDIT] Moving the pet by dragging it with a finger. Off, because the home screen
        // reacts to the same drag (opens the app list, switches pages). With true,
        // both ways of moving the pet work.
        private val dragToMove = false
        // [EDIT] Show the soft glow around a listening pet and at the spot it's going to.
        private val showMarks = true
        // [EDIT] The glow: its colour (a warm white) with a wider, softer edge in a
        // second colour (amber, so the glow also shows on a white picture); how
        // strong it is at most (0f = invisible, 1f = strongest; 0.7f is subtle);
        // how far it reaches around the pet as a share of the pet's width; and
        // the size of the glow at the spot the pet is going to (in dp; 26 is
        // about the size of a fingertip).
    }

```

```

private val glowColor = Color.rgb(255, 248, 225)
private val glowEdgeColor = Color.rgb(255, 186, 84)
private val glowStrength = 0.7f
private val glowReach = 0.09f
private val spotGlowDp = 26f
// [EDIT] true = where the home screen reports taps on empty space, only those
//         taps send the pet somewhere (a tap on an app icon just opens the app).
//         false = any tap sends it. See onCommand() in SECTION 7.
private val useHomeScreenTapReports = true

// --- What the pet does on which screen ---
// [EDIT] Doing things by itself now and then (the ACTIONS list: travelling,
//         hopping, playing a movement). false = it only rests and blinks there.
//         WHAT it does and HOW OFTEN is not set here: inside the Godot game's app
//         that is ACTIONS and ACTION_WAIT_MIN / MAX in pet.gd (SECTION 0); in the
//         wallpaper-only app it is ACTIONS in PetCatalog.kt.
private val roamOnLockScreen = true
private val roamOnHomeScreen = true
// [EDIT] Reacting to shaking and tilting the phone. Off on the home screen by
//         default, because normal handling of the phone would keep setting it off.
private val motionOnLockScreen = true
private val motionOnHomeScreen = false
// [EDIT] A little hop when the lock screen appears (screen turned on).
//         (A sleeping pet doesn't greet.)
private val greetOnLockScreen = true
// [EDIT] Sleeping at night and low energy on a low battery (SECTION 13). The
//         hours, the battery percent and the low-energy list are set in pet.gd
//         (SECTION 0) or, for the wallpaper-only app, in PetCatalog.kt.
private val sleepsAtNight = true
private val tiresOnLowBattery = true
// [EDIT] Moving backgrounds (a GIF the user picked, or the animated Project P
//         wallpaper): where they play. Where they don't play, they stand still.
//         Playing on the home screen too costs more battery.
private val animateBackgroundOnLockScreen = true
private val animateBackgroundOnHomeScreen = false
// [EDIT] Speed of the animated Project P wallpaper, in frames per second.
//         8 is calm; use 12 for smoother movement (needs more frames).
private val defaultAnimationFps = 8

// --- Travelling ---
// [EDIT] Travel speed, in screen widths per second (0.25 = 4 seconds to cross).
private val moveSpeed = 0.25f
// [EDIT] How high flying pets bob while travelling, as a share of the pet's height.
private val moveBob = 0.06f
// [EDIT] Which way the pet faces in its fly_/walk_ frames. If it travels
//         backwards, change this to false.
private val moveFramesFaceRight = true

// --- Hopping (greeting, tilt up) and jumping to a spot (tap to move) ---
// [EDIT] Hop height as a share of the pet's height, and how long a hop lasts.
private val hopHeight = 0.5f
private val hopSeconds = 0.7f
// [EDIT] Speed of a jump to another spot, in screen heights per second (0.8 = the
//         whole screen in 1.25 seconds), and how much higher the arc gets with
//         distance (0 = flat, 0.25 = a quarter of the distance).
private val jumpSpeed = 0.8f
private val jumpArc = 0.25f

// --- Phone movement ---
// [EDIT] How hard a shake must be (higher = shake harder). 7 is a clear shake.

```

```

private val shakeStrength = 7f
// [EDIT] How far to tilt sideways, from the way the phone is normally held, before
// the pet travels that way (higher = more tilt). 2.5 needs a clear tilt of
// about 20 degrees.
private val tiltSideways = 2.5f
// [EDIT] How many degrees the top of the phone must be raised, within about a
// second, for a hop.
private val tiltUpDegrees = 18f

// ===== SECTION 3: LOADED IMAGES AND SAVED DATA =====
// Small saved settings: chosen pet, pet position ("x", "y") with the pet's size
// at that moment ("w", "h"), and, for each pet, whether it has hatched
// ("hatched_<id>").
// [FIX] To reset the pets to eggs while testing: clear the app's storage in
// Android Settings > Apps > (your app) > Storage.
private val prefs = getSharedPreferences(PetCatalog.PREFS, Context.MODE_PRIVATE)
// Every pet this app can show (from the game's pet.gd, or PetCatalog.PETS).
private val pets = availablePets(this@PetWallpaperService)

// The chosen pet and its frames. Loaded by loadPet() (SECTION 6), and reloaded
// when another pet is picked in the game or the setup screen.
// frames["egg_idle"] = the egg_idle pictures in order, and so on for every action.
// [FIX] If the pet doesn't show, check the export's Output line names this pet.
private var pet: PetType = PetCatalog.byId(pets, null)
private var frames: Map<String, List<Bitmap>> = emptyMap()
// The animations this pet flies with (fly_ frames): it bobs in the air with them.
private var bobAnims: Set<String> = emptySet()
// Size of one frame file in pixels, measured from the first PNG (e.g. 128).
private var frameW = 128
private var frameH = 128
// Size of the pet on screen, in pixels. Set by applySize() (SECTION 6).
private var petW = frameW * drawScale
private var petH = frameH * drawScale

// Paint without smoothing (hard pixel edges) and with smoothing (soft edges).
private val pixelPaint = Paint().apply { isFilterBitmap = false; isAntiAlias = false }
private val smoothPaint = Paint().apply { isFilterBitmap = true }
private val petPaint = if (petIsPixelArt) pixelPaint else smoothPaint
// The glow around a listening pet and at its destination (SECTION 9).
private val glowPaint = Paint().apply { isAntiAlias = true; isFilterBitmap = true }
private val spotPaint = Paint().apply { isAntiAlias = true }
// Made the first time a frame glows, then kept (see glowOf).
private val glows = HashMap<Bitmap, Glow>()
// The round glow at the destination: bright in the middle, fading to nothing.
private val spotLight: Shader by lazy {
    RadialGradient(
        0f, 0f, 1f,
        intArrayOf(
            glowColor, Color.argb(110, Color.red(glowColor), Color.green(glowColor),
                Color.blue(glowColor)),
            Color.argb(0, Color.red(glowColor), Color.green(glowColor),
                Color.blue(glowColor))),
        floatArrayOf(0f, 0.4f, 1f), Shader.TileMode.CLAMP)
}
// Its wider, softer edge in the second colour (shows on a light picture too).
private val spotEdge: Shader by lazy {
    val r = Color.red(glowEdgeColor)
    val g = Color.green(glowEdgeColor)
    val b = Color.blue(glowEdgeColor)
    RadialGradient(

```

```

        0f, 0f, 1.35f,
        intArrayOf(Color.rgb(150, r, g, b), Color.rgb(90, r, g, b),
            Color.rgb(0, r, g, b)),
        floatArrayOf(0f, 0.6f, 1f), Shader.TileMode.CLAMP)
    }
    // [EDIT] Colour used if pet_default_bg.png is missing.
    private val fallbackColor = Color.parseColor("#F6E7F0")
    // The Project P wallpaper (res/drawable-nodpi/pet_default_bg.png) cut and scaled
    // to the screen. Loaded the first time it's needed (SECTION 8).
    private var defaultFitted: Bitmap? = null
    private var defaultTried = false
    // The user's own pictures (null inside = none chosen): one for the home screen and,
    // if they said their lock screen is different, one for the lock screen.
    private val homePicture = UserPicture(SetWallpaperActivity.BACKGROUND_FILE)
    private val lockPicture = UserPicture(SetWallpaperActivity.LOCK_BACKGROUND_FILE)
    // The animated Project P wallpaper: the names of its frame files, in order (empty =
    // no animation, the still picture is used). See "ANIMATED PROJECT P WALLPAPER" in
    // SECTION 8.
    private val defaultFrameFiles: List<String> = try {
        (assets.list(PetCatalog.DEFAULT_BG_FRAMES) ?: emptyArray())
            .filter { it.substringAfterLast('.').lowercase() in PetCatalog.PICTURE_TYPES }
            .sortedWith(PetCatalog.FRAME_ORDER)
    } catch (e: Throwable) {
        emptyList()
    }
    private var shownFrame: Bitmap? = null // the frame on screen, fitted to the screen
    private var readyFrame: Bitmap? = null // the next frame, prepared and waiting
    private var spareFrame: Bitmap? = null // a finished frame, reused for the next one
    private var shownFrameIndex = -1
    private var readyFrameIndex = -1
    private var frameIsLoading = false
    private var frameShownSince = 0L
    private var frameBatch = 0 // goes up when the screen size changes
    private var frameFailures = 0 // unreadable frames in a row
    // Runs the animation timer on the main thread.
    private val handler = Handler(Looper.getMainLooper())
    // How far a finger can move and still count as a tap (Android's standard value).
    private val tapSlop =
        ViewConfiguration.get(this@PetWallpaperService).scaledTouchSlop.toFloat()
    // A touch counts as a tap only if the finger lifts within this time (milliseconds).
    private val MAX_TAP_MILLIS = 350L
    // A trip is undone if the wallpaper is hidden this soon after the tap (SECTION 11).
    private val UNDO_TRIP_MILLIS = 800L
    // Tells us whether the phone is showing its lock screen (SECTION 10).
    private val keyguard = getSystemService(Context.KEYGUARD_SERVICE) as KeyguardManager
    // Tells us whether the screen is really on (not dimmed to an always-on display).
    private val power = getSystemService(Context.POWER_SERVICE) as PowerManager
    // The phone's movement sensor (SECTION 12). null on devices without one.
    private val sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
    private val accelerometer: Sensor? =
        sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

    // ===== SECTION 4: PET STATE =====
    // Is the wallpaper on screen right now? (false while an app covers it)
    private var visible = false
    // Is it the lock screen we're showing on? (SECTION 10)
    private var onLockScreen = false
    // Is the screen fully on? false while it is off or showing an always-on display.
    private var awake = true
    // Screen size in pixels.

```

```

private var screenW = 0f
private var screenH = 0f
// Top-left corner of the pet on screen.
private var petX = 0f
private var petY = 0f

// Has the egg hatched? Saved per pet, so each pet keeps its own progress.
private var hatched = false
// Name of the animation playing now (a key in PetCatalog.ANIMS), its frame, and
// when that frame was shown.
private var current = "egg_idle"
private var frame = 0
private var frameShownAt = 0L
// true while a play-once animation (hatching, tickle, hop) must not be interrupted.
private var busy = false
// Taps counted on the egg so far.
private var eggTaps = 0

// Finger tracking for taps and drags (SECTION 7).
private var fingerDown = false // a finger is on the screen
private var downOnPet = false // ...and it first touched the pet
private var dragging = false // the pet is being carried (only with dragToMove)
private var moved = false // finger moved too far to count as a tap
private var downX = 0f // where the finger first touched
private var downY = 0f
private var grabX = 0f // where on the pet the finger grabbed it
private var grabY = 0f
// Tap to move (SECTION 7): the pet listens for a "go there" tap until this time.
private var listeningUntil = 0L
private var listeningSince = 0L // when it started listening (the glow fades in)
// The spot it was sent to, marked until it arrives.
private var markShown = false
private var markX = 0f
private var markY = 0f
// Where and when that trip began (to undo it if the tap really opened an app).
private var tripFromX = 0f
private var tripFromY = 0f
private var tripStartedAt = 0L
// true once the home screen has told us about a tap on empty space (see
// onCommand in SECTION 7). Remembered, so it's known right after a restart.
private var homeReportsTaps = prefs.getBoolean("home_reports_taps", false)
private var unreportedTaps = 0 // taps in a row the home screen didn't report

// Travelling (SECTION 11).
private var moving = false // the pet is travelling across the screen
private var travelAnim = "move" // ...with this animation
private var movingByTilt = false // ...because the phone is tilted (not wandering)
private var moveTargetX = 0f // where it's going
private var moveTargetY = 0f
private var moveLineY = 0f // its height along the way, without the bob
private var moveTime = 0f // seconds since setting off (for the bob)
private var nextMoveIn = 10f // seconds until it wanders off again
private var facingLeft = false // the pet is drawn mirrored (see travelPick)
private var headingLeft = false // which way its current or last trip goes
// Hopping and jumping (SECTION 11). hopStartedAt = 0 means "not in the air".
private var hopStartedAt = 0L
private var hopMillis = 700f // how long this hop or jump lasts
private var hopArc = 0f // how high its arc is, in pixels
private var hopFromX = 0f // where it took off
private var hopFromY = 0f

```

```

private var hopToX = 0f          // where it lands (same spot for a hop in place)
private var hopToY = 0f
private var lastGreetAt = 0L

// Sleeping and low energy (SECTION 13)
private var bedtime = false     // the phone's clock is in the sleeping hours
private var clockReadAt = 0L
private var lowEnergy = false    // the battery is low and the phone isn't charging
private var batteryReadAt = 0L

// Phone movement (SECTION 12).
private var sensorOn = false
private var gravityKnown = false
private var gravX = 0f          // the slow, steady part of the sensor reading:
private var gravY = 0f          // which way is "down" for the phone
private var gravZ = 0f
private var restX = 0f          // how far sideways the phone is normally held
private var sideTilt = 0f        // sideways tilt compared with that
private var slowLift = 0f        // how far the top has been raised lately
private var lastSensorAt = 0L    // when the last reading arrived
private var joltCount = 0        // sharp movements counted towards a shake
private var firstJoltAt = 0L
private var lastJoltAt = 0L
private var lastCountedJoltAt = 0L
private var lastShakeAt = 0L
private var lastTiltHopAt = 0L

// Timer bookkeeping.
private var lastTickAt = 0L
private var lastLockCheckAt = 0L
private var gifStartedAt = 0L

// The animation timer. Each tick it moves the pet's animation on, moves the pet if
// it's travelling or hopping, draws, and schedules itself again. It ticks quickly
// (about 30 times a second) only while something is moving; otherwise it ticks at
// the animation's own speed to save battery. It stops when the wallpaper is hidden.
private val tick = object : Runnable {
    override fun run() {
        if (!visible) return
        val now = SystemClock.uptimeMillis()
        val dt = (now - lastTickAt).coerceIn(0L, 250L) / 1000f
        lastTickAt = now
        try {
            // Another engine switched to a new pet: follow it
            if (frames !== loadedFrames) loadPet()
            if (now - lastLockCheckAt > 100L) updateLockState()
            if (!awake) {
                // Screen off or always-on display: do nothing, look again shortly
                handler.postDelayed(this, 200L)
                return
            }
            stepAnimation(now)
            stepMoving(dt)
            stepHop(now)
            stepRest()
            stepActionTimer(dt)
            stepDefaultAnimation(now)
            drawFrame()
        } catch (e: Throwable) {
            // Never let one bad frame take the whole wallpaper down.

```

```

        }
        handler.postDelayed(this, nextTickDelay())
    }
}

private fun nextTickDelay(): Long =
    if (moving || hopStartedAt != 0L || backgroundIsAnimating() ||
        (showMarks && isListening())) // the glow breathes smoothly
    ) 33L
    else (1000L / animOf(current).fps).coerceAtMost(250L) // timers need ticks

// Run the timer right now (used when something starts moving between ticks).
private fun tickNow() {
    if (!visible) return
    handler.removeCallbacks(tick)
    handler.post(tick)
}

init {
    loadPet() // load the chosen pet before anything is drawn
}

// ===== SECTION 5: WALLPAPER LIFECYCLE =====
// Called once when the wallpaper starts.
override fun onCreate(surfaceHolder: SurfaceHolder) {
    super.onCreate(surfaceHolder)
    setTouchEventsEnabled(true) // without this the pet can't be tapped or dragged
}

// Called when the screen size is known or changes (e.g. first start, rotation).
override fun onSurfaceChanged(
    holder: SurfaceHolder, format: Int, width: Int, height: Int
) {
    super.onSurfaceChanged(holder, format, width, height)
    screenW = width.toFloat()
    screenH = height.toFloat()
    applySize() // the pet's size depends on the screen's size
    loadPosition()
    try {
        // Screen size changed: every picture is fitted again at the new size
        defaultFitted = null
        defaultTried = false
        homePicture.stamp = 0L
        lockPicture.stamp = 0L
        forgetDefaultFrames()
        loadPicturesIfChanged()
    } catch (e: Throwable) {
        // Not enough memory for a fitted copy: the plain colour is drawn instead.
    }
    // The surface can be made again while the wallpaper stays visible: make sure
    // the animation timer is running.
    visible = isVisible
    drawFrame()
    tickNow()
}

// Called with true when the home/lock screen appears and false when an app covers it
// or the screen turns off. This is what hides the pet in other apps and saves battery.
override fun onVisibilityChanged(isVisible: Boolean) {
    visible = isVisible
}

```

```

handler.removeCallbacks(tick)
try {
    if (isVisible) {
        if (PetCatalog.byId(pets, chosenPetId()).id != pet.id) {
            // Another pet was picked (in the game or the setup screen): switch.
            // Only the pet changes; the background stays as it is.
            loadPet()
        } else if (!hatched && GameSave.isHatched(filesDir, pet.id)) {
            // The pet hatched in the game since we last looked
            hatched = true
            playIdle()
        }
        applySize() // resized in the game since last time?
        loadPosition()
        refreshRest() // asleep or awake, tired or not: know
                     // it before the first picture is drawn
        loadPicturesIfChanged() // picks up newly chosen pictures
        updateLockState()
        lastTickAt = SystemClock.uptimeMillis()
        gifStartedAt = lastTickAt
        drawFrame() // right picture for this screen, at once
        if (onLockScreen && awake && greetOnLockScreen) greet()
    } else {
        settleDown() // finish any trip or hop cleanly
        dragging = false // a finger that never lifted is forgotten
        moved = false
        // The screen is switching off and the lock screen has its own picture:
        // draw it now, so it's already there when the screen comes back on.
        if (lockPicture.fitted != null && !power.isInteractive && !onLockScreen) {
            onLockScreen = true
            drawFrame()
        }
    }
} catch (e: Throwable) {
    // Keep the wallpaper running even if something above goes wrong.
}
updateSensor()
tickNow() // restart the animation (if visible)
}

// Called when the wallpaper's drawing surface goes away: stop everything.
override fun onSurfaceDestroyed(holder: SurfaceHolder) {
    visible = false
    handler.removeCallbacks(tick)
    updateSensor()
    super.onSurfaceDestroyed(holder)
}

// Called when the wallpaper is removed or replaced: stop and give the memory back.
override fun onDestroy() {
    visible = false
    handler.removeCallbacks(tick)
    updateSensor()
    try {
        forgetGlows()
        forgetDefaultFrames()
        defaultFitted?.recycle()
        defaultFitted = null
        for (p in listOf(homePicture, lockPicture)) {
            p.fitted?.recycle()
        }
    }
}

```

```

        p.fitted = null
        p.gif = null
    }
} catch (e: Throwable) {
    // nothing to do: the memory is freed later by Android
}
super.onDestroy()
}

// ===== SECTION 6: LOADING A PET AND ANIMATION RULES =====
// Loads the chosen pet's frames and saved progress, and starts it idling.
// The background is not touched here.
private fun loadPet() {
    // If the old pet was mid-trip or mid-hop, put it back down first
    if (moving) petY = moveLineY
    if (hopStartedAt != 0L) {
        petX = hopToX
        petY = hopToY
        savePosition()
    }
    movingByTilt = false
    markShown = false
    listeningUntil = 0L
    pet = PetCatalog.byId(pets, chosenPetId())
    forgetGlows() // they belong to the old pet's pictures
    frames = framesFor(pet) // loads the pictures (see the top of this file)
    bobAnims = loadedBobAnims
    frameW = loadedFrameW
    frameH = loadedFrameH
    applySize()
    // Hatched if it hatched on the wallpaper, or in the game (one app with the game)
    hatched = prefs.getBoolean("hatched_${pet.id}", false) ||
        GameSave.isHatched(filesDir, pet.id)
    eggTaps = 0
    moving = false
    hopStartedAt = 0L
    facingLeft = false
    playIdle()
    resetRoamTimer()
    clampPet()
}

// [LOGIC] HOW BIG THE PET IS ON SCREEN.
// One app with the game: the size the player gave the pet in the game (pinch
// it there; saved as "size" in the game's save file), or pet_size from pet.gd
// if it was never resized. The game draws a frame at
// frame size x pet size x (phone screen / game screen)
// pixels, and so does the wallpaper: the pet looks the same in both.
// Wallpaper-only app (or sizeFollowsGame = false): frame size x drawScale.
// When the size changes, the pet keeps its middle where it was.
// Why there is no resizing on the home screen itself: a wallpaper only gets a
// copy of each touch, and a pinch there already belongs to the home screen (it
// opens its own menu on most phones), so both would react at once.
// [FIX] The pet is the wrong size on the home screen: open the game, pinch the pet
// to the size you want, go back to the home screen. "Check my pet" in the
// wallpaper setup shows the size it found.
private fun applySize() {
    var scale = drawScale
    if (sizeFollowsGame && table.fromGame && screenW > 0f) {
        val size = GameSave.size(filesDir, pet.id)
    }
}

```

```

        ?: table.defaultSize.takeIf { it > 0f }
        if (size != null) scale = size * table.gamePixel(screenW, screenH)
    }
    if (screenW > 0f) {          // never bigger than the screen
        scale = min(scale, min(screenW / frameW, screenH / frameH))
    }
    val w = frameW * scale
    val h = frameH * scale
    if (w == petW && h == petH) return
    petX += (petW - w) / 2f      // keep its middle where it was
    petY += (petH - h) / 2f
    petW = w
    petH = h
    if (screenW > 0f) clampPet()
}

// Which pet to show: the one chosen in the Godot game if the wallpaper is inside
// the game's app (see GameSave in PetCatalog.kt), otherwise the one picked in the
// wallpaper's own setup screen.
private fun chosenPetId(): String? =
    GameSave.chosenPet(filesDir) ?: prefs.getString(PetCatalog.KEY_PET, null)

// Does this pet have pictures for an animation?
private fun has(name: String) = (frames[name]?.size ?: 0) > 0

// Start an animation from its first frame.
private fun play(name: String) {
    current = name
    frame = 0
    frameShownAt = SystemClock.uptimeMillis()
}

// Go back to resting: egg_idle before hatching; after it pet_idle (blinking), or
// "sleep" at night (see restAnim in SECTION 13).
private fun playIdle() {
    busy = false
    play(restAnim())
}

// Shows the next frame when the current one has been on screen long enough.
// Looping animations wrap around; play-once animations call onAnimationFinished().
private fun stepAnimation(now: Long) {
    val a = animOf(current)
    if (now - frameShownAt < 1000L / a.fps) return
    frameShownAt = now
    val count = frames[current]?.size ?: 0    // frames actually found for it
    when {
        frame < count - 1 -> frame++
        a.loop -> frame = 0
        else -> onAnimationFinished(current)
    }
}

// [LOGIC] What plays after each play-once animation ends.
// Change these lines to change the order of animations.
private fun onAnimationFinished(name: String) {
    when (name) {
        // egg cracks -> pet comes out. If hatching and emerging are one animation
        // (frames named egg_hatch_emerge_..), there is no pet_emerge to play.
        "egg_hatch" -> if (has("pet_emerge")) play("pet_emerge") else finishHatching()
    }
}

```

```

        "pet_emerge" -> finishHatching()
        "ticklish" -> play("recovery")           // tickled -> calms down
        "recovery", "landing", "greet" -> playIdle() // -> back to resting
        // "jump" holds its last frame while the pet is in the air or carried
        "jump" -> if (!moving && hopStartedAt == 0L && !dragging) playIdle()
        // Any other play-once animation (e.g. "trick#2", "low_energy#1", or one
        // you added to ACTIONS): back to resting. While the pet is travelling or in the air
        // with it, it holds the last frame until it arrives.
        else -> if (!moving && hopStartedAt == 0L) playIdle()
    }
}

// The pet is out of its egg: remember it and start resting.
private fun finishHatching() {
    hatched = true
    prefs.edit().putBoolean("hatched_${pet.id}", true).apply()
    playIdle()
}

// Can the pet start something new right now? (hatched, not mid-animation, not held)
private fun isFree() = hatched && !busy && !dragging

// [LOGIC] Tickle: ticklish, then recovery (see onAnimationFinished), then idle.
//      Used by a tap on the home screen and by shaking the phone.
private fun tickle() {
    if (!isFree()) return
    stopMoving()
    busy = true
    play("ticklish")
    tickNow()
}

// [LOGIC] What a tap on the pet does.
//      Egg: count taps, hatch after tapsToHatch (SECTION 2).
//      Pet: tickle, and listen for a "go there" tap (SECTION 7).
//      Sleeping pet: the tap wakes it up (SECTION 13); the next tap tickles.
private fun onTap() {
    val wokeIt = wakeUp() // (also keeps an awake pet awake at night)
    if (hatched && tapToMove && current != "sleep") {
        // Listen (again) for the next tap, even if it is still giggling
        val now = SystemClock.uptimeMillis()
        if (!isListening()) listeningSince = now
        listeningUntil = now + (listenSeconds * 1000f).toLong()
    }
    if (wokeIt || current == "sleep") { // it was asleep: this tap woke it,
        drawFrame() // nothing more
        return
    }
    if (busy) { // hatching, being tickled or in the air: nothing new
        drawFrame()
        return
    }
    if (!hatched) {
        eggTaps++
        if (eggTaps >= tapsToHatch) {
            busy = true
            when {
                has("egg_hatch") -> play("egg_hatch") // hatches (and emerges)
                has("pet_emerge") -> play("pet_emerge") // only emerge frames
                else -> finishHatching() // no hatching frames
            }
        }
    }
}

```

```

        }
    } else {
        tickle()
    }
    drawFrame()
}

// ===== SECTION 7: TOUCH (TAP AND TAP TO MOVE, HOME SCREEN) =====
// [LOGIC] A tap is a finger that goes down and up without moving.
// Tap on the pet -> onTap(): the egg counts taps; the pet giggles and
// listens for listenSeconds (it glows softly).
// Tap somewhere else while it listens -> goTo(): the pet travels there.
// Anything that isn't a tap (a swipe, a long drag) is left to the home
// screen: it opens the app list or switches pages, and the pet stays put.
// With dragToMove (SECTION 2) the pet can also be carried by a finger.
// The lock screen keeps all touches for itself, so none of this runs there.
// A wallpaper only gets a copy of each touch; it can't keep the home screen from
// reacting to it too. That's why moving the pet uses taps: the home screen does
// nothing on a tap on empty space.
override fun onTouchEvent(event: MotionEvent) {
    try {
        handleTouch(event)
    } catch (e: Throwable) {
        // ignore: a touch must never crash the wallpaper
    }
    super.onTouchEvent(event)
}

private fun handleTouch(event: MotionEvent) {
    when (event.actionMasked) {
        // Finger touches the screen: remember where, and whether it was on the pet
        MotionEvent.ACTION_DOWN -> {
            fingerDown = true
            dragging = false
            moved = false
            downX = event.x
            downY = event.y
            downOnPet = isOnPet(event.x, event.y)
            // Carrying the pet (only with dragToMove); the hatching egg is left alone
            if (dragToMove && downOnPet && !(busy && !hatched)) {
                stopMoving() // grabbing it ends any trip
                if (hopStartedAt != 0L) { // ...or jump, right where it is
                    hopStartedAt = 0L
                    playIdle()
                    savePosition()
                }
                markShown = false
                dragging = true
                grabX = event.x - petX
                grabY = event.y - petY
            }
        }
        // A second finger: this is a gesture for the home screen, not a tap
        MotionEvent.ACTION_POINTER_DOWN -> moved = true
        // Finger moves: past tapSlop it's no longer a tap
        MotionEvent.ACTION_MOVE -> if (fingerDown) {
            if (!moved && hypot(event.x - downX, event.y - downY) > tapSlop) {
                moved = true
                if (dragging && hatched) { busy = false; play("jump") } // picked up
            }
        }
    }
}

```

```

        }
        if (dragging && moved) {
            petX = event.x - grabX
            petY = event.y - grabY
            clampPet()
            drawFrame()
        }
    }
    // Finger lifts: a tap, or the end of carrying the pet
    MotionEvent.ACTION_UP, MotionEvent.ACTION_CANCEL -> if (fingerDown) {
        fingerDown = false
        // A tap is short; a long press belongs to the home screen (its menu)
        val wasTap = !moved && event.actionMasked == MotionEvent.ACTION_UP &&
            event.eventTime - event.downTime < MAX_TAP_MILLIS
        if (dragging) {
            dragging = false
            if (moved) {
                if (hatched) play("landing")
                savePosition()
                resetRoamTimer()
            }
        }
        if (wasTap) {
            if (downOnPet) onTap()
            // Home screens that report taps on empty space do so through
            // onCommand() below; for the others, any tap counts.
            else if (isListening()) onTapElsewhere(event.x, event.y)
        }
    }
}

// [LOGIC] Many home screens (Pixel and others built on the same base) tell the
// wallpaper when the user taps EMPTY space, not an app icon or a widget.
// Once a home screen has done that, only those taps send the pet
// somewhere, so tapping an app icon while the pet listens just opens the
// app. On home screens that never report taps, every tap counts.
// If a home screen stops reporting (the user switched to another one),
// three taps in a row without a report switch back to "every tap counts".
// [FIX] The pet doesn't go to tapped spots although it glows: set
// useHomeScreenTapReports (SECTION 2) to false.

// A tap away from the pet, seen through the touch copy, while the pet listens.
private fun onTapElsewhere(x: Float, y: Float) {
    if (!homeReportsTaps) {
        goTo(x, y) // this home screen doesn't report taps
        return
    }
    // The home screen's own report (onCommand) normally follows and sends the pet.
    // Three taps in a row without one: it doesn't report (any more).
    unreportedTaps++
    if (unreportedTaps >= 3) {
        setHomeReportsTaps(false)
        goTo(x, y)
    }
}

private fun setHomeReportsTaps(reports: Boolean) {
    unreportedTaps = 0
    if (homeReportsTaps == reports) return

```

```

        homeReportsTaps = reports
        prefs.edit().putBoolean("home_reports_taps", reports).apply()
    }

    override fun onCommand(
        action: String?, x: Int, y: Int, z: Int, extras: Bundle?, resultRequested: Boolean
    ): Bundle? {
        try {
            if (useHomeScreenTapReports && action == WallpaperManager.COMMAND_TAP) {
                setHomeReportsTaps(true)
                if (!isOnPet(x.toFloat(), y.toFloat()) && isListening()) {
                    goTo(x.toFloat(), y.toFloat())
                }
            }
        } catch (e: Throwable) {
            // ignore: a command must never crash the wallpaper
        }
        return super.onCommand(action, x, y, z, extras, resultRequested)
    }

    // [LOGIC] Is this spot on the pet? Inside its frame, AND on (or within a
    //          fingertip of) the drawn part of the picture. The empty corners of the
    //          frame don't count, so a big pet doesn't swallow taps meant for the app
    //          icons around it.
    // [EDIT] tapReachDp: how far from the drawn part a tap still counts (in dp).
    private val tapReachDp = 14f
    private fun isOnPet(x: Float, y: Float): Boolean {
        if (x < petX || x > petX + petW || y < petY || y > petY + petH) return false
        if (petW <= 0f || petH <= 0f) return false
        try {
            val picture = frames[current]?.getOrNull(frame)?: return true
            if (picture.isRecycled) return true
            val reach = tapReachDp * resources.displayMetrics.density
            for (stepY in -1..1) for (stepX in -1..1) {
                var px = ((x + stepX * reach - petX) / petW * picture.width).toInt()
                val py = ((y + stepY * reach - petY) / petH * picture.height).toInt()
                if (facingLeft) px = picture.width - 1 - px // drawn mirrored
                if (px < 0 || py < 0 || px >= picture.width || py >= picture.height) continue
                if (Color.alpha(picture.getPixel(px, py)) > 40) return true
            }
            return false
        } catch (e: Throwable) {
            return true // can't look at the picture: the whole frame counts
        }
    }

    // Is the pet waiting for a "go there" tap? (never on the lock screen)
    private fun isListening() = tapToMove && hatched && !onLockScreen &&
        SystemClock.uptimeMillis() < listeningUntil

    // [LOGIC] Tap to move: sends the pet to the tapped spot (its middle ends up there).
    //          Mostly to the left or right -> it travels with its move frames (fly,
    //          walk...), facing the way it goes.
    //          Mostly up or down -> it jumps there: jump frames while it is
    //          in the air, landing frames on arrival.
    //          (With flyersJumpUpAndDown = false, flying pets fly everywhere.)
    //          A pet without the needed frames uses the other way; with neither, it
    //          simply appears at the spot.
    private fun goTo(x: Float, y: Float) {
        if (!hatched || dragging || current == "sleep") return

```

```

// A giggle is cut short; a greeting, a jump or a landing finish first (the pet
// keeps listening, so the tap can simply be repeated)
if (busy && (current == "ticklish" || current == "recovery")) playIdle()
if (busy || hopStartedAt != 0L) return
if (moving && markShown) return // already on its way (same tap, twice)
stopMoving()
val toX = (x - petW / 2f).coerceIn(0f, max(0f, screenW - petW))
val toY = (y - petH / 2f).coerceIn(0f, max(0f, screenH - petH))
val dx = toX - petX
val dy = toY - petY
if (hypot(dx, dy) < tapSlop) return // already there
listeningUntil = 0L // one trip per tap on the pet
tripFromX = petX
tripFromY = petY
tripStartedAt = SystemClock.uptimeMillis()
markX = x
markY = y
markShown = true
val upOrDown = abs(dy) > abs(dx)
val flies = listOf("move", "move_left", "move_right").any { it in bobAnims }
val mayJump = has("jump") && (flyersJumpUpAndDown || !flies)
when {
    upOrDown && mayJump -> jumpTo(toX, toY)
    canTravel("move") -> startMoving(toX, false, toY)
    has("jump") -> jumpTo(toX, toY)
    else -> { // no frames for either: just be there
        petX = toX
        petY = toY
        markShown = false
        savePosition()
        drawFrame()
    }
}
}

// Where the pet sits. Saved when it arrives somewhere (or is put down after a drag),
// together with how big it was then.
private fun savePosition() {
    prefs.edit().putFloat("x", petX).putFloat("y", petY)
        .putFloat("w", petW).putFloat("h", petH).apply()
}

// [EDIT] Where the pet starts the very first time: centred, 60% down the screen.
// After that, the saved position is used.
// If the pet has been resized since its position was saved, it stays centred on
// the same spot.
private fun loadPosition() {
    if (screenW <= 0f || dragging || moving || hopStartedAt != 0L) return
    if (prefs.contains("x")) {
        val savedW = prefs.getFloat("w", frameW * drawScale)
        val savedH = prefs.getFloat("h", frameH * drawScale)
        petX = prefs.getFloat("x", 0f) + (savedW - petW) / 2f
        petY = prefs.getFloat("y", 0f) + (savedH - petH) / 2f
    } else {
        petX = (screenW - petW) / 2f
        petY = screenH * 0.6f
    }
    clampPet()
}

```

```

// ===== SECTION 8: BACKGROUND =====
// What is behind the pet, in this order:
// 1. On the lock screen: the user's lock screen picture, if they chose one.
// 2. The user's home screen picture, if they chose one (also used on the lock
//    screen when they said both screens are the same).
// 3. Otherwise the Project P wallpaper: animated if you supplied frames,
//    else the still picture.
// The pictures are chosen in SetWallpaperActivity and saved in the app's private
// storage (files/background.img and files/background_lock.img).
// Changing the pet never changes any of this.

// Loads the user's pictures if they are new or were replaced.
private fun loadPicturesIfChanged() {
    loadPicture(homePicture)
    loadPicture(lockPicture)
    if (homePicture.fitted != null) {
        // The user's own picture is in use: the Project P wallpaper isn't needed
        forgetDefaultFrames()
        defaultFitted?.recycle()
        defaultFitted = null
        defaultTried = false
    }
}

private fun loadPicture(p: UserPicture) {
    val file = File(filesDir, p.fileName)
    if (!file.exists()) { // none chosen (or it was removed)
        p.fitted?.recycle()
        p.fitted = null
        p.gif = null
        p.stamp = 0L
        return
    }
    if (file.lastModified() == p.stamp && p.fitted != null) return
    if (screenW <= 0f || screenH <= 0f) return
    try {
        // Read the size first, then load a smaller copy so big pictures
        // don't use too much memory
        // [FIX] If big photos crash the app (out of memory), this is the place.
        val bounds = BitmapFactory.Options().apply { inJustDecodeBounds = true }
        BitmapFactory.decodeFile(file.path, bounds)
        var sample = 1
        while (bounds.outWidth / (sample * 2) >= screenW &&
            bounds.outHeight / (sample * 2) >= screenH
        ) sample *= 2
        val opts = BitmapFactory.Options().apply { inSampleSize = sample }
        var bmp = BitmapFactory.decodeFile(file.path, opts) ?: return
        // Turn photos the right way up if the camera stored them sideways
        // [FIX] If a photo shows sideways, check this rotation part.
        val rotation = when (ExifInterface(file.path).getAttributeInt(
            ExifInterface.TAG_ORIENTATION, ExifInterface.ORIENTATION_NORMAL)) {
            ExifInterface.ORIENTATION_ROTATE_90 -> 90f
            ExifInterface.ORIENTATION_ROTATE_180 -> 180f
            ExifInterface.ORIENTATION_ROTATE_270 -> 270f
            else -> 0f
        }
        if (rotation != 0f) {
            bmp = Bitmap.createBitmap(bmp, 0, 0, bmp.width, bmp.height,
                Matrix().apply { postRotate(rotation) }, true)
        }
    }
}

```

```

        val fitted = fitToScreen bmp, smoothPaint
        bmp.recycle()
        p.fitted?.recycle()
        p.fitted = fitted
        p.stamp = file.lastModified()
        p.gif = loadGif(file)
    } catch (e: Throwable) {
        p.fitted = null // unreadable picture -> the next choice in the list
        p.gif = null
    }
}

// If the picture is a GIF with more than one frame, returns it ready to animate.
// [FIX] A GIF that stays still on the lock screen: check it really is a .gif file
//       (some gallery apps save a still copy), and animateBackgroundOnLockScreen.
//       Very large GIFs (full-HD with many frames) may be too big to load; they
//       then show as a still picture. Smaller GIFs (about 480px wide) play best.
private fun loadGif(file: File): Movie? {
    return try {
        val header = ByteArray(4)
        file.inputStream().use { it.read(header) }
        if (String(header, Charsets.US_ASCII) != "GIF8") return null
        Movie.decodeFile(file.path)?.takeIf { it.duration() > 0 }
    } catch (e: Throwable) {
        null
    }
}

// [LOGIC] Which of the user's pictures belongs on the screen showing now
//       (null = none chosen: the Project P wallpaper is used).
private fun pictureForThisScreen(): UserPicture? = when {
    onLockScreen && lockPicture.fitted != null -> lockPicture
    homePicture.fitted != null -> homePicture
    else -> null
}

// May a moving background play on the screen showing now? (SECTION 2)
private fun animationAllowedHere() =
    if (onLockScreen) animateBackgroundOnLockScreen else animateBackgroundOnHomeScreen
// Is the Project P wallpaper in use, and does it have frames that can be read?
private fun defaultHasFrames() = pictureForThisScreen() == null &&
    defaultFrameFiles.isNotEmpty() && frameFailures < defaultFrameFiles.size

// Is the background moving right now (a GIF, or the animated Project P wallpaper)?
private fun backgroundIsAnimating() = awake && animationAllowedHere() &&
    (pictureForThisScreen()?.gif != null ||
        (defaultHasFrames() && defaultFrameFiles.size > 1))

// Draws the background (see the list at the top of this section).
private fun drawBackground(canvas: Canvas) {
    // Start from a plain colour, so see-through pictures don't show leftovers
    canvas.drawColor(fallbackColor)
    val own = pictureForThisScreen()
    val movie = own?.gif
    val picture = when {
        own != null -> own.fitted
        // Animated Project P wallpaper: the frame on screen. Where it doesn't
        // play (home screen), the last frame shown simply stays.
        defaultHasFrames() && shownFrame != null -> shownFrame
    }
}

```

```

        else -> defaultPicture()
    }
    if (movie != null && backgroundIsAnimating()) {
        drawGifCover(canvas, movie)
    } else if (picture != null && !picture.isRecycled) {
        canvas.drawBitmap(picture, 0f, 0f, null) // already fitted to the screen
    }
}

// The still Project P wallpaper (res/drawable-nodpi/pet_default_bg.png), fitted to
// the screen. The setup screens use the same picture.
// [EDIT] To change it, replace that PNG (same file name). In the Godot project
// that is addons/projectp_wallpaper/default_wallpaper.png.
// It's looked up by name, so this file works unchanged in the wallpaper-only app
// and inside the Godot game's app. If the PNG is missing, a plain colour is used.
private fun defaultPicture(): Bitmap? {
    if (defaultFitted != null || defaultTried || screenW <= 0f) return defaultFitted
    defaultTried = true
    try {
        val id = resources.getIdentifier("pet_default_bg", "drawable", packageName)
        if (id == 0) return null
        val raw = BitmapFactory.decodeResource(
            resources, id, BitmapFactory.Options().apply { inScaled = false }
        ) ?: return null
        defaultFitted = fitToScreen(
            raw, if (defaultBackgroundIsPixelArt) pixelPaint else smoothPaint
        )
        raw.recycle()
    } catch (e: Throwable) {
        defaultFitted = null
    }
    return defaultFitted
}

// Returns a copy of a picture that is exactly the size of the screen, "center
// cropped": it fills the whole screen without stretching, like a normal wallpaper
// (edges that don't fit are cut off evenly on both sides).
private fun fitToScreen(bmp: Bitmap, paint: Paint): Bitmap {
    val w = screenW.toInt().coerceAtLeast(1)
    val h = screenH.toInt().coerceAtLeast(1)
    val fitted = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888)
    drawFitted(Canvas(fitted), bmp, w, h, paint)
    return fitted
}

private fun drawFitted(canvas: Canvas, bmp: Bitmap, w: Int, h: Int, paint: Paint) {
    val scale = max(w.toFloat() / bmp.width, h.toFloat() / bmp.height)
    val drawnW = bmp.width * scale
    val drawnH = bmp.height * scale
    val left = (w - drawnW) / 2f
    val top = (h - drawnH) / 2f
    canvas.drawBitmap(bmp, null, RectF(left, top, left + drawnW, top + drawnH), paint)
}

// The same "center crop" for the GIF's current frame.
private fun drawGifCover(canvas: Canvas, movie: Movie) {
    val elapsed = SystemClock.uptimeMillis() - gifStartedAt
    movie.setTime((elapsed % movie.duration()).toInt())
    val scale = max(screenW / movie.width(), screenH / movie.height())
    canvas.save()
    canvas.translate(

```

```

        (screenW - movie.width() * scale) / 2f, (screenH - movie.height() * scale) / 2f)
        canvas.scale(scale, scale)
        movie.draw(canvas, 0f, 0f, smoothPaint)
        canvas.restore()
    }

    // ----- ANIMATED PROJECT P WALLPAPER -----
    // If you supply frames (pictures named in order, e.g. frame_00.jpg, frame_01.jpg...
    // in addons/projectp_wallpaper/default_wallpaper_frames/ of the Godot project, or
    // assets/wallpaper_bg_frames/ of the wallpaper-only app), the Project P wallpaper
    // plays them in a loop where moving backgrounds are allowed (SECTION 2).
    // [LOGIC] Only two or three frames are in memory at any time: the one on screen
    //         and the next one, which a helper thread loads and fits to the screen
    //         while the current one is showing. So the number of frames costs app
    //         size, not memory. If loading is slower than defaultAnimationFps, the
    //         animation just runs a little slower.
    // [FIX] Still picture although frames were added: the export's Output line must
    //       say "... background frames added", and the user must be using the
    //       Project P wallpaper (not their own picture).
    // Called every tick: shows the next frame when it's time, and orders the one after.
    // Where the animation doesn't play (home screen), only the first frame is loaded,
    // and after that whichever frame was last shown stays.
    private fun stepDefaultAnimation(now: Long) {
        if (!defaultHasFrames() || screenW <= 0f) return
        val playing = backgroundIsAnimating()
        if (!playing && shownFrame != null) return
        val next = readyFrame
        val interval = 1000L / defaultAnimationFps
        if (next != null && (shownFrame == null || now - frameShownSince >= interval)) {
            if (shownFrame == null) {
                // The first frame replaces the still picture, which is no longer needed
                defaultFitted?.recycle()
                defaultFitted = null
                defaultTried = false
            }
            spareFrame = shownFrame // the old frame's memory is used again
            shownFrame = next
            shownFrameIndex = readyFrameIndex
            readyFrame = null
            // Keep an even rhythm; start counting afresh after a pause
            frameShownSince =
                if (now - frameShownSince > 2 * interval) now else frameShownSince + interval
        }
        val wantNext = shownFrame == null || (playing && defaultFrameFiles.size > 1)
        if (wantNext && readyFrame == null && !frameIsLoading) {
            requestDefaultFrame((shownFrameIndex + 1) % defaultFrameFiles.size)
        }
    }

    // Asks the helper thread for one frame, fitted to the screen.
    private fun requestDefaultFrame(index: Int) {
        val name = defaultFrameFiles.getOrNull(index) ?: return
        val w = screenW.toInt().coerceAtLeast(1)
        val h = screenH.toInt().coerceAtLeast(1)
        val reuse = spareFrame?.takeIf { it.width == w && it.height == h && !it.isRecycled }
        spareFrame = null
        val batch = frameBatch
        frameIsLoading = true
        try {

```

```

        frameLoader.execute {
            // (helper thread) load the picture and draw it fitted into `target`
            val target = try {
                val raw = assets.open("${PetCatalog.DEFAULT_BG_FRAMES}/${name}")
                .use { BitmapFactory.decodeStream(it) }
                if (raw == null) null else {
                    val out = reuse ?: Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888)
                    out.eraseColor(fallbackColor) // wipe the frame it held before
                    drawFitted(Canvas(out), raw, w, h, Paint().apply {
                        isFilterBitmap = !defaultBackgroundIsPixelArt
                    })
                    raw.recycle()
                    out
                }
            } catch (e: Throwable) {
                null
            }
            // (back on the main thread) keep it, unless the screen size changed
            handler.post {
                if (batch == frameBatch) {
                    frameIsLoading = false
                    if (target != null) {
                        readyFrame = target
                        readyFrameIndex = index
                        frameFailures = 0
                    } else {
                        // Unreadable frame: skip it and carry on with the next.
                        // If no frame at all can be read, the still picture is
                        // used (see defaultHasFrames).
                        frameFailures++
                        shownFrameIndex = index
                        spareFrame = reuse
                    }
                }
            }
        }
    } catch (e: Throwable) {
        frameIsLoading = false // the wallpaper is shutting down
    }
}
// Drops the loaded frames (screen size changed). Frames still being prepared by
// the helper thread are ignored when they arrive.
private fun forgetDefaultFrames() {
    frameBatch++
    frameIsLoading = false
    // (a frame the helper thread is still working on is never one of these three)
    shownFrame?.recycle()
    readyFrame?.recycle()
    spareFrame?.recycle()
    shownFrame = null
    readyFrame = null
    spareFrame = null
    shownFrameIndex = -1
    readyFrameIndex = -1
    frameFailures = 0
}

// ===== SECTION 9: DRAWING EACH FRAME =====
// Keeps the pet fully on screen.

```

```

private fun clampPet() {
    petX = petX.coerceIn(0f, maxOf(0f, screenW - petW))
    petY = petY.coerceIn(0f, maxOf(0f, screenH - petH))
}

// Draws one picture: background first, then the current pet frame on top.
// Called by the animation timer and while dragging.
// [FIX] If the pet shows the wrong picture, check loadFrames() (top of this file) and
//       the names in PetCatalog.ANIMS.
private fun drawFrame() {
    val holder = surfaceHolder
    val canvas = try { holder.lockCanvas() } catch (e: Exception) { null } ?: return
    try {
        drawBackground(canvas)
        // The current frame; if this pet has no pictures for the animation, show
        // its resting picture instead of nothing.
        val picture = frames[current]?.getOrNull(frame)
            ?: frames["pet_idle"]?.firstOrNull().takeIf { hatched }
            ?: frames["egg_idle"]?.firstOrNull()
        val now = SystemClock.uptimeMillis()
        if (showMarks && markShown) drawSpotGlow(canvas, now)
        if (picture != null && !picture.isRecycled) { // (recycled = pet just changed)
            val dst = RectF(petX, petY, petX + petW, petY + petH)
            val mirrored = facingLeft
            if (mirrored) {
                // Mirror the pet so it faces the way it travels
                canvas.save()
                canvas.scale(-1f, 1f, petX + petW / 2f, petY + petH / 2f)
            }
            val glow = listeningGlow(now)
            if (glow > 0f) drawPetGlow(canvas, picture, glow) // under the pet
            canvas.drawBitmap(picture, null, dst, petPaint)
            if (mirrored) canvas.restore()
        }
    } catch (e: Throwable) {
        // skip this frame
    } finally {
        try { holder.unlockCanvasAndPost(canvas) } catch (e: Exception) { }
    }
}

// [LOGIC] Tap to move shows two soft glows instead of marks:
// - around the pet while it listens for the "go there" tap: the pet's own
//   shape, blurred and drawn under it, so it seems to light up;
// - at the spot it is travelling to: a small round light.
// Both breathe slowly, fade in, and the pet's glow fades out as the
// listening time ends. Each is a warm white light with a wider amber
// edge, so it shows on dark and on light pictures.
// [EDIT] Colours, strength and sizes: glowColor, glowEdgeColor, glowStrength,
// glowReach and spotGlowDp in SECTION 2; showMarks = false hides both.

// How strong the pet's glow is right now: 0 = none ... 1 = full.
private fun listeningGlow(now: Long): Float {
    if (!showMarks || !isListening() || moving || hopStartedAt != 0L) return 0f
    val fadeIn = ((now - listeningSince) / 250f).coerceIn(0f, 1f)
    val fadeOut = ((listeningUntil - now) / 600f).coerceIn(0f, 1f)
    return fadeIn * fadeOut * breath(now)
}

// A slow breath between 0.75 and 1, once every 1.6 seconds.

```

```

private fun breath(now: Long): Float =
    0.875f + 0.125f * sin((now % 1600L) / 1600f * 2f * PI.toFloat())

// The blurred outline of one frame. Made once per frame and kept, because
// blurring takes a moment. It is made from a small copy of the frame (a glow has
// no detail), which keeps it quick and light on memory.
private fun glowOf(picture: Bitmap): Glow? {
    glows[picture]?.let { return it }
    if (glows.size >= 160) forgetGlows() // (keeps memory small)
    var small: Bitmap? = null
    return try {
        val shrink = min(1f, 96f / max(picture.width, picture.height))
        small = Bitmap.createScaledBitmap(
            picture, max(1, (picture.width * shrink).toInt()),
            max(1, (picture.height * shrink).toInt()), true)
        val blur = Paint().apply {
            maskFilter = BlurMaskFilter(
                max(2f, small.width * glowReach), BlurMaskFilter.Blur.NORMAL)
        }
        val corner = IntArray(2)
        val shape = small.extractAlpha(blur, corner)
        Glow(shape, corner[0], corner[1], small.width, small.height)
        .also { glows[picture] = it }
    } catch (e: Throwable) {
        null // no glow rather than no pet
    } finally {
        if (small != null && small != picture) small.recycle()
    }
}

private fun forgetGlows() {
    val old = glows.values.toList()
    glows.clear()
    old.forEach { it.shape.recycle() }
}

// The glow around the pet. `level` = how strong (0..1).
private fun drawPetGlow(canvas: Canvas, picture: Bitmap, level: Float) {
    val glow = glowOf(picture) ?: return
    if (glow.shape.isRecycled) return
    val sx = petW / glow.frameW // glow pixels -> screen pixels
    val sy = petH / glow.frameH
    val dst = RectF(
        petX + glow.left * sx, petY + glow.top * sy,
        petX + (glow.left + glow.shape.width) * sx,
        petY + (glow.top + glow.shape.height) * sy)
    // The softer edge in the second colour, a little wider than the light
    val edge = RectF(dst)
    edge.inset(-petW * 0.04f, -petH * 0.04f)
    glowPaint.color = glowEdgeColor
    glowPaint.alpha = (level * glowStrength * 150f).toInt().coerceIn(0, 255)
    canvas.drawBitmap(glow.shape, null, edge, glowPaint)
    // The light itself
    glowPaint.color = glowColor
    glowPaint.alpha = (level * glowStrength * 255f).toInt().coerceIn(0, 255)
    canvas.drawBitmap(glow.shape, null, dst, glowPaint)
}

// The glow at the spot the pet is travelling to.
private fun drawSpotGlow(canvas: Canvas, now: Long) {

```

```

        val fadeIn = ((now - tripStartedAt) / 200f).coerceIn(0f, 1f)
        val level = fadeIn * breath(now)
        val radius = spotGlowDp * resources.displayMetrics.density * (0.9f + 0.1f * level)
        canvas.save()
        canvas.translate(markX, markY)
        canvas.scale(radius, radius)
        spotPaint.shader = spotEdge
        spotPaint.alpha = (level * glowStrength * 255f).toInt().coerceIn(0, 255)
        canvas.drawCircle(0f, 0f, 1.35f, spotPaint)
        spotPaint.shader = spotLight
        spotPaint.alpha = (level * min(1f, glowStrength * 1.3f) * 255f).toInt()
            .coerceIn(0, 255)
        canvas.drawCircle(0f, 0f, 1f, spotPaint)
        canvas.restore()
    }

    // ===== SECTION 10: HOME SCREEN OR LOCK SCREEN? =====
    // The lock screen gets the greeting, the phone-movement reactions and the animated
    // GIF. Android 14 and newer can tell a wallpaper which screen it's on; older
    // versions are asked whether the phone is locked.
    // [FIX] If the lock screen pet behaves like the home screen one (or the other way
    // round), this is where the decision is made.
    private val flagsMethod = try {
        Engine::class.java.getMethod("getWallpaperFlags") // Android 14+
    } catch (e: Throwable) {
        null
    }

    private fun updateLockState() {
        lastLockCheckAt = SystemClock.uptimeMillis()
        val flags = try { flagsMethod?.invoke(this) as? Int } catch (e: Throwable) { null }
        val nowOnLock = when {
            isPreview -> false
            flags == 2 -> true // this copy is only for the lock screen
            // this copy is only for the home screen (reliable from Android 15 on)
            flags == 1 && Build.VERSION.SDK_INT >= 35 -> false
            else -> keyguard.isKeyguardLocked // otherwise: is the phone locked?
        }
        val nowAwake = try { power.isInteractive } catch (e: Throwable) { true }
        val wokeUp = nowAwake && !awake
        val changed = nowOnLock != onLockScreen || nowAwake != awake
        onLockScreen = nowOnLock
        awake = nowAwake
        if (!changed) return
        gifStartedAt = SystemClock.uptimeMillis()
        if (!awake) {
            settleDown() // screen dimmed: put the pet down
            drawFrame() // and show this screen's picture
        }
        if (!onLockScreen && movingByTilt) stopMoving()
        updateSensor()
        // Phones with an always-on display keep the wallpaper "visible" while dimmed,
        // so the greeting is also given when the screen wakes up.
        if (wokeUp) refreshRest() // (unless the pet is asleep by now)
        if (wokeUp && onLockScreen && greetOnLockScreen) greet()
    }

    // ===== SECTION 11: WHAT THE PET DOES BY ITSELF; TRAVELLING, JUMPING =====
    // [LOGIC] While the hatched pet rests (pet_idle: blinking) and nobody is touching
    // it, a timer runs down (table.waitMin..waitMax seconds). Then, between

```

```

//      two blinks, it picks ONE action from the ACTIONS list at random (by
//      weight), does it, and goes back to resting; the timer starts again.
//      "blink" -> nothing new: it keeps resting and blinking
//      "travel" -> goes sideways to a random spot with that animation, facing
//                  the way it goes; fly_ frames bob in the air
//      "hop"    -> jumps up on the spot, then "landing"
//      "play"   -> plays that animation once where it stands. If the
//                  animation has several versions (ANIM_SETS, e.g. "trick"
//                  with two tricks of 8 frames), ONE version is played.
//      On a low battery it picks from the low-energy list instead, and while
//      it sleeps it does nothing by itself (SECTION 13).
//      Actions the pet has no frames for are left out. The list itself is in
//      pet.gd (one app with the game) or PetCatalog.kt (wallpaper-only app).
// [FIX] The pet only blinks and never does anything else: tap "Check my pet" in
//      the wallpaper setup. An action needs frames for its animation.
private fun resetRoamTimer() {
    nextMoveIn = table.waitMin + Random.nextFloat() * (table.waitMax - table.waitMin)
}

private fun stepActionTimer(dt: Float) {
    val allowed = if (onLockScreen) roamOnLockScreen else roamOnHomeScreen
    if (!allowed || !isFree() || moving || hopStartedAt != 0L) return
    if (current != "pet_idle" || screenW <= 0f) return
    if (isListening()) return // it's waiting to be sent somewhere
    // While the phone is held tilted to one side, the tilt decides where it goes
    val tilted = sensorOn && gravityKnown && abs(sideTilt) > tiltSideways &&
        SystemClock uptimeMillis() - lastSensorAt < 1000L
    if (tilted) return
    nextMoveIn -= dt
    if (nextMoveIn > 0f) return
    if (frame % table.blinkFrames != 0) return // let the current blink finish first
    resetRoamTimer()
    val action = pickAction() ?: return
    doAction(action)
}

// Can the pet do this action? (switched on, and it has the frames for it)
private fun canDo(action: Action) = action.weight > 0f && (action.kind == "blink" ||
    (if (action.kind == "travel") canTravel(action.anim) else has(action.anim)))
// Picks one action at random; a bigger weight is picked more often.
private fun pickAction(): Action? {
    val choices = actionsNow().filter { canDo(it) }
    var pick = Random.nextFloat() * choices.sumOf { it.weight.toDouble() }.toFloat()
    for (action in choices) {
        pick -= action.weight
        if (pick <= 0f) return action
    }
    return choices.lastOrNull()
}

// Starts one action.
private fun doAction(action: Action) {
    when (action.kind) {
        "travel" -> {
            val room = max(0f, screenW - petW)
            var target = Random.nextFloat() * room
            if (abs(target - petX) < screenW * 0.25f) {
                // Too short a trip: go towards the farther side of the screen
                target = if (petX > room / 2f) 0f else room
            }
        }
    }
}

```

```

        }
        startMoving(target, false, null, action.anim)
    }
    "hop" -> jumpTo(petX, petY, action.anim)
    "play" -> {
        if (animOf(action.anim).loop) return // a looping one would never end
        busy = true // no tickling until it has finished
        play(oneVersion(action.anim)) // ends in onAnimationFinished -> resting
        tickNow()
    }
    // "blink": nothing to do, it simply keeps resting
}
}

// [LOGIC] An animation listed in ANIM_SETS has several versions, made when the
// pet is loaded: "trick#1", "trick#2", ... This picks one of them at
// random. For any other animation it returns the animation itself.
private fun oneVersion(anim: String): String {
    var count = 0
    while (has("$anim#${count + 1}")) count++
    return if (count == 0) anim else "$anim#${1 + Random.nextInt(count)}"
}

// Sets off towards a spot (the pet's top-left corner). byTilt = started by tilting
// the phone. Without targetY the pet stays at its height (wandering, tilting).
private fun startMoving(
    targetX: Float, byTilt: Boolean, targetY: Float? = null, anim: String = "move"
) {
    if (!isFree() || hopStartedAt != 0L || !canTravel(anim)) return
    if (byTilt && current == "sleep") return // tilting doesn't wake it
    val lineY = if (moving) moveLineY else petY
    val toX = targetX.coerceIn(0f, max(0f, screenW - petW))
    val toY = (targetY ?: lineY).coerceIn(0f, max(0f, screenH - petH))
    if (hypot(toX - petX, toY - lineY) < 4f) return // already there
    val wasMoving = moving
    if (!moving) {
        moveLineY = petY
        moveTime = 0f
    }
    moving = true
    movingByTilt = byTilt
    moveTargetX = toX
    moveTargetY = toY
    if (abs(toX - petX) >= 4f) headingLeft = toX < petX // (straight up/down: as before)
    val (picked, mirrored) = travelPick(anim, headingLeft) ?: return
    travelAnim = picked
    facingLeft = mirrored
    if (current != picked) play(picked)
    if (!wasMoving) tickNow()
}

// [LOGIC] Which animation the pet travels with, and whether it is drawn mirrored.
// `base` is the animation named in ACTIONS, normally "move".
// going left -> "move_left" frames, if the pet has them
// going right -> "move_right" frames, if the pet has them
// only one of the two -> that one, mirrored for the other direction
// neither -> "move" itself (fly_ / walk_ / run_ frames), mirrored when
// it goes the other way than the frames face
// (moveFramesFaceRight, SECTION 2)
// Returns null if the pet has no frames to travel with.

```

```

private fun travelPick(base: String, goingLeft: Boolean): Pair<String, Boolean>? {
    val own = base + if (goingLeft) "_left" else "_right"
    val other = base + if (goingLeft) "_right" else "_left"
    return when {
        has(own) -> own to false
        has(other) -> other to true
        has(base) -> base to (goingLeft == moveFramesFaceRight)
        else -> null
    }
}

// Does the pet have frames to travel with (this animation or its _left / _right)?
private fun canTravel(base: String) =
    has(base) || has(base + "_left") || has(base + "_right")

// Moves the pet a little each tick, in a straight line, until it reaches its spot.
private fun stepMoving(dt: Float) {
    if (!moving) return
    moveTime += dt
    val step = moveSpeed * screenW * dt
    val gapX = moveTargetX - petX
    val gapY = moveTargetY - moveLineY
    val gap = hypot(gapX, gapY)
    if (gap <= step) {
        petX = moveTargetX
        moveLineY = moveTargetY
        stopMoving()
        return
    }
    petX += gapX / gap * step
    moveLineY += gapY / gap * step
    // Flying pets bob above their line of travel
    val bobs = travelAnim in bobAnims
    petY = moveLineY - (if (bobs) abs(sin(moveTime * 3f)) * moveBob * petH else 0f)
}

// Ends a trip: back to resting, facing forward, position saved.
private fun stopMoving() {
    if (!moving) return
    moving = false
    movingByTilt = false
    markShown = false
    petY = moveLineY
    facingLeft = false
    playIdle()
    savePosition()
    resetRoamTimer()
}

// [LOGIC] A hop in place: the pet plays "jump" while it rises and comes back down,
// then "landing", then rests. Used for the greeting and for tilting the
// phone up.
private fun hop() {
    if (!isFree() || hopStartedAt != 0L) return
    if (current == "sleep" || isAsleep()) return // a sleeping pet stays asleep
    stopMoving() // first back onto its line, then up from there
    jumpTo(petX, petY)
}

// [LOGIC] A jump to another spot (tap to move, up or down): the same as a hop, but

```

```

//      the pet travels to the spot in an arc. The 8 jump frames play as it takes
//      off and the last one is held while it is in the air; the 8 landing frames
//      play when it arrives. Jumping down, it rises a little first and then
//      drops. Longer jumps take longer (jumpSpeed) and arc higher (jumpArc).
private fun jumpTo(toX: Float, toY: Float, anim: String = "jump") {
    if (!isFree() || hopStartedAt != 0L) return
    stopMoving()
    busy = true
    hopFromX = petX
    hopFromY = petY
    hopToX = toX.coerceIn(0f, max(0f, screenW - petW))
    hopToY = toY.coerceIn(0f, max(0f, screenH - petH))
    val distance = hypot(hopToX - hopFromX, hopToY - hopFromY)
    val seconds = max(hopSeconds, distance / (jumpSpeed * max(screenH, 1f)))
    hopMillis = seconds * 1000f
    hopArc = hopHeight * petH + jumpArc * distance
    hopStartedAt = SystemClock.uptimeMillis()
    play(if (has(anim)) anim else "jump")
    tickNow()
}
private fun stepHop(now: Long) {
    if (hopStartedAt == 0L) return
    val t = (now - hopStartedAt) / hopMillis // 0 at the start, 1 at the end
    if (t >= 1f) {
        endHop()
        return
    }
    petX = hopFromX + (hopToX - hopFromX) * t
    petY = hopFromY + (hopToY - hopFromY) * t - sin(t * PI.toFloat()) * hopArc
}

// Touchdown: the pet is at its spot; landing frames, then resting.
private fun endHop() {
    if (hopStartedAt == 0L) return
    hopStartedAt = 0L
    markShown = false
    val travelled = hopToX != hopFromX || hopToY != hopFromY
    petX = hopToX
    petY = hopToY
    if (travelled) {
        savePosition()
        resetRoamTimer()
    }
    if (has("landing")) play("landing") else playIdle() // landing -> idle
}

// [LOGIC] Greeting when the lock screen appears. Not more often than every
//      5 seconds, so quick screen flickers don't repeat it.
//      If the pet has its own greeting frames (greet_00.png, hello_00.png or
//      wave_00.png, see "greet" in PetCatalog.ANIMS) they are played; otherwise
//      the pet greets with a hop.
private fun greet() {
    val now = SystemClock.uptimeMillis()
    if (now - lastGreetAt < 5000L) return
    if (isAsleep()) return // it sleeps through the screen turning on
    lastGreetAt = now
    if (has("greet") && isFree()) {
        stopMoving()
        busy = true
    }
}

```

```

        play("greet")
        tickNow()
    } else {
        hop()
    }
}

// Called when the wallpaper is hidden: put the pet down where it belongs.
// [LOGIC] A pet on its way to a tapped spot is put at that spot. But if the
// wallpaper is hidden within a moment of the tap, the tap really opened an
// app (its icon was under the finger): then the pet goes back where it was.
private fun settleDown() {
    val onTapTrip = markShown && (moving || hopStartedAt != 0L)
    val tapOpenedAnApp =
        onTapTrip && SystemClock.uptimeMillis() - tripStartedAt < UNDO_TRIP_MILLIS
    if (moving && onTapTrip) { // finish the trip at once
        petX = moveTargetX
        moveLineY = moveTargetY
    }
    stopMoving()
    if (hopStartedAt != 0L) { // finish the jump at once
        hopStartedAt = 0L
        petX = hopToX
        petY = hopToY
        savePosition()
        playIdle()
    }
    if (tapOpenedAnApp) {
        petX = tripFromX
        petY = tripFromY
        savePosition()
    }
    // never left hanging in the air, or frozen on a travelling frame
    if (hatched && current != restAnim()) playIdle()
    markShown = false
    listeningUntil = 0L
    fingerDown = false
    dragging = false
    moved = false
}

// ===== SECTION 12: PHONE MOVEMENT (SHAKE AND TILT) =====
// Uses the phone's movement sensor, which also works on the lock screen.
// [LOGIC] Shake the phone -> tickle (a sleeping pet wakes up instead).
// Tilt the top up quickly -> hop.
// Tilt sideways -> travel towards the lower side, for as long as
// the phone stays tilted.
// The sensor is only switched on while the wallpaper is visible on a screen where
// it's allowed (SECTION 2), so it costs no battery the rest of the time.
private val sensorListener = object : SensorEventListener {
    override fun onSensorChanged(event: SensorEvent) {
        try {
            onPhoneMoved(event.values[0], event.values[1], event.values[2])
        } catch (e: Throwable) {
            // ignore
        }
    }
    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) { }
}

private fun updateSensor() {
    val allowed = if (onLockScreen) motionOnLockScreen else motionOnHomeScreen
    val want = visible && awake && allowed && accelerometer != null
    if (want && !sensorOn) {
        gravityKnown = false
        sensorOn = sensorManager.registerListener(
            sensorListener, accelerometer, SensorManager.SENSOR_DELAY_UI)
    } else if (!want && sensorOn) {
        sensorManager.unregisterListener(sensorListener)
        sensorOn = false
    }
}

// x, y, z = the sensor reading: x points to the phone's right, y to its top, z out
// of the screen. At rest the reading points "up" with a strength of about 9.8.
// (This assumes the phone is used upright, like the game.)
private fun onPhoneMoved(x: Float, y: Float, z: Float) {
    val now = SystemClock.uptimeMillis()
    if (!gravityKnown) {
        gravX = x; gravY = y; gravZ = z
    }
}

```

```

        restX = x.coerceIn(-0.8f * tiltSideways, 0.8f * tiltSideways)
        sideTilt = gravX - restX
        slowLift = liftOf(x, y, z)
        gravityKnown = true
        lastSensorAt = now
        return
    }
    // Seconds since the last reading. The smoothing below uses real time, so it
    // behaves the same whether the phone sends 15 or 100 readings a second.
    val dt = ((now - lastSensorAt).coerceIn(1L, 200L)) / 1000f
    lastSensorAt = now
    // Split the reading: the slow part is how the phone is held (gravity), the
    // fast part is the hand moving it.
    val follow = 1f - exp(-dt / 0.4f)
    gravX += follow * (x - gravX)
    gravY += follow * (y - gravY)
    gravZ += follow * (z - gravZ)
    val dx = x - gravX
    val dy = y - gravY
    val dz = z - gravZ
    val jolt = sqrt(dx * dx + dy * dy + dz * dz)

    // --- Shake: two sharp movements within 0.7 seconds ---
    if (jolt > shakeStrength) {
        if (now - firstJoltAt > 700L) {
            firstJoltAt = now
            joltCount = 0
            lastCountedJoltAt = 0L
        }
        if (now - lastCountedJoltAt > 150L) { // one count per back-and-forth
            joltCount++
            lastCountedJoltAt = now
        }
        lastJoltAt = now
        if (joltCount >= 2 && now - lastShakeAt > 1500L) {
            lastShakeAt = now
            joltCount = 0
            if (!wakeUp()) tickle() // asleep: the shake wakes it
        }
        return
    }
    if (now - lastJoltAt < 400L) return // still being shaken: ignore tilts

    // --- Tilt sideways: travel towards the lower side ---
    // The tilt is measured against how the phone is normally held (restX), not
    // against perfectly level: most people hold a phone a little to one side, and
    // that must not send the pet to the edge or stop it doing things by itself.
    // restX follows the hand slowly, so a tilt has to be a clear movement.
    // sideTilt is below zero when the right edge was lowered, above zero for the left.
    // It only follows up to a point, so a phone held far over to one side still
    // counts as tilted, and levelling it again doesn't send the pet the other way.
    restX += (1f - exp(-dt / 6f)) * (gravX - restX)
    restX = restX.coerceIn(-0.8f * tiltSideways, 0.8f * tiltSideways)
    sideTilt = gravX - restX
    val onTapTrip = moving && !movingByTilt && markShown // sent by a tap: let it go
    if (abs(sideTilt) > tiltSideways && !onTapTrip) {
        val room = max(0f, screenW - petW)
        startMoving(if (sideTilt < 0f) room else 0f, true)
    } else if (moving && movingByTilt && abs(sideTilt) < tiltSideways * 0.6f) {
        stopMoving() // phone is back as it was held
    }

```

```

    }

    // --- Tilt up: the top of the phone raised quickly ---
    // slowLift follows the phone's angle with a delay of about a second. A quick
    // raise gets ahead of it; a slow one (just picking the phone up) doesn't.
    val lift = liftOf(gravX, gravY, gravZ)
    if (now - lastTiltHopAt < 2000L) {
        slowLift = lift // just hopped: start measuring afresh
    } else {
        slowLift += (1f - exp(-dt / 1.0f)) * (lift - slowLift)
        if (lift - slowLift > tiltUpDegrees) {
            lastTiltHopAt = now
            slowLift = lift
            hop()
        }
    }
}

// How far the top of the phone is raised, in degrees: 0 = lying flat (or on its
// side), 90 = standing upright, below 0 = top pointing down.
private fun liftOf(x: Float, y: Float, z: Float): Float {
    val strength = sqrt(x * x + y * y + z * z)
    if (strength < 1f) return 0f // falling or no reading: no angle
    return Math.toDegrees(asin((y / strength).coerceIn(-1f, 1f)).toDouble()).toFloat()
}

// ===== SECTION 13: SLEEPING AT NIGHT AND LOW ENERGY =====
// [LOGIC] SLEEP. Between table.sleepFrom and table.sleepUntil by the phone's own
// clock (11 PM to 6 AM unless changed) the hatched pet rests with its
// "sleep" frames instead of pet_idle. Asleep, it does nothing by itself,
// doesn't greet and isn't moved by tilting the phone.
// WAKING IT UP: tap it (home screen) or shake the phone (lock screen). It
// is then awake for table.awakeMinutes (5): it blinks, does its actions,
// can be tickled and sent to a spot. Every tap or shake in that time
// starts the minutes again. After that it goes back to sleep; in the
// morning it wakes up by itself.
// A pet without sleep frames simply stays awake.
// [LOGIC] LOW ENERGY. While the battery is at or below table.lowBatteryPercent
// (15) and the phone isn't charging, the pet picks what it does from
// the low-energy list (its low_energy movements, one version at a
// time) instead of ACTIONS. Plug the phone in and it is its old self.
// [EDIT] The hours, the minutes awake, the percent and the low-energy list: in
// pet.gd SECTION 0 (one app with the game) or PetCatalog.kt (wallpaper-
// only app). sleepsAtNight / tiresOnLowBattery (SECTION 2) switch them off
// for the wallpaper alone.
// [FIX] The pet never sleeps or never looks tired: tap "Check my pet" in the
// wallpaper setup. It shows whether the pet has sleep and low_energy
// frames, the hours, and the battery level the wallpaper sees.

// Looks at the phone's clock (at most every 5 seconds).
private fun readClock(force: Boolean = false) {
    val now = SystemClock.uptimeMillis()
    if (!force && now - clockReadAt < 5000L) return
    clockReadAt = now
    val hour = Calendar.getInstance().get(Calendar.HOUR_OF_DAY) // 0..23
    val from = table.sleepFrom
    val until = table.sleepUntil
    bedtime = sleepsAtNight && (
        if (from <= until) hour >= from && hour < until // e.g. 1 to 6
        else hour >= from || hour < until // e.g. 23 to 6
    )
}

```

```

    }

    // Looks at the battery (at most every 30 seconds).
    private fun readBattery(force: Boolean = false) {
        val now = SystemClock.uptimeMillis()
        if (!force && now - batteryReadAt < 30000L) return
        batteryReadAt = now
        val (percent, plugged) = phoneBattery(this@PetWallpaperService)
        lowEnergy = tiresOnLowBattery && percent >= 0 && !plugged &&
            percent <= table.lowBatteryPercent
    }

    // Is the pet woken up for a while?
    private fun isWokenUp() = SystemClock.elapsedRealtime() < awakeUntil

    // Should the pet be asleep right now?
    private fun isAsleep() = hatched && bedtime && has("sleep") && !isWokenUp()

    // The animation the pet rests with right now.
    private fun restAnim() = when {
        !hatched -> "egg_idle"
        isAsleep() -> "sleep"
        else -> "pet_idle"
    }

    // The screen just came on: look at the clock and the battery right away, and put
    // the pet to sleep or wake it without waiting, so it is never seen "wrong".
    private fun refreshRest() {
        readClock(true)
        readBattery(true)
        if (!busy && !moving && hopStartedAt == 0L &&
            (current == "pet_idle" || current == "sleep") && current != restAnim())
            playIdle()
    }

    // Falling asleep and waking up by itself: the resting animation it should have
    // now is another one than it is playing (the clock passed the sleeping hour, the
    // morning came, or its minutes awake are over).
    private fun stepRest() {
        readClock()
        readBattery()
        if (!hatched || busy || dragging || moving || hopStartedAt != 0L) return
        if (current != "pet_idle" && current != "sleep") return
        if (current == restAnim()) return
        // (an awake pet finishes its blink first)
        if (current == "sleep" || frame % table.blinkFrames == 0) playIdle()
    }

    // A tap or a shake: keeps the pet awake for awakeMinutes (this only matters at
    // night). Returns true if the pet was asleep and this woke it up.
    private fun wakeUp(): Boolean {
        if (!hatched) return false
        if (bedtime) {
            awakeUntil =
                SystemClock.elapsedRealtime() + (table.awakeMinutes * 60000f).toLong()
        }
        if (current != "sleep") return false
        playIdle() // back to blinking
        resetRoamTimer()
        tickNow()
        return true
    }

    // The list the pet picks its next action from: the low-energy list while the
    // battery is low and the pet has frames for something in it, otherwise ACTIONS.
    private fun actionsNow(): List<Action> =
        if (lowEnergy && table.lowActions.any { it.kind != "blink" && canDo(it) }) {
            table.lowActions
        } else {
            table.actions
        }
    }
}

```

B5. The app screen: a short setup that asks the user

Opening the **Project P Pet** app icon (in Part C: tapping **Wallpaper Selection** on the game's main menu) walks the user through these questions. Every screen shows the Project P wallpaper picture with the question and buttons on a soft panel.

1. "Show your pet on your home screen and lock screen?"

- **No, not now** - nothing changes; the pet stays in the app. They can open the app again any time.
- **Yes, show my pet** - goes to question 2.

2. "Keep your current wallpaper behind the pet?"

- **Yes, keep my wallpaper** - Android's photo picker opens and they pick the picture they use as their wallpaper now. Their home screen then looks the same as before, with the pet added. A GIF can be picked too (step C9).
- **No, use the Project P wallpaper** - the living-room picture from step B2 is used.

2b. "Is your lock screen wallpaper a different picture?" (asked after the picture is picked)

- **No, it's the same picture** - the one picture is used on both screens.
- **Yes, pick my lock screen picture** - the photo picker opens again for the lock screen picture. Each screen then keeps its own picture, and the pet is on both.

The pictures only have to be picked once. After that, question 2 offers **Keep the picture(s) I chose before**, **Pick different pictures** and **Use the Project P wallpaper**.

3. "Last step" - Android's wallpaper preview opens, and they tap **Set wallpaper** and choose **Home and lock screens**.

When the pet is already on the home screen, opening the app shows **Change background** (back to question 2) and **Remove pet from home screen** (opens Android's wallpaper settings so they can pick another wallpaper) instead.

Why question 2 uses the photo picker: a live wallpaper always replaces the phone's wallpaper, and Android can't make it see-through. An app can't fetch the current wallpaper by itself either. On Android 13 the system hands back the phone's built-in default wallpaper instead of the user's, and on Android 14 and newer it refuses. The only way around that is the "All files access" permission, which Google Play doesn't allow for this kind of app. The photo picker needs **no permissions** and only hands the app the one picture the user picks.

Create another Kotlin file in the same folder, name it **SetWallpaperActivity**, and replace its contents with the file **SetWallpaperActivity.kt** (code shown below for reference):

```
// =====  
// SetWallpaperActivity.kt - THE WALLPAPER SETUP SCREENS  
// =====  
// What this file does:  
//   Asks the user, step by step:  
//   1. Show your pet on your home screen and lock screen?   Yes / No  
//   1b. Which pet? (only shown when PetCatalog.kt lists more than one pet)  
//   2. (Yes) Keep your current wallpaper behind the pet?  
//       Yes -> pick that picture (or GIF) from the gallery (photo picker)  
//       No  -> use the Project P wallpaper  
//   2b. (after picking) Is your lock screen wallpaper different?  
//       Yes -> pick the lock screen picture too  
//       No  -> the same picture is used on both screens  
//   3. Opens Android's wallpaper preview -> user chooses "Home and lock screens"  
//   If the pet is already on the home screen, it offers "Change pet" (when there are  
//   several), "Change background" and "Remove pet from home screen" instead.  
//   "Check my pet" shows what the wallpaper found in the app for the chosen pet: how  
//   many frames each animation has and which of its actions it can do. Use it when the  
//   pet doesn't fly, walk or do its other movements on the phone.  
//   Every screen shows the Project P wallpaper picture (pet_default_bg.png) behind a
```

```

// soft panel with the question and the buttons.
//
// How it is opened:
// - One app with the Godot game: from the "Wallpaper Selection" button in the game's
//   main menu (main_menu.gd -> WallpaperBridge.kt). There is no separate app icon.
//   Closing these screens, or pressing Back, returns to the main menu.
// - Wallpaper-only app: from the app icon.
//
// Adding more pets: you don't need to change this file. Add them in PetCatalog.kt
// ([ADD PET]); the "Which pet?" screen lists them automatically.
//
// One app with the Godot game: the pet is chosen in the game, so "Which pet?" and
// "Change pet" are skipped here. Changing the pet never changes the background.
//
// Google Play safe: needs no permissions. Android's own photo picker only gives the
// app the one picture the user picks. (Android doesn't let apps read the wallpaper that
// is already set, which is why the user picks the picture once.)
//
// How to find things:
// [EDIT] = text or buttons you can safely change
// [LOGIC] = which screen leads to which
// [FIX] = places to look first if something goes wrong
//
// Sections in this file:
// 1. Settings (file names of the saved pictures) -> search "SECTION 1"
// 2. The question screens (incl. "Which pet?") -> search "SECTION 2"
// 3. Helpers (screen look, status checks) -> search "SECTION 3"
// 4. Photo picker -> search "SECTION 4"
// 5. Opening Android's wallpaper preview -> search "SECTION 5"
//
// Related files:
// PetCatalog.kt - the list of pets shown on the "Which pet?" screen
// PetWallpaperService.kt - the wallpaper itself (reads the photo and pet saved here)
// WallpaperBridge.kt - lets the Godot game open these screens (one app)
// res/drawable-nodpi/pet_default_bg.png - the picture behind these screens
// =====

package com.yourname.projectp.wallpaper

import android.app.Activity
import android.app.WallpaperManager
import android.content.ComponentName
import android.content.Intent
import android.content.res.ColorStateList
import android.graphics.Color
import android.graphics.drawable.GradientDrawable
import android.os.Build
import android.os.Bundle
import android.provider.MediaStore
import android.view.Gravity
import android.view.ViewGroup.LayoutParams.MATCH_PARENT
import android.view.ViewGroup.LayoutParams.WRAP_CONTENT
import android.widget.Button
import android.widget.FrameLayout
import android.widget.ImageView
import android.widget.LinearLayout
import android.widget.ScrollView
import android.widget.TextView
import android.widget.Toast
import java.io.File

```

```

class SetWallpaperActivity : Activity() {

    // ===== SECTION 1: SETTINGS =====
    companion object {
        // Names of the user's pictures inside the app's private storage: the one for the
        // home screen (also used on the lock screen if there is no separate one), and the
        // one for the lock screen. PetWallpaperService reads these same files.
        const val BACKGROUND_FILE = "background.img"
        const val LOCK_BACKGROUND_FILE = "background_lock.img"
        // IDs used to recognise the photo picker's answer in onActivityResult():
        // which picture was being picked.
        private const val PICK_HOME_PICTURE = 1
        private const val PICK_LOCK_PICTURE = 2

        // [EDIT] Colours of the setup screens (the picture behind them is
        // res/drawable-nodpi/pet_default_bg.png). "#E6" at the front of the panel
        // colour is how solid it is: FF = solid, 00 = see-through.
        private const val PANEL_COLOR = "#E6FFF8EC" // the soft panel
        private const val TEXT_COLOR = "#4A3423" // the question text
        private const val BUTTON_COLOR = "#B5683C" // the buttons
        private const val BUTTON_TEXT_COLOR = "#FFFFFF" // the text on the buttons
        private const val EDGE_COLOR = "#F3E3CF" // shown if the picture is missing
    }

    // Saved settings shared with the wallpaper (which pet was chosen).
    private val prefs by lazy { getSharedPreferences(PetCatalog.PREFS, MODE_PRIVATE) }
    // Every pet this app can show (from the game's pet.gd, or PetCatalog.PETS).
    private val pets by lazy { availablePets(this) }

    // true after we send the user to an Android screen (wallpaper preview or wallpaper
    // settings), so we show the first screen again with the new status when they return.
    private var backFromPreview = false

    // Runs when the setup is opened (from the main menu's Wallpaper Selection button, or
    // the app icon in the wallpaper-only app).
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        showStart()
    }

    // Runs every time this screen comes back into view.
    override fun onResume() {
        super.onResume()
        if (backFromPreview) { // user came back from Android's wallpaper preview
            backFromPreview = false
            showStart()
        }
    }

    // ===== SECTION 2: THE QUESTION SCREENS =====
    // [EDIT] All the text the user sees is in the screen(...) calls below.
    // Each screen is: screen("message", "button text" to { what it does }, ...)

    // [LOGIC] First screen.
    // Pet already on the home screen -> Change pet / Change background / Remove pet / Close
    // Not yet -> Question 1: show the pet? Yes / No
    private fun showStart() {
        if (isPetWallpaperOn()) {
            val buttons = mutableListOf

```

```

    },
    "No, not now" to {
        screen(
            "Okay! Your pet will stay in the app.\n\n" +
            "Come back here any time if you change your mind.",
            "Close" to { finish() },
            "Back" to { showStart() }
        )
    },
    "Check my pet" to { showPetCheck() }
)
}
}

// [LOGIC] "Check my pet": what the wallpaper found in this app for the chosen pet.
// - how many frames each animation got (0 = it can't play that one),
// - which of its actions (the things it does by itself) are ready,
// - its tricks and low-energy movements (how many versions), sleep and size,
// - whether it has hatched (an egg doesn't move about).
// [FIX] An animation shows 0 although the game plays it: export the game again with
// the add-on enabled, and read the "Project P Wallpaper:" lines in Godot's
// Output panel. They list the same numbers at export time.
private fun showPetCheck() {
    val pet = chosenPet()
    val table = petTable(this)
    val files = petFrameFiles(this, pet.folder)
    fun count(anim: String) = files[anim]?.size ?: 0
    val text = StringBuilder("Pet check: ${pet.name}\n\nFrames per animation\n")
    for (name in table.anims.keys) {
        // Optional ones are left out when empty: the greeting, and pet_emerge when
        // hatching and emerging are one animation (egg_hatch_emerge_.. frames).
        if ((name == "greet" || name == "pet_emerge") && count(name) == 0) continue
        text.append("$name: ${count(name)}")
        // Movements with several versions: one version is played at a time
        val perVersion = table.sets[name] ?: 0
        if (perVersion > 0 && count(name) > 0) {
            val versions = count(name) / perVersion
            val spare = count(name) - versions * perVersion
            text.append(
                if (versions > 0) " = $versions x $perVersion (one is played at a time)"
                else " (fewer than $perVersion: played as one)"
            )
            if (versions > 0 && spare > 0) text.append(", $spare left over")
        }
    }
    text.append("\n")
}

// One line per action: ready, or why it is skipped.
fun describe(actions: List<Action>) {
    for (action in actions) {
        if (action.kind == "blink" || action.weight <= 0f) continue
        val loops = table.anims[action.anim]?.loop == true
        // Travelling also works with the _left / _right frames of its animation
        val frames = if (action.kind == "travel") {
            count(action.anim) + count(action.anim + "_left") +
            count(action.anim + "_right")
        } else {
            count(action.anim)
        }
        text.append("${action.kind} with ${action.anim}: ").append(
            when {
                frames == 0 -> "NO FRAMES, so it is skipped\n"
            }
        )
    }
}

```

```

        action.kind == "play" && loops -> "skipped, the animation loops\n"
        else -> "ready\n"
    }
}
}
text.append("\nWhat it does by itself\n")
describe(table.actions)
text.append("every ${table.waitMin.toInt()} to ${table.waitMax.toInt()} seconds\n")

// Low energy and sleep
val (percent, plugged) = phoneBattery(this)
text.append("\nOn a low battery (${table.lowBatteryPercent}% or less, not charging)\n")
describe(table.lowActions)
val hasLowFrames = table.lowActions.any { a ->
    a.kind != "blink" && a.weight > 0f && (count(a.anim) > 0 || (a.kind == "travel" &&
        count(a.anim + "_left") + count(a.anim + "_right") > 0))
}
if (!hasLowFrames) text.append("nothing ready, so it keeps doing the list above\n")
text.append("Battery now: ")
    .append(if (percent < 0) "unknown" else "$percent%")
    .append(if (plugged) ", charging\n" else "\n")
text.append("\nSleep: ")
if (count("sleep") == 0) {
    text.append("NO sleep frames, so it stays awake\n")
} else if (table.sleepFrom == table.sleepUntil) {
    text.append("switched off (both hours are the same)\n")
} else {
    text.append("from ${table.sleepFrom}:00 to ${table.sleepUntil}:00; ").append(
        if (table.awakeMinutes <= 0f) "it can't be woken (AWAKE_MINUTES is 0)\n"
        else "a tap or a shake wakes it for ${table.awakeMinutes} minutes\n")
}

// Size
text.append("\nSize: ")
val savedSize = GameSave.size(filesDir, pet.id)
text.append(
    when {
        !table.fromGame -> "drawScale in PetWallpaperService.kt\n"
        savedSize != null -> "$savedSize, as set in the game\n"
        table.defaultSize > 0f -> "${table.defaultSize}, pet_size in pet.gd " +
            "(not resized in the game yet)\n"
        else -> "drawScale in PetWallpaperService.kt\n"
    })
val hatched = prefs.getBoolean("hatched_${pet.id}", false) ||
    GameSave.isHatched(filesDir, pet.id)
text.append("\nHatched: ").append(if (hatched) "yes" else "no (an egg stays put)")
text.append("\nLists come from: ")
    .append(if (table.fromGame) "the game (pet.gd)" else "this app (PetCatalog.kt)")
screen(text.toString(), "Back" to { showStart() })
}

// [LOGIC] Question 1b: which pet? One button per available pet.
// Saves the choice for the wallpaper, then runs `next` (the following question).
// [ADD PET] Nothing to change here: new pets in PetCatalog.kt appear automatically.
private fun askWhichPet(next: () -> Unit) {
    val buttons = pets.map { p ->
        p.name to {
            prefs.edit().putString(PetCatalog.KEY_PET, p.id).apply()
            next()
        }
    }
}

```

```

    }
}
screen(
    "Which pet should live on your home screen?",
    *buttons.toTypedArray(),
    "Back" to { showStart() }
)
}

// [LOGIC] Question 2: keep the current wallpaper?
// Yes -> photo picker (SECTION 4), then question 2b
// No -> delete any saved pictures so the Project P wallpaper is used, then finishSetup()
// If pictures were already picked earlier, they can simply be kept (no gallery needed).
private fun askKeepWallpaper() {
    if (hasOwnPhoto()) {
        screen(
            "Which background should be behind the pet?",
            (if (hasLockPhoto()) "Keep the pictures I chose before"
            else "Keep the picture I chose before") to { finishSetup() },
            "Pick different pictures" to { pickBackground(PICK_HOME_PICTURE) },
            "Use the Project P wallpaper" to { useProjectPWallpaper() },
            "Back" to { showStart() }
        )
        return
    }
    screen(
        "Keep your current wallpaper behind the pet?\n\n" +
        "If yes, pick the picture you use as your home screen wallpaper now from " +
        "your gallery. A GIF is animated on the lock screen.",
        "Yes, keep my wallpaper" to { pickBackground(PICK_HOME_PICTURE) },
        "No, use the Project P wallpaper" to { useProjectPWallpaper() },
        "Back" to { showStart() }
    )
}

// [LOGIC] Question 2b, asked after the home screen picture was picked:
// is the lock screen wallpaper a different picture?
// Yes -> photo picker again, for the lock screen picture, then finishSetup()
// No -> delete any saved lock screen picture (both screens use the same one)
// The pet is on both screens either way.
private fun askLockScreenPicture() {
    val buttons = mutableListOf<Pair<String, () -> Unit>>()
    buttons += "No, it's the same picture" to {
        File(filesDir, LOCK_BACKGROUND_FILE).delete()
        finishSetup()
    }
    buttons += "Yes, pick my lock screen picture" to { pickBackground(PICK_LOCK_PICTURE) }
    if (hasLockPhoto()) {
        buttons += "Keep the lock screen picture I chose before" to { finishSetup() }
    }
    buttons += "Back" to { askKeepWallpaper() }
    screen(
        "Is your lock screen wallpaper a different picture?\n\n" +
        "If yes, pick it too, so each screen keeps its own picture behind the pet.",
        *buttons.toTypedArray()
    )
}

// Removes the user's pictures, so the Project P wallpaper is used on both screens.
private fun useProjectPWallpaper() {

```

```

        File(filesDir, BACKGROUND_FILE).delete()
        File(filesDir, LOCK_BACKGROUND_FILE).delete()
        finishSetup()
    }

    // [LOGIC] Last screen.
    //   Pet already on -> just confirm (the wallpaper reloads the background by itself)
    //   Not yet       -> explain the last step, then open Android's wallpaper preview
    private fun finishSetup() {
        if (isPetWallpaperOn()) {
            // Already on: the wallpaper picks up the new pet or background by itself
            screen(
                "Done! You'll see the change next time you go to your home screen.",
                "Close" to { finish() }
            )
        } else {
            screen(
                "Last step!\n\nOn the next screen, tap \"Set wallpaper\" " +
                    "and choose \"Home and lock screens\".",
                "Continue" to { openWallpaperPreview() },
                "Back" to { askKeepWallpaper() }
            )
        }
    }
}

// ===== SECTION 3: HELPERS =====

/** Shows a message with a column of buttons. Every screen above uses this. */
// [EDIT] To change how all screens look (text size, spacing, colours), edit this
//         function and the colours in SECTION 1.
// Layout: the Project P wallpaper picture fills the screen; on top of it a rounded
// panel holds the message and the buttons. The panel scrolls if there are many
// buttons (e.g. a long "Which pet?" list).
private fun screen(message: String, vararg buttons: Pair<String, () -> Unit>) {
    val dp = resources.displayMetrics.density
    val pad = (24 * dp).toInt() // 24dp spacing

    // The panel: message at the top, then one button per choice
    val panel = LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        gravity = Gravity.CENTER_HORIZONTAL
        setPadding(pad, pad, pad, pad)
        background = GradientDrawable().apply {
            setColor(Color.parseColor(PANEL_COLOR))
            cornerRadius = 24 * dp
        }
    }
    panel.addView(TextView(this).apply {
        text = message
        textSize = 18f
        setTextColor(Color.parseColor(TEXT_COLOR))
        gravity = Gravity.CENTER
        setPadding(0, 0, 0, pad / 2)
    })
    for ((label, action) in buttons) {
        panel.addView(Button(this).apply {
            text = label
            isAllCaps = false
            setTextColor(Color.parseColor(BUTTON_TEXT_COLOR))
            backgroundTintList = ColorStateList.valueOf(Color.parseColor(BUTTON_COLOR))
        })
    }
}

```

```

        setOnClickListener { action() }
    }, LinearLayout.LayoutParams(MATCH_PARENT, WRAP_CONTENT).apply {
        topMargin = (8 * dp).toInt()
    })
}

// The picture behind everything, filling the screen without stretching
val root = FrameLayout(this)
root.setBackgroundColor(Color.parseColor(EDGE_COLOR))
val pictureId = resources.getIdentifier("pet_default_bg", "drawable", packageName)
if (pictureId != 0) {
    root.addView(ImageView(this).apply {
        setImageResource(pictureId)
        scaleType = ImageView.ScaleType.CENTER_CROP
    }, FrameLayout.LayoutParams(MATCH_PARENT, MATCH_PARENT))
}
// The panel sits in the middle; the scroll area keeps it usable on small screens
val holder = FrameLayout(this).apply { setPadding(pad, pad, pad, pad) }
holder.addView(panel, FrameLayout.LayoutParams(MATCH_PARENT, WRAP_CONTENT, Gravity.CENTER))
root.addView(ScrollView(this).apply {
    isFillViewport = true // keeps short screens centred
    addView(holder, FrameLayout.LayoutParams(MATCH_PARENT, WRAP_CONTENT))
}, FrameLayout.LayoutParams(MATCH_PARENT, MATCH_PARENT))
setContentView(root)
}

// Should we ask "Which pet?" Only with more than one pet in PetCatalog.kt, and only
// when the Godot game hasn't chosen one (inside the game's app, the game decides).
private fun hasSeveralPets() =
    pets.size > 1 && GameSave.chosenPet(filesDir) == null

// The pet that will be shown: the game's choice, else the one picked here, else
// the first pet.
private fun chosenPet() = PetCatalog.byId(
    pets, GameSave.chosenPet(filesDir) ?: prefs.getString(PetCatalog.KEY_PET, null)
)

// Has the user picked their own picture? (false = Project P wallpaper is used)
private fun hasOwnPhoto() = File(filesDir, BACKGROUND_FILE).exists()
// Has the user picked a separate picture for the lock screen?
private fun hasLockPhoto() = hasOwnPhoto() && File(filesDir, LOCK_BACKGROUND_FILE).exists()

// Is our pet wallpaper the phone's current wallpaper?
// [FIX] If the first screen shows the wrong status, check this.
private fun isPetWallpaperOn(): Boolean {
    val info = WallpaperManager.getInstance(this).wallpaperInfo ?: return false
    return info.packageName == packageName &&
        info.serviceName == PetWallpaperService::class.java.name
}

// "Remove pet from home screen": apps can't put back the user's old wallpaper,
// so this opens Android's own wallpaper settings for the user to pick one.
private fun removePet() {
    // Let the user pick a different wallpaper in Android's own wallpaper settings
    val intent = Intent(Intent.ACTION_SET_WALLPAPER)
    try {
        startActivity(Intent.createChooser(intent, "Choose a new wallpaper"))
        backFromPreview = true
    } catch (e: Exception) {
    }
}

```

```

        val msg = "Open your phone's wallpaper settings to choose another wallpaper."
        Toast.makeText(this, msg, Toast.LENGTH_LONG).show()
    }
}

// ===== SECTION 4: PHOTO PICKER (no permission needed) =====
// Opens Android's photo picker so the user can choose their wallpaper picture.
// Android 13+ uses the new photo picker; older phones use the file chooser.
// Only pictures are offered (photos, PNGs, GIFs...), not videos.
// `which` says which picture is being picked: PICK_HOME_PICTURE or PICK_LOCK_PICTURE.
private fun pickBackground(which: Int) {
    val intent = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        // Android 13+ photo picker, pictures only
        Intent(MediaStore.ACTION_PICK_IMAGES).apply { type = "image/*" }
    } else {
        Intent(Intent.ACTION_GET_CONTENT).apply { type = "image/*" }
    }
    try {
        startActivityForResult(intent, which)
    } catch (e: Exception) {
        // [FIX] Some phones don't have the new picker: fall back to the file chooser.
        try {
            val fallback = Intent(Intent.ACTION_GET_CONTENT).apply { type = "image/*" }
            startActivityForResult(fallback, which)
        } catch (e2: Exception) {
            val msg = "No gallery app was found to pick a picture."
            Toast.makeText(this, msg, Toast.LENGTH_LONG).show()
        }
    }
}

// Runs when the photo picker closes.
// Saves a copy of the chosen picture for PetWallpaperService. After the home screen
// picture it asks about the lock screen (question 2b); after the lock screen picture
// it goes to finishSetup().
// A GIF is copied as it is, so the wallpaper can animate it on the lock screen.
@Deprecated("Deprecated in Java")
override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {
    super.onActivityResult(requestCode, resultCode, data)
    if (requestCode != PICK_HOME_PICTURE && requestCode != PICK_LOCK_PICTURE) return
    val fileName =
        if (requestCode == PICK_LOCK_PICTURE) LOCK_BACKGROUND_FILE else BACKGROUND_FILE
    val uri = data?.data
    if (resultCode != RESULT_OK || uri == null) return // cancelled: stay on the question
    try {
        // Copy the photo into the app's own storage so the wallpaper can always read it.
        // Written to a .tmp file first, then renamed, so the wallpaper never reads a
        // half-written photo.
        val temp = File(filesDir, "$fileName.tmp")
        temp.delete() // leftover from an earlier try
        val input = contentResolver.openInputStream(uri)
            ?: throw IllegalStateException("picture can't be opened")
        input.use { temp.outputStream().use { output -> it.copyTo(output) } }
        if (temp.length() == 0L || !temp.renameTo(File(filesDir, fileName))) {
            throw IllegalStateException("picture can't be saved")
        }
        if (requestCode == PICK_HOME_PICTURE) askLockScreenPicture() else finishSetup()
    } catch (e: Exception) {
        // [FIX] Shown if the photo can't be read (e.g. a cloud photo that isn't downloaded).
        val msg = "Couldn't use that picture. Try another one."
    }
}

```

```

        Toast.makeText(this, msg, Toast.LENGTH_LONG).show()
    }
}

// ===== SECTION 5: OPEN ANDROID'S WALLPAPER PREVIEW =====
// Opens the preview for our pet wallpaper; the user taps "Set wallpaper" there.
// Apps can't set a live wallpaper by themselves, so this step is always the user's.
// [FIX] "This phone doesn't support live wallpapers": a few phones (some Android Go
// models) have no live wallpapers at all; the pet then stays in the app.
private fun openWallpaperPreview() {
    try {
        startActivity(
            Intent(WallpaperManager.ACTION_CHANGE_LIVE_WALLPAPER).putExtra(
                WallpaperManager.EXTRA_LIVE_WALLPAPER_COMPONENT,
                ComponentName(this, PetWallpaperService::class.java)
            )
        )
        backFromPreview = true
    } catch (e: Exception) {
        val msg = "This phone doesn't support live wallpapers."
        Toast.makeText(this, msg, Toast.LENGTH_LONG).show()
    }
}
}

```

B6. Register everything in the manifest

Open `app/src/main/AndroidManifest.xml`. Add the `uses-feature` line **above** `<application>`, and paste the `activity` and `service` blocks **inside** `<application>` (leave its existing attributes as they are). No `uses-permission` lines are needed.

```

<uses-feature android:name="android.software.live_wallpaper" android:required="true" />

<application ... (leave the existing attributes as they are)>

    <activity
        android:name=".SetWallpaperActivity"
        android:exported="true"
        android:label="Project P Pet"
        android:theme="@android:style/Theme.DeviceDefault.Light.NoActionBar">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <service
        android:name=".PetWallpaperService"
        android:exported="true"
        android:label="Project P Pet"
        android:permission="android.permission.BIND_WALLPAPER">
        <intent-filter>
            <action android:name="android.service.wallpaper.WallpaperService" />
        </intent-filter>
        <meta-data
            android:name="android.service.wallpaper"
            android:resource="@xml/pet_wallpaper" />
    </service>

</application>

```

B7. Build and install

- 1 With USB debugging on (see step A8), plug in your phone, choose it in the device dropdown at the top, and press the green **Run** button.
- 2 If you'd rather have a file: go to **Build > Build APK(s)** (in newer versions, **Build > Generate App Bundles or APKs > Generate APKs**). The APK is saved in `app/build/outputs/apk/debug/app-debug.apk`. Install it the same way as the game.

- 3 Open the **Project P Pet** icon and answer the questions: **Yes, show my pet**, then either **Yes, keep my wallpaper** (pick the picture you use as your wallpaper now) or **No, use the Project P wallpaper**.
- 4 On the last screen tap **Continue**, then **Set wallpaper**, and choose **Home and lock screens**.
- 5 To change the background or remove the pet later, open the app again. A new background shows the next time you go to the home screen.

If you want to keep a different picture on your lock screen, choose **Home screen** only in step 4. The pet then won't appear on the lock screen.

Part C - One app: the wallpaper inside the game

Godot can build extra Android code into the game's app when **Use Gradle Build** is turned on. The add-on in `ProjectP_Wallpaper_Addon.zip` uses that. Every time you export for Android, it:

- copies the wallpaper's code and resources into Godot's Android build folder, so they're compiled into the same app,
- copies the Project P wallpaper picture (`default_wallpaper.png`) into the app,
- registers the wallpaper, its setup screens and the bridge to the game in the app's manifest,
- reads the pet list (`PETS`) from `pet.gd` and adds each pet's PNG frames as plain files, so the wallpaper shows the same pets with the same frames. The frames can be in any folder or subfolder.
- sorts the frames into animations with `pet.gd`'s own rules and hands the wallpaper `pet.gd`'s `ANIMS` and `ACTIONS`, so the wallpaper pet has the same animations, speeds and behaviour as the pet in the game.
- adds the animation frames of the Project P wallpaper, if you supplied any (step C10).

The result is **one APK with one app icon**. The game is the app; the wallpaper setup opens from the **Wallpaper Selection** button in the game's main menu. They share the same save, so the wallpaper shows the pet chosen in the game and hatches when it hatches there. You don't need the Android Studio project from Part B.

What has been tested: all the wallpaper code, including the bridge to the game, compiles against the Android API and Godot's Android library; the add-on puts the files in the right places for both Godot build layouts (4.5 and older, 4.6 and newer) and its sorted frame lists are read back correctly by the wallpaper's code; the pet's new behaviour runs in Godot 4.3 and 4.5; the pet picker with the picture runs in Godot 4.3 and 4.5. **What has not been tested:** the final Gradle build and the behaviour on a phone (the Wallpaper Selection button, tap to move, the lock screen reactions, the separate lock screen picture, the GIF, the animated Project P wallpaper), because Android's build tools and a phone weren't available where this guide was made. If the build shows an error, copy the first red lines from Godot's Output panel.

C1. Finish Part A first

- 1 The game exports and runs on your phone (steps A7 and A8).
- 2 `PETS` in `pet.gd` lists every pet with the exact folder of its frames, including capital letters, for example `"dragon": "res://Pets/Dragon/frames"`, . The folders can be anywhere in the project; the add-on uses whatever `PETS` points to.
- 3 The pet grid and the main menu are in the scene (steps A5 and A6). The menu provides the **Wallpaper Selection** button.

C2. Turn on Gradle Build

- 1 Go to **Project > Export** and select your **Android** preset.
- 2 Under **Gradle Build**, tick **Use Gradle Build**.
- 3 If red text appears about the Android build template, follow it. In older Godot versions choose **Project > Install Android Build Template** and click **Install**. In newer versions, make sure everything under **Android** is installed in **Editor > Manage Export Templates** (step A7), including the source template.
- 4 Export once as in step A8 to check the game still builds. The first Gradle build downloads extra tools, so it needs internet and takes several minutes.

Check: your project folder now contains `android/build`, and the exported game installs and runs as before.

C3. Add the wallpaper add-on

- 1 Unzip `ProjectP_Wallpaper_Addon.zip`. It contains one folder named `addons`.
- 2 In your computer's file explorer, copy that `addons` folder into your Godot project folder, next to `project.godot`. If the project already has an `addons` folder, copy `projectp_wallpaper` into it instead.
- 3 In Godot, go to **Project > Project Settings**, open the **Plugins** tab, and tick **Enable** next to **Project P Wallpaper**.

Check: the FileSystem panel shows `res://addons/projectp_wallpaper/` with `plugin.gd` and `default_wallpaper.png`. Its `android` folder doesn't show in Godot on purpose; you can see it in your file explorer.

Updating from an earlier version of the add-on:

- 1 Close Godot.
- 2 In your file explorer, delete the folder `addons/projectp_wallpaper` in your project, then copy the new `projectp_wallpaper` folder from the zip into `addons`.
- 3 Replace `pet_picker.gd` and `main_menu.gd` with the new files (steps A5 and A6). Replace `pet.gd` with the new one and put your pets back into its `PETS` list, or update your own `pet.gd` with the eleven edits in `pet_gd_edits_2.txt` (step A4).
- 4 If you had put animation frames in the old add-on's `default_wallpaper_frames` folder, copy them into the new one (step C10).
- 5 Open Godot. In **Project > Project Settings > Plugins**, check **Project P Wallpaper** is still enabled (version 1.8).
- 6 Export again (step C5). Installing the new APK over the old one removes a second icon left by the first version.

C4. Nothing to set up for the pets

The wallpaper uses the same pets as the game. On every export the add-on:

- reads `PETS` from `res://pet.gd`,
- collects each pet's PNGs from its folder, subfolders included, and sorts them into animations exactly as the game does,

- packs them into the app together with the list of pet ids, the animation list (**ANIMS**), the action list (**ACTIONS**), the settings for tricks, low energy and sleep, and the game's screen size (so the pet can be shown as big as in the game).

So whatever you change in **pet.gd** (a new pet, new file names in **ANIMS**, a new action, other waiting times, other sleeping hours) reaches the wallpaper with the next export. There is nothing to keep in sync by hand.

When you add a pet to **PETS** in **pet.gd**, it's on the wallpaper after the next export. The name shown on the wallpaper's screens is the id with capitals, so **ice_dragon** becomes "Ice Dragon".

Only if your pet script is not **res://pet.gd** (renamed, or in a folder): open **addons/projectp_wallpaper/plugin.gd** and change **PET_SCRIPT** in SECTION 1 to its path.

C5. Export and install the one APK

- 1 Export as in step A8, or click the Android icon at the top right of Godot to install straight to your phone.
- 2 Watch the **Output** panel. For each pet it lists the frames per animation, for example **Project P Wallpaper: dragon: egg_idle 8, ... pet_idle 32, move_left 8, move_right 8, trick 16, low_energy 8, sleep 8**, and then **Project P Wallpaper: wallpaper code installed, 152 frame PNGs added for pets: dragon** (your numbers and pets may differ). A yellow line such as **pet 'dragon' has no frames for 'move'** means the wallpaper pet can't do that; check the file names against **ANIMS**.
- 3 On the phone there is one icon: **Project P**. Open it. On the main menu tap **Pet Selection**, choose your pet and hatch it.
- 4 Tap **Menu** (top left), then **Wallpaper Selection**. The setup opens on the living-room picture.
- 5 Answer the questions: **Yes, show my pet**, then keep your wallpaper (pick the home screen picture, then say whether the lock screen has a different one and pick that too) or use the Project P wallpaper, then **Continue, Set wallpaper** and **Home and lock screens**.
- 6 Press Back to return to the main menu.

Check: the pet on your home screen is the one chosen in the game, it's already hatched, and it's as big as in the game. Tap it (it glows softly), then tap another spot: it travels there. Switch pets in the game, go back to the home screen, and the pet changes while the background stays the same. Lock the phone and turn the screen on: the pet hops to greet you.

C6. What the game and the wallpaper share

- **Chosen pet:** the game decides. The wallpaper's own "Which pet?" question is skipped. Changing the pet changes only the pet; the background is kept.
- **Hatched:** when a pet hatches in the game, it's hatched on the wallpaper too. Hatching the egg on the wallpaper (3 taps) doesn't hatch it in the game.
- **Size:** the game decides. The wallpaper pet is as big as you made it in the game, and follows when you resize it there (step C12).
- **Position:** separate. The wallpaper pet stays where you send it on the home screen (and where its last trip ended).
- **Awake at night:** separate. Waking the pet in the game doesn't wake the one on the home screen, and the other way round.
- **Background:** belongs to the wallpaper only (your pictures or the Project P wallpaper).

- **Save file:** the wallpaper only reads the game's save (`pet.cfg`). It never changes it.
- **Separate processes:** the wallpaper runs in its own process, named `:wallpaper`. Godot shuts its own process down whenever the game is closed; when the two shared a process, that took the wallpaper down with it, and Android puts the phone's default wallpaper back when a wallpaper dies twice in a few seconds. Kept apart, closing the game doesn't touch the wallpaper.

C7. Moving the pet on the home screen (tap to move)

A live wallpaper only gets a copy of each touch. The home screen sees the same finger, and Android gives a wallpaper no way to keep a touch for itself. So a finger drag on the pet also opened the app list or switched pages. Taps don't have that problem: a tap on empty space does nothing on the home screen. The pet is therefore moved with two taps:

- 1 **Tap the pet.** It giggles (ticklish, then recovery) and a soft glow appears around it: it is listening for 4 seconds. The glow fades as the time runs out.
- 2 **Tap the spot where it should go.** A soft light glows at the spot and the pet travels there, so that its middle ends up on the spot. The light goes out when the pet arrives.

How it travels depends on where the spot is, seen from the pet:

- **Mostly to the left or right:** it travels in a straight line with its `move_left` or `move_right` frames (or its one `move` set, turned that way), as when it travels by itself. Flying pets bob on the way.
- **Mostly up or down:** it jumps there. The 8 jump frames (`jump_land_a`) play as it takes off, and the last one is held while it is in the air. The 8 landing frames (`jump_land_b`) play when it arrives, then it rests. Jumping down, it rises a little first and then drops.
- **Diagonally:** whichever direction is larger decides. A jump can also drift sideways.
- **A pet without move frames** jumps everywhere, and a pet without jump frames travels everywhere with its move frames.

Swipes and long presses are left to the home screen: they open the app list, switch pages or open the home screen menu as usual, and the pet stays where it is. The egg can be tapped to hatch it, but it doesn't travel. On the lock screen nothing changes (step C8).

Settings (SECTION 2 of `PetWallpaperService.kt`):

- `tapToMove` - the whole feature, on or off.
- `listenSeconds` - how long the pet waits for the second tap (4).
- `flyersJumpUpAndDown` - `true`: every pet jumps to spots above or below it. `false`: pets with `fly_` frames fly there, and only walking pets jump.
- `jumpSpeed`, `jumpArc`, `hopHeight` - how fast a jump travels and how high its arc is.
- `showMarks` - the glow around the listening pet and the light at the destination. `false` hides both.
- `glowColor`, `glowEdgeColor`, `glowStrength`, `glowReach`, `spotGlowDp` - how the glow looks: its colour (a warm white), the colour of its softer outer edge (amber, so it also shows on a white picture), how strong it is (`0.7f`; use `0.4f` for fainter, `1f` for strongest), how far it reaches around the pet, and the size of the light at the destination.
- `dragToMove` - `false` by default. Set it to `true` to also allow carrying the pet with a finger, with the overlap described above.
- `useHomeScreenTapReports` - see the note below.

Taps on app icons. Many home screens (the Pixel one and those built on the same base) tell the wallpaper when the user taps empty space. Where that happens, only those taps send the pet somewhere, so tapping an app icon while the pet listens just opens the app. On home screens that don't report taps, any tap counts; if that tap opens an app, the pet is put back where it was. Which kind your phone's home screen is can only be seen on the phone. If the pet glows but never goes to the tapped spot, set `useHomeScreenTapReports` to `false`.

A tap counts as "on the pet" when it lands on the drawn part of the picture, or within a fingertip of it. The empty corners of the frame don't count, so a big pet doesn't swallow taps meant for the app icons around it (`tapReachDp` in SECTION 7).

The Godot game is not affected: inside the app there is no home screen underneath, so the pet is still dragged there as before.

C8. The pet on the lock screen

Android doesn't pass touches to a wallpaper on the lock screen, so the pet can't be tapped or dragged there. These make it feel alive without touch:

- **Greeting:** when the screen turns on and shows the lock screen, the pet hops (its jump animation with a small leap, then landing). If a pet has frames named `greet_00.png`, `hello_00.png` or `wave_00.png`, those are played instead.
- **Shake the phone:** the pet is tickled (ticklish, then recovery). At night a shake wakes the sleeping pet instead (step C12).
- **Tilt the top of the phone up quickly:** the pet hops. Slowly picking the phone up doesn't count.
- **Tilt the phone sideways:** the pet flies or walks towards the lower side while the phone is tilted. The tilt is measured from the way you normally hold the phone, so holding it a little to one side doesn't count; it takes a clear tilt of about 20 degrees.
- **By itself:** between blinks it does the things from the `ACTIONS` list (travels, hops, plays one of its tricks), every 3-7 seconds, like in the game. This also happens on the home screen.
- **Asleep:** a sleeping pet doesn't greet, hop or travel when the phone is tilted. It sleeps until it is shaken awake or the morning comes.

An egg doesn't do any of this; hatch it first (in the game, or with 3 taps on the home screen).

Settings (SECTION 2 of `PetWallpaperService.kt`, in `addons/projectp_wallpaper/android/src/`; export again after changing them):

- `greetOnLockScreen` - the greeting, on or off.
- `motionOnLockScreen`, `motionOnHomeScreen` - shake and tilt on each screen. The home screen is off by default, because ordinary handling of the phone would keep setting the pet off.
- `roamOnLockScreen`, `roamOnHomeScreen` - doing things by itself on each screen. What it does and how often comes from `ACTIONS`, `ACTION_WAIT_MIN` and `ACTION_WAIT_MAX` in `pet.gd`.
- `moveSpeed`, `moveBob`, `moveFramesFaceRight` - the same as the travelling settings in `pet.gd`. `moveSpeed` is in screen widths per second: 0.25 crosses the screen in 4 seconds.
- `hopHeight`, `hopSeconds` - how high and how long a hop is.
- `shakeStrength` - higher means the phone must be shaken harder. `tiltSideways` - higher means more sideways tilt is needed (2.5 needs about 20 degrees). `tiltUpDegrees` - how far the top must be raised for a hop.

Which screen is it? Android 14 and newer tell a wallpaper whether it is showing on the lock screen. On older versions the wallpaper asks whether the phone is locked. If the lock screen pet behaves like the home screen one, look at SECTION 10 of `PetWallpaperService.kt`.

C9. Backgrounds: your pictures, a GIF, the Project P wallpaper

What is drawn behind the pet, in this order:

- 1 On the lock screen: your lock screen picture, if you picked one.
 - 2 Your home screen picture, if you picked one. It is also used on the lock screen when you said both screens are the same.
 - 3 Otherwise the Project P wallpaper: animated if you supplied frames (step C10), else the still picture.
- **Different home and lock wallpapers:** after you pick the home screen picture, the setup asks **Is your lock screen wallpaper a different picture?** Pick it too, and each screen keeps its own picture with the pet on both. Set the pet wallpaper for **Home and lock screens** as usual; the wallpaper itself swaps the picture when the phone locks and unlocks.
 - **Your pictures stay:** they are copied into the app's private storage, so they remain even if the originals are deleted. Changing the pet doesn't affect them.
 - **A GIF:** pick a GIF as either picture. It plays on the lock screen and stays still (its first frame) on the home screen. Smaller GIFs (about 480 pixels wide) play best; a very large one may be too big to load and then shows as a still picture. Videos (MP4) aren't supported; the picker only offers pictures.
 - **The Project P wallpaper** is `addons/projectp_wallpaper/default_wallpaper.png`. The same file is the default wallpaper, the picture behind the wallpaper setup screens, and the picture behind the game's Choose your pet screen. To change all three, replace that file with your own PNG (same name) and export again.
 - **Where moving backgrounds play:** `animateBackgroundOnLockScreen` (on) and `animateBackgroundOnHomeScreen` (off) in SECTION 2 of `PetWallpaperService.kt`. They apply to GIFs and to the animated Project P wallpaper. Playing on the home screen as well costs more battery.

On phones older than Android 14 one copy of the wallpaper serves both screens, so the picture is swapped at the moment of locking and unlocking. You may see the other picture for a split second during the unlock animation.

Why the app can't fetch your current wallpaper by itself: Android stopped letting apps read the wallpaper. On Android 13 an app gets the phone's built-in default wallpaper instead of yours, and on Android 14 and newer the request is refused. Only apps with "All files access" can still read it, and Google Play grants that permission only to apps such as file managers. So each picture is picked once in the gallery, and after that the setup offers **Keep the picture(s) I chose before**.

C10. An animated Project P wallpaper (your own frames)

The Project P wallpaper can be a short looping animation instead of a still picture, for example the curtain swaying and the light shifting in the living room. You supply the frames; the add-on packs them and the wallpaper plays them. With no frames, the still picture is used and nothing else changes.

Prepare the frames:

- **Names:** in playing order, with two digits: `frame_00.jpg`, `frame_01.jpg`, `frame_02.jpg`, ... Any name works as long as the numbers put them in order.
- **File type:** JPG, PNG or WebP. JPG is best here: the files are much smaller and load faster.

- **Size:** all the same size, portrait, the same shape as `default_wallpaper.png`. 1080 x 1920 pixels is a good size.
- **How many:** 8 to 24 frames make a good loop. They play at 8 frames per second, so 16 frames is a 2-second loop. The last frame should lead back into the first.
- **The still picture:** save your first frame as a PNG named `default_wallpaper.png` as well (next step), so the still and the animation match.

Put them in the project:

- 1 In your computer's file explorer (not in Godot), open your project folder, then `addons/projectp_wallpaper/default_wallpaper_frames/`. It contains a `README.txt` and a hidden `.gdignore` file; leave both.
- 2 Copy your frames into that folder, directly, not in a subfolder.
- 3 Replace `addons/projectp_wallpaper/default_wallpaper.png` with your first frame saved as PNG (same file name).
- 4 In Godot, export again (step C5).

Check: the Output panel shows `Project P Wallpaper: animated default wallpaper, 16 background frames added`. (your number). On the phone, open the wallpaper setup, choose **Use the Project P wallpaper**, lock the phone and turn the screen on: the background plays. On the home screen it stands still.

Godot doesn't show the `default_wallpaper_frames` folder on purpose. The `.gdignore` file keeps the frames out of the game itself, where they aren't used and would only make the app bigger.

Settings (SECTION 2 of `PetWallpaperService.kt`):

- `defaultAnimationFps` - the speed. 8 is calm; use 12 for smoother movement, which needs more frames for the same loop length.
- `animateBackgroundOnLockScreen`, `animateBackgroundOnHomeScreen` - where it plays.

Good to know:

- **Memory:** only the frame on screen and the next one are loaded at any time, so more frames make the app bigger but don't use more memory.
- **App size:** 16 JPG frames at 1080 x 1920 add roughly 4-8 MB. The same frames as PNG can add 30 MB or more.
- **Home screen:** it shows the frame the animation stopped on when you unlocked (the first frame after a restart). Keep the frames close to each other so this looks natural.
- **If it stays still:** the user must be on the Project P wallpaper, not their own picture; the Output line must say frames were added; and the folder must hold at least two frames.

C11. Changing things later

- **Wallpaper code:** edit the files in `addons/projectp_wallpaper/android/src/` and export again. Don't edit the copies inside `android/build`; they're replaced on every export.
- **After updating Godot:** nothing to do. If Godot resets `android/build`, the add-on copies the wallpaper files in again on the next export.
- **Names on the phone:** the labels "Project P Wallpaper" (title of the setup screens) and "Project P Pet" (name in Android's wallpaper list) are in SECTION 1 of `addons/projectp_wallpaper/plugin.gd`.

- **Colours of the setup screens:** SECTION 1 of `SetWallpaperActivity.kt` (`PANEL_COLOR`, `TEXT_COLOR`, `BUTTON_COLOR`).
- **A second icon for the wallpaper setup:** off by default. Set `SHOW_SETUP_ICON` to `true` in SECTION 1 of `plugin.gd` only if the Wallpaper Selection button doesn't work on some phone.
- **Opening the setup from your own button:** call `$PetPicker.open_wallpaper_setup()` from any script in the main scene. To open the main menu from your own script, call `$MainMenu.open_menu()`.
- **One set of frames for the menu and the wallpaper:** set `USE_MENU_BACKGROUND_AS_WALLPAPER` to `true` in SECTION 1 of `plugin.gd`. If `default_wallpaper_frames` is empty, the menu's background frames (`res://menu/background/`) are then also used as the animated Project P wallpaper (step C10). Leave it `false` if your menu background has a title or logo painted into it.
- **If you installed the Part B app:** uninstall it, so the phone doesn't list two "Project P Pet" wallpapers.

C12. Size, tricks, low energy and sleep on the wallpaper

The wallpaper takes all of this from the game, so there is nothing to set up. After changing anything in `pet.gd`, export again.

Size: the same as in the game

- The wallpaper pet is as big on the home and lock screen as the pet is in the game.
- **To resize it:** open the game, put one finger on the pet, add a second finger, and spread or pinch them. Go back to the home screen: the pet has the new size and stays centred on the same spot.
- A pet that was never resized in the game has the size of **Pet Size** in `pet.gd` (2).
- **Why it isn't resized on the home screen itself:** a wallpaper only gets a copy of each touch (step C7), and a pinch on the home screen already belongs to the home screen: on most phones it opens the home screen's own menu. Both would happen at once. Copying the size from the game has no such overlap, and the pet looks the same in both places.
- To give the wallpaper its own fixed size instead, set `sizeFollowsGame` to `false` in SECTION 2 of `PetWallpaperService.kt` and change `drawScale` there.

Tricks: one at a time

- As in the game, a trick is one row of 8 frames, and the pet picks one each time (`ANIM_SETS` in `pet.gd`, step A4).

Low energy

- While the battery is at 15% or less and the phone isn't charging, the wallpaper pet picks from `LOW_ENERGY_ACTIONS` instead of `ACTIONS`: it does one of its low-energy movements, or keeps blinking. Tapping it, sending it to a spot and the lock screen reactions still work.
- It looks at the battery about every 30 seconds while the screen is on, so it can take that long after plugging in or unplugging.

Sleep, and how to wake the pet

- From 11 PM to 6 AM by the phone's clock the pet plays its sleep frames instead of blinking. Asleep, it does nothing by itself, doesn't greet you on the lock screen and isn't moved by tilting the phone.
- **Home screen: tap the pet.** It wakes up and blinks again. Tap it once more and it giggles, glows, and can be sent to a spot as usual.
- **Lock screen: shake the phone.** (The lock screen doesn't pass touches to a wallpaper.)

- Once woken it stays awake for 5 minutes (**AWAKE_MINUTES**), on both screens. Every tap or shake in that time starts the 5 minutes again. Then it goes back to sleep. At 6 AM it wakes up by itself.

Settings: the hours, the minutes awake, the battery percent and the two lists are in SECTION 0 of **pet.gd** (step A4). To switch sleeping or low energy off for the wallpaper only, set **sleepsAtNight** or **tiresOnLowBattery** to **false** in SECTION 2 of **PetWallpaperService.kt**.

Check on the phone: open the game, go to **Wallpaper Selection** and tap **Check my pet**. It lists **trick: 16 = 2 x 8 (one is played at a time)**, the low-energy actions with the battery level the wallpaper sees, the sleeping hours (or "NO sleep frames"), and the size it uses.

To try sleep on the phone without staying up: set **SLEEP_FROM_HOUR** in **pet.gd** to the current hour and **SLEEP_UNTIL_HOUR** to the next one, export, and look at the home screen. Put them back to 23 and 6 afterwards.

For reference, this is what **WallpaperBridge.kt** contains (it is already inside the add-on; nothing to copy):

```
// =====
// WallpaperBridge.kt - THE DOOR BETWEEN THE GODOT GAME AND ANDROID (one app only)
// =====
// What this file does:
//   The game and the wallpaper are one app with one icon. This small file is the door
//   between them: it gives the game (GDScript) these things it can call.
//   openWallpaperSetup() -> opens the setup screens (SetWallpaperActivity.kt)
//   isWallpaperOn()      -> true if the pet wallpaper is the phone's wallpaper now
//   batteryPercent()     -> how full the battery is, 0..100 (-1 = unknown)
//   isCharging()         -> true while the phone is plugged in
//   The last two are what pet.gd uses for the pet's low-energy behaviour (search
//   "_is_low_energy" there). Reading the battery level needs no permission.
//
// How the game uses it (see main_menu.gd, the "Wallpaper Selection" button):
//   if Engine.has_singleton("ProjectPWallpaper"):
//       Engine.get_singleton("ProjectPWallpaper").openWallpaperSetup()
//
// Only for the one-app setup (Part C). The Godot add-on copies this file into the
// Android build and registers it in the manifest. Do NOT add it to the wallpaper-only
// Android Studio project (Part B); it needs Godot's library and won't compile there.
//
// How to find things:
//   [EDIT] = something you can safely change
//   [FIX]  = places to look first if something goes wrong
//
// [FIX] "Wallpaper Selection" does nothing on the phone (or shows the message meant
//       for computers): the name returned by getPluginName() below must match
//       PLUGIN_NAME in the add-on's plugin.gd and WALLPAPER_PLUGIN in main_menu.gd and
//       pet_picker.gd ("ProjectPWallpaper").
//
// Related files:
//   SetWallpaperActivity.kt - the screens this opens
//   PetWallpaperService.kt  - the wallpaper itself
//   addons/projectp_wallpaper/plugin.gd - registers this file when the game is exported
// =====

package com.yourname.projectp.wallpaper

import android.app.WallpaperManager
import android.content.Intent
```

```

import org.godotengine.godot.Godot
import org.godotengine.godot.plugin.GodotPlugin
import org.godotengine.godot.plugin.UsedByGodot

class WallpaperBridge(godot: Godot) : GodotPlugin(godot) {

    // The name the game asks for: Engine.get_singleton("ProjectPWallpaper")
    override fun getPluginName() = "ProjectPWallpaper"

    /** Opens the wallpaper setup screens on top of the game. */
    @UsedByGodot
    fun openWallpaperSetup() {
        val game = activity ?: return
        game.runOnUiThread {
            try {
                game.startActivity(Intent(game, SetWallpaperActivity::class.java))
            } catch (e: Exception) {
                // [FIX] Nothing opens: check the <activity> entry in the add-on's plugin.gd.
            }
        }
    }

    /** Is the pet wallpaper the phone's current wallpaper? */
    @UsedByGodot
    fun isWallpaperOn(): Boolean {
        val game = activity ?: return false
        return try {
            val info = WallpaperManager.getInstance(game).wallpaperInfo
            info != null && info.packageName == game.packageName &&
                info.serviceName == PetWallpaperService::class.java.name
        } catch (e: Exception) {
            false
        }
    }

    /** How full the battery is: 0..100, or -1 if the phone doesn't say. */
    @UsedByGodot
    fun batteryPercent(): Int {
        val game = activity ?: return -1
        return phoneBattery(game).first
    }

    /** Is the phone plugged in (charging or full)? */
    @UsedByGodot
    fun isCharging(): Boolean {
        val game = activity ?: return false
        return phoneBattery(game).second
    }
}

```

If the pet doesn't fly, walk or do its movements on the phone:

- 1 On the phone, open the game, go to **Wallpaper Selection**, and tap **Check my pet**.
- 2 Read **Frames per animation**. **egg_hatch** must be more than 0 for the egg to hatch with its animation, **move_left** and **move_right** (or **move**) for flying or walking, **jump** and **landing** for hopping, **trick** for the tricks, **low_energy** for the low-energy movements, **sleep** for sleeping. A 0 means the wallpaper got no frames for that animation.
- 3 Read **What it does by itself**. Each action says "ready" or why it is skipped.
- 4 Read **Hatched**. An egg doesn't move about; hatch it in the game or with 3 taps on the home screen.
- 5 If an animation shows 0: in Godot, check the same animation in the export's Output line for that pet. If it is 0 there too, the file names don't match its list in **ANIMS (pet.gd)**; add the name and export again.
- 6 If everything is "ready": watch the home screen for about 15 seconds without touching it. The pet acts every 3-7 seconds of blinking, and only while the home or lock screen is showing.

If something goes wrong:

- **Output says "'Use Gradle Build' is off ... the wallpaper is NOT included"**: do step C2.

- **Output says "Android build template not found":** install the build template (step C2, point 3), then export again.
- **No "Project P Wallpaper:" line in Output at all:** the add-on isn't enabled (step C3, point 3).
- **The build fails with red Gradle text:** check the Java SDK Path points to JDK 17 (step A7) and that you're online for the first build. If it still fails, copy the first red lines that mention an error.
- **Still two icons:** the old add-on is still in the project, or the old APK is still installed. Redo "Updating from an earlier version" in step C3, export, and install again. `SHOW_SETUP_ICON` in `plugin.gd` must be `false`.
- **Wallpaper Selection shows "The wallpaper setup opens on the phone." on the phone:** the export was made without Gradle Build or without the add-on enabled. Check C2 and C3 and export again. On the computer this message is normal.
- **The wallpaper shows the background but no pet:** it found no frames. Check the Output line says more than 0 frame PNGs and names your pet. If it says "no PNG frames found for pet", the folder in `PETS (pet.gd)` doesn't exist; fix the path there.
- **Output says "could not read PETS":** the add-on can't find your pet script. Set `PET_SCRIPT` in the add-on's `plugin.gd` (step C4).
- **The phone's default wallpaper comes back by itself:** note what you did just before (closed the game, changed the pet, restarted the phone) and which Android version the phone has. The wallpaper now runs in its own process and skips a frame instead of stopping when something goes wrong, so this should no longer happen.
- **The pet never sleeps, or never looks tired:** tap **Check my pet**. `sleep` and `low_energy` must be more than 0; if they are 0, the files have other names than `sleep_..` and `low_energy_..`, so add those names to the two lines in `ANIMS (pet.gd)` and export again. Sleeping follows the phone's clock (11 PM to 6 AM), and low energy needs the phone unplugged at 15% or less.
- **The pet is the wrong size on the home screen:** resize it in the game (pinch it), then go back to the home screen. **Check my pet** shows the size the wallpaper found.
- **The pet doesn't react to shaking or tilting on the lock screen:** it must be hatched and resting. Try a firmer shake, or lower `shakeStrength` and `tiltSideways`.

What to expect on a real phone

- **Hiding when you open apps:** works automatically. Android stops drawing the wallpaper when an app covers it, and the code pauses the animation and the movement sensor so they don't drain the battery.
- **Moving the pet on the home screen:** tap the pet (it glows softly), then tap a spot, and it flies, walks or jumps there (step C7). Swipes and long presses stay with the home screen, so opening the app list or switching pages no longer moves the pet. If the pet sits under an app icon, tapping it also opens that app; keep it on empty space.
- **Lock screen:** the pet appears there in the same spot as on the home screen. Phones don't pass touches to the wallpaper on the lock screen, so there it greets you, reacts to shaking and tilting, and does things by itself (step C8). Some Samsung models don't allow live wallpapers on the lock screen at all.
- **Hatching on the home screen:** as its own app (Part B), the wallpaper keeps its own egg, which hatches when you tap it 3 times. Inside the game's app (Part C), it also hatches when the pet hatches in the game.
- **Size:** inside the game's app (Part C) the wallpaper pet is as big as you made it in the game; resize it there by pinching. It isn't pinched on the home screen, because home screens use that gesture themselves. As its own app (Part B), its size is `drawScale` in `PetWallpaperService.kt`.

- **Night and low battery:** the pet sleeps from 11 PM to 6 AM (tap it on the home screen or shake the phone on the lock screen to wake it for 5 minutes) and does its low-energy movements at 15% battery or less (step C12).
- **Background:** the Project P living-room picture is used unless the user picks their own. They can pick one picture for both screens, or one for the home screen and another for the lock screen. Every picture fills the screen the same way a normal wallpaper does (cropped to fit, never stretched). A GIF, and the animated Project P wallpaper, play on the lock screen and stay still on the home screen.
- **Battery:** the pet's animation only runs while the home or lock screen is visible. A travelling pet and a moving background redraw about 30 times a second, so a moving background costs the most, but only for the seconds the lock screen is on.
- **Parallax:** some launchers slide the wallpaper a little when you swipe between home pages. This background stays still, so it may feel slightly different from a normal wallpaper.

Publishing on Google Play

Everything in this guide is allowed on Google Play: the Godot game, the live wallpaper, tapping and dragging, the built-in background, the photo picker and the shake and tilt reactions. The app asks for no permissions; the movement sensor, the clock and the battery level it uses don't need one. When you get there:

- **Target a recent Android version.** Google raises the required target level every year. Android Studio and Godot show the current setting in the project/export options.
- **Upload an app bundle (.aab), not an APK.** In Godot: with **Use Gradle Build** on (step C2), set **Export Format** to **AAB** in the Android export options. With Part C that single bundle contains the game and the wallpaper.
- **Sign it with your own release key,** not the debug key. Keep the key file and its password safe; you need the same key for every future update.
- **Fill in the Data safety form** in Play Console. For these apps it's simple: the chosen picture, the pet's position and the movement sensor readings stay on the phone and are never sent anywhere.

Next steps

- **A greeting animation:** draw 8 frames of the pet waving or stretching and name them `greet_00.png` ... `greet_07.png`. The lock screen plays them instead of the hop, with no code changes.
- **Evolutions:** growth stages per pet, drawn on the same 128x128 canvas so every stage matches.